

JAVA 17 OCP · YDS STUDY SERIES

Unit 01 · Building Blocks · Eksiksiz Çift Dilli Kaynak Notu

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu düzenlenebilir ana not, kaynak PDF'deki bölümün sayfa sırasını korur. Her anlamlı English kaynak satırı hemen ardından doğal Türkçe karşılığıyla verilir; kod ve terminal komutları ise çevrilmeden yalnızca bir kez gösterilir. Ayrıntılı dil çalışması için [vocabulary](#) ve [grammar notes](#) dosyalarını kullan.

Kaynak ve kapsam özeti

- **Kaynak:** [exam_lecture/OCP_Java_SE17_Chapter1den_Itibaren.pdf](#)
- **PDF kapsamı:** 0001–0064
- **İşlenen sayfa sayısı:** 64
- **İşlenen anlamlı kaynak girdisi:** 2176
- **Çıkarılan öğeler:** Yalnızca tekrarlanan running header, footer ve sayfa numarası
- **OCR düzeltmeleri:** Soft hyphen ve satır bölünmesi kaynaklı tireler yazıma getirildi
- **İzlenebilirlik:** Her kaynak PDF sayfası kaydedilir ve kapsam doğrulaması belgenin sonunda özetlenir

İçindekiler

- 1 [Learning about the Environment](#)
- 2 [Understanding the Class Structure](#)
- 3 [Writing a main\(\) Method](#)
- 4 [Packages, imports and classpath](#)
- 5 [Creating Objects](#)
- 6 [Understanding Data Types](#)
- 7 [Declaring Variables and var](#)
- 8 [Managing Variable Scope](#)
- 9 [Destroying Objects and garbage collection](#)
- 10 [Summary](#)
- 11 [Exam Essentials](#)
- 12 [Review Questions](#)

Kaynak PDF sayfası 1

Chapter

Türkçe başlık: Bölüm

English: 1

Türkçe: 1

Building Blocks**Türkçe başlık:** Yapı Taşları**OCP EXAM OBJECTIVES COVERED IN****Türkçe başlık:** OCP SINAVININ HEDEFLERİ**English:** THIS CHAPTER:**Türkçe:** BU BÖLÜM:**English:** [x] Handling date, time, text, numeric and boolean values**Türkçe:** [x] Tarih, saat, metin, sayısal ve boolean değerlerini işleyin**English:** [x] Use primitives and wrapper classes including Math API, parentheses, type promotion, and casting to evaluate arithmetic and boolean expressions**Türkçe:** [x] Aritmetik ve boolean ifadeleri değerlendirmek için Math API, parantezler, type promotion ve casting dahil temel öğeleri ve wrapper class'ları kullanın**[x] Utilizing Java Object-Oriented Approach****Türkçe başlık:** [x] Java Object-Oriented Yaklaşımını Kullanmak**English:** [x] Declare and instantiate Java objects including nested class objects, and explain the object life-cycle including creation, reassigning references, and garbage collection**Türkçe:** [x] Nested class object'leri dahil Java object'lerini declare edin ve instance oluşturun; ayrıca oluşturma, reference'ları yeniden atama ve garbage collection aşamalarını içeren object life cycle'ını açıklayın.**English:** [x] Understand variable scopes, use local variable type inference, apply encapsulation, and make objects immutable**Türkçe:** [x] Variable scope'larını anlayın, local variable type inference kullanın, encapsulation uygulayın ve object'leri immutable hâle getirin**Kaynak PDF sayfası 2****English:** Welcome to the beginning of your journey to achieve a Java 17 certification. We assume this isn't the first Java programming book you've read. Although we do talk about the basics, we do so only because we want to make sure you have all the terminology and detail you need for the exam. If you've never written a Java program before, we recommend you pick up an introductory book on Java 8 or higher. Examples include *Head First Java, 3rd Edition* (O'Reilly Media, 2022) and *Beginning Programming with Java for Dummies* (For Dummies, 2021). Then come back to this certification study guide.**Türkçe:** Java 17 sertifikasını alma yolculuğunuzun başlangıcına hoş geldiniz. Bunun okuduğunuz ilk Java programlama kitabı olmadığını varsayıyoruz. Temel konulara değiniyoruz; ancak bunu yalnızca sınav için gereken bütün terminolojiye ve ayrıntılara sahip olduğunuzdan emin olmak için yapıyoruz. Daha önce hiç Java programı yazmadıysanız Java 8 veya daha yeni bir sürümü anlatan giriş kitabı edinmenizi öneririz. Bunlara *Head First Java, 3rd Edition* (O'Reilly Media, 2022) ve *Beginning Programming with Java for Dummies* (For Dummies, 2021) örnek verilebilir. Ardından bu sertifika çalışma kılavuzuna geri dönün.

English: As the old saying goes, you have to learn how to walk before you can run. Likewise, you have to learn the basics of Java before you can build complex programs. In this chapter, we present the basics of Java packages, classes, variables, and data types, along with the aspects of each that you need to know for the exam. For example, you might use Java every day but be unaware that you cannot create a variable called `3dMap` or `this`. The exam expects you to know and understand the rules behind these principles. While most of this chapter should be review, there may be aspects of the Java language that are new to you since they don't come up in practical use often.

Türkçe: Eski deyişte belirtildiği gibi, koşmadan önce yürümeyi öğrenmeniz gerekir. Benzer biçimde, karmaşık programlar oluşturmada önce Java'nın temellerini öğrenmelisiniz. Bu bölümde Java package'larının, class'larının, variable'larının ve data type'lerinin temelleri ile bunların sınav için bilmeniz gereken yönlerini sunuyoruz. Örneğin Java'yı her gün kullanıyor olsanız bile `3dMap` veya `this` adında bir variable oluşturamayacağınızı bilmiyor olabilirsiniz. Sınav, bu ilkelerin arkasındaki kuralları bilmenizi ve anlamınızı bekler. Bu bölümün çoğu tekrar niteliğindedir; yine de pratik kullanımda sık karşılaşılmadıkları için Java dilinin bazı yönleri sizin için yeni olabilir.

Learning about the Environment

Türkçe başlık: Çevreyi Öğrenmek

English: The Java environment consists of understanding a number of technologies. In the following sections, we go over the key terms and acronyms you need to know and then discuss what software you need to study for the exam.

Türkçe: Java ortamı bir dizi teknolojinin anlaşılmasından oluşur. Aşağıdaki bölümlerde bilmeniz gereken temel terimleri ve kısaltmaları gözden geçireceğiz ve ardından sınav için hangi yazılımları incelemeniz gerektiğini tartışacağız.

Major Components of Java

Türkçe başlık: Java'nın Ana Bileşenleri

English: The Java Development Kit (JDK) contains the minimum software you need to do Java development. Key commands include:

Türkçe: Java Development Kit (JDK), Java geliştirmek için gereken asgari yazılımı içerir. Temel komutlar şunlardır:

English: - `javac`: Converts `.java` source files into `.class` bytecode

Türkçe: - `javac`: `.java` source file'larını `.class` bytecode'a dönüştürür.

English: - `java`: Executes the program

Türkçe: - `java`: Programı execute eder.

English: - `jar`: Packages files together

Türkçe: - `jar`: File'ları birlikte package'lar.

English: - `javadoc`: Generates documentation

Türkçe: - `javadoc`: Documentation üretir.

English: The javac program generates instructions in a special format called bytecode that the java command can run. Then java launches the Java Virtual Machine (JVM) before

Türkçe: `javac` programı, `java` command'ının çalıştırabildiği bytecode adlı özel formatta instruction'lar üretir. Ardından `java`, kodu çalıştırmadan önce Java Virtual Machine'i (JVM) başlatır.

Kaynak PDF sayfası 3

English: running the code. The JVM knows how to run bytecode on the actual machine it is on. You can think of the JVM as a special magic box on your machine that knows how to run your .class file within your particular operating system and hardware.

Türkçe: JVM daha sonra kodu çalıştırır. Üzerinde bulunduğu fiziksel makinede bytecode'un nasıl yürütüleceğini bilir. JVM'yi, `.class` file'ınızı işletim sisteminiz ve donanımınız üzerinde nasıl çalıştıracakını bilen özel bir kutu gibi düşünebilirsiniz.

Where Did the JRE Go?

Türkçe başlık: JRE nereye gitti?

English: In Java 8 and earlier, you could download a Java Runtime Environment (JRE) instead of the full JDK. The JRE was a subset of the JDK that was used for running a program but could not compile one. Now, people can use the full JDK when running a Java program. Alternatively, developers can supply an executable that contains the required pieces that would have been in the JRE.

Türkçe: Java 8 ve önceki sürümlerde, JDK'nın tamamı yerine bir Java Çalışma Zamanı Ortamı (JRE) indirebilirsiniz. JRE, JDK'nın bir programı çalıştırmak için kullanılan ancak derleyemeyen bir alt kümesiydi. Artık insanlar bir Java programını çalıştırırken JDK'nın tamamını kullanabilirler. Alternatif olarak geliştiriciler, JRE'de bulunması gereken gerekli parçaları içeren bir yürütülebilir dosya sağlayabilirler.

English: When writing a program, there are common pieces of functionality and algorithms that developers need. Luckily, we do not have to write each of these ourselves. Java comes with a large suite of application programming interfaces (APIs) that you can use. For example, there is a `StringBuilder` class to create a large String and a method in `Collections` to sort a list. When writing a program, it is helpful to determine what pieces of your assignment can be accomplished by existing APIs.

Türkçe: Bir program yazarken geliştiricilerin sık kullanılan işlevlere ve algoritmalara ihtiyacı olur. Neyse ki bunların her birini kendimiz yazmak zorunda değiliz. Java, kullanabileceğiniz geniş bir application programming interface (API) kümesiyle gelir. Örneğin büyük bir `String` oluşturmak için `StringBuilder` class'ı, bir listeyi sıralamak için de `Collections` içinde bir method vardır. Program yazarken görevinizin hangi kısımlarını mevcut API'lerle gerçekleştirebileceğinizi belirlemek yararlıdır.

English: You might have noticed that we said the JDK contains the minimum software you need.

Türkçe: JDK'nın ihtiyacınız olan minimum yazılımı içerdiğini söylediğimizi fark etmişsinizdir.

English: Many developers use an integrated development environment (IDE) to make writing and running code easier. While we do not recommend using one while studying for the exam, it is still good to know that they exist. Common Java IDEs include Eclipse, IntelliJ IDEA, and Visual Studio Code.

Türkçe: Çoğu geliştirici, kod yazmayı ve çalıştırmayı kolaylaştırmak için entegre geliştirme ortamı (IDE) kullanır. Sınava çalışırken bunlardan birini kullanmanızı önermesek de bunların var olduğunu bilmek yine de güzel. Yaygın Java IDE'leri arasında Eclipse, IntelliJ IDEA ve Visual Studio Code bulunur.

Downloading a JDK

Türkçe başlık: JDK'yi indirme

English: Every six months, Oracle releases a new version of Java. Java 17 came out in September 2021. This means that Java 17 will not be the latest version when you download the JDK to study for the exam. However, you should still use Java 17 to study with since this is a Java 17 exam. The rules and behavior can change with later versions of Java. You wouldn't want to get a question wrong because you studied with a different version of Java!

Türkçe: Oracle her altı ayda bir yeni bir Java sürümü yayımlar. Java 17, Eylül 2021'de çıktı. Bu nedenle sınava çalışmak için JDK'yi indirdiğinizde Java 17 en yeni sürüm olmayacaktır. Yine de bu bir Java 17 sınavı olduğu için çalışmalarınızda Java 17 kullanmalısınız. Kurallar ve davranışlar Java'nın sonraki sürümlerinde değişebilir. Farklı bir Java sürümüyle çalıştığınız için bir soruyu yanlış cevaplamak istemezsiniz!

English: You can download Oracle's JDK on the Oracle website, using the same account you use to register for the exam. There are many JDKs available, the most popular of which, besides Oracle's JDK, is OpenJDK.

Türkçe: Sınava kaydolmak için kullandığınız hesabı kullanarak Oracle'ın JDK'sını Oracle web sitesinden indirebilirsiniz. Pek çok JDK mevcut olup, Oracle'ın JDK'sının yanı sıra en popüler olanı OpenJDK'dir.

English: Many versions of Java include preview features that are off by default but that you can enable. Preview features are not on the exam. To avoid confusion about when a feature was added to the language, we will say "was officially introduced in" to denote when it was moved out of preview.

Türkçe: Java'nın pek çok sürümü, varsayılan olarak kapalı olan ancak etkinleştirebileceğiniz önizleme özellikleri içerir. Önizleme özellikleri sınavda bulunmamaktadır. Bir özelliğin dile ne zaman eklendiğiyle ilgili karışıklığı önlemek için, özelliğin önizlemeden çıkarıldığı zamanı belirtmek üzere "resmi olarak tanıtıldı" diyeceğiz.

Kaynak PDF sayfası 4

Check Your Version of Java

Türkçe başlık: Java Sürümünüzü Kontrol Edin

English: Before we go any further, please take this opportunity to ensure that you have the right version of Java on your path.

Türkçe: Daha ileri gitmeden önce **PATH** environment variable'ınızda doğru Java sürümünün bulunduğundan emin olun.

```
javac -version
java -version
```

English: Both of these commands should include version number 17.

Türkçe: Bu komutların her ikisi de 17 sürüm numarasını içermelidir.

Understanding the Class Structure

Türkçe başlık: class Yapısını Anlamak

English: In Java programs, classes are the basic building blocks. When defining a class, you describe all the parts and characteristics of one of those building blocks. In later chapters, you see other building blocks such as interfaces, records, and enums.

Türkçe: Java programlarında class'lar temel yapı taşlarıdır. Bir class'ı tanımlarken bu yapı taşlarından birinin bütün parçalarını ve özelliklerini açıklarsınız. Sonraki bölümlerde interface, record ve `enum` (numaralandırma) gibi başka yapı taşlarını göreceksiniz.

English: To use most classes, you have to create objects. An object is a runtime instance of a class in memory. An object is often referred to as an instance since it represents a single representation of the class. All the various objects of all the different classes represent the state of your program. A reference is a variable that points to an object.

Türkçe: Çoğu class'ı kullanmak için object'ler oluşturmanız gerekir. Bir object, bellekteki bir class'ın runtime instance'ıdır. Class'ın tek bir temsilini oluşturduğu için object'e çoğu zaman instance da denir. Farklı class'lara ait bütün object'ler programınızın state'ini temsil eder. Reference ise bir object'e işaret eden variable'dır.

English: In the following sections, we look at fields, methods, and comments. We also explore the relationship between classes and files.

Türkçe: Aşağıdaki bölümlerde field'ları, method'ları ve comment'leri inceleyeceğiz. Ayrıca class'lar ile file'lar arasındaki ilişkiye bakacağız.

Fields and Methods

Türkçe başlık: Field'lar ve Method'lar

English: Java classes have two primary elements: methods, often called functions or procedures in other languages, and fields, more generally known as variables. Together these are called the members of the class. Variables hold the state of the program, and methods operate on that state. If the change is important to remember, a variable stores that change. That's all classes really do. It's the programmer's job to create and arrange these elements in such a way that the resulting code is useful and, ideally, easy for other programmers to understand.

Türkçe: Java class'larının iki temel ögesi vardır: Diğer dillerde function veya procedure denebilen method'lar ve daha genel anlamda variable olan field'lar. Bunlar birlikte class member'larını oluşturur. Variable'lar programın state'ini saklar; method'lar bu state üzerinde işlem yapar. Bir değişikliğin korunması gerekiyorsa onu variable saklar. Programcının görevi, bu öğeleri kullanışlı ve başka geliştiricilerin anlayabileceği bir kod oluşturacak şekilde düzenlemektir.

English: The simplest Java class you can write looks like this:

Türkçe: Yazabileceğiniz en basit Java class'ı şuna benzer:

```
1: public class Animal {  
2: }
```

English: Java calls a word with special meaning a keyword, which we've marked bold in the previous snippet. Throughout the book, we often bold parts of code snippets to call attention to them. Line 1 includes the public keyword, which allows other classes to use it. The class keyword indicates you're defining a class. Animal gives the name of the class.

Türkçe: Java, özel anlam taşıyan bir sözcüğe keyword der; önceki code snippet'ta bunu kalın gösterdik. Kitap boyunca dikkat çekmek istediğimiz code snippet bölümlerini sık sık kalın yazacağız. 1. satırdaki **public** keyword'ü diğer class'ların bu class'ı kullanabilmesini sağlar. **class** keyword'ü bir class tanımlandığını belirtir; **Animal** ise class'ın adıdır.

English: Granted, this isn't an interesting class, so let's add your first field.

Türkçe: Bunun pek ilginç bir class olmadığı açık; o hâlde ilk field'ımızı ekleyelim.

```
1: public class Animal {  
2: String name;  
3: }
```

Kaynak PDF sayfası 5

English: The line numbers aren't part of the program; they're just there to make the code easier to talk about.

Türkçe: Satır numaraları programın parçası değildir; yalnızca kodu açıklamayı kolaylaştırır.

English: On line 2, we define a variable named name. We also declare the type of that variable to be String. A String is a value that we can put text into, such as "this is a string".

Türkçe: 2. satırda **name** adlı bir variable tanımlar ve type'ını **String** olarak declare ederiz. **String**, kaynak örnekteki exact literal olan **"this is a string"** ("bu bir metindir") gibi text saklayabilen bir value'dur.

English: String is also a class supplied with Java. Next we can add methods.

Türkçe: **String**, Java ile birlikte sunulan bir class'tır. Sırada method'ları ekleyebiliriz.

```
1: public class Animal {  
2: String name;  
3: public String getName() {  
4: return name;  
5: }  
6: public void setName(String newName) {  
7: name = newName;  
8: }  
9: }
```

English: On lines 3–5, we define a method. A method is an operation that can be called. Again, `public` is used to signify that this method may be called from other classes. Next comes the return type— in this case, the method returns a `String`. On lines 6–8 is another method.

Türkçe: 3–5. satırlarda bir method tanımlarız. Method, çağrılabilen bir operation'dır. `public`, bu method'un başka class'lar tarafından çağrılabilceğini belirtir. Ardından return type gelir; bu örnekte method bir `String` döndürür. 6–8. satırlarda ikinci bir method vardır.

English: This one has a special return type called `void`. The `void` keyword means that no value at all is returned. This method requires that information be supplied to it from the calling method; this information is called a parameter. The `setName()` method has one parameter named `newName`, and it is of type `String`. This means the caller should pass in one `String` parameter and expect nothing to be returned.

Türkçe: Bu method'un `void` adı verilen özel bir return type'ı vardır. `void` keyword'ü hiçbir value döndürülmediğini belirtir. Method, calling method tarafından kendisine bilgi verilmesini gerektirir; bu bilgiye parameter denir. `setName()` method'unun `String` type'ında, `newName` adlı bir parameter'ı vardır. Dolayısıyla caller bir `String` argument geçirmeli ve return value beklememelidir.

English: The method name and parameter types are called the method signature. In this example, can you identify the method name and parameters?

Türkçe: Method adına ve parameter type'larına method signature denir. Bu örnekte method adını ve parameter'ları belirleyebilir misiniz?

```
public int numberVisitors(int month) {
    return 10;
}
```

English: The method name is `numberVisitors`. There's one parameter named `month`, which is of type `int`, which is a numeric type. Therefore, the method signature is `numberVisitors(int)`.

Türkçe: Method adı `numberVisitors`dır. Numeric type olan `int` type'ında, `month` adlı tek bir parameter vardır. Bu nedenle method signature `numberVisitors(int)` olur.

Comments

Türkçe başlık: Yorumlar

English: Another common part of the code is called a comment. Because comments aren't executable code, you can place them in many places. Comments can make your code easier to read.

Türkçe: Kodun diğer bir ortak kısmına yorum adı verilir. Yorumlar çalıştırılabilir kod olmadığından onları birçok yere yerleştirebilirsiniz. Yorumlar kodunuzun okunmasını kolaylaştırabilir.

English: While the exam creators are trying to make the code harder to read, they still use comments to call attention to line numbers. We hope you use comments in your own code. There are three types of comments in Java. The first is called a single-line comment:

Türkçe: Sınavı oluşturanlar kodun okunmasını zorlaştırmaya çalışırken satır numaralarına dikkat çekmek için yorumları kullanmaya devam ediyorlar. Yorumları kendi kodunuzda kullanmanızı umuyoruz. Java'da üç tür yorum vardır. İlkine tek satırlık yorum denir:

```
// comment until end of line
```

Kaynak PDF sayfası 6

English: A single-line comment begins with two slashes. The compiler ignores anything you type after that on the same line. Next comes the multiple-line comment:

Türkçe: Single-line comment iki eğik çizgiyle (//) başlar. Compiler aynı satırda bunlardan sonra gelen her şeyi yok sayar. Sırada multiline comment vardır:

```
/* Multiple
 * line comment
 */
```

English: A multiple-line comment (also known as a multiline comment) includes anything starting from the symbol `/*` until the symbol `*/`. People often type an asterisk (*) at the beginning of each line of a multiline comment to make it easier to read, but you don't have to. Finally, we have a Javadoc comment:

Türkçe: Multiline comment (block comment olarak da bilinir), `/*` ile `*/` arasındaki her şeyi kapsar. Okumayı kolaylaştırmak için her satırın başına çoğunlukla yıldız (*) konur, ancak bu zorunlu değildir. Son olarak Javadoc comment vardır:

```
/**
 * Javadoc multiple-line comment
 * @author Jeanne and Scott
 */
```

English: This comment is similar to a multiline comment, except it starts with `/**`. This special syntax tells the Javadoc tool to pay attention to the comment. Javadoc comments have a specific structure that the Javadoc tool knows how to read. You probably won't see a Javadoc comment on the exam. Just remember it exists so you can read up on it online when you start writing programs for others to use.

Türkçe: Bu comment, `/**` ile başlaması dışında multiline comment'e benzer. Bu özel syntax, `javadoc` aracına comment'i işlemesini söyler. Javadoc comment'leri aracın okuyabildiği belirli bir yapıya sahiptir. Sınavda büyük olasılıkla Javadoc comment görmezsiniz; ancak başkalarının kullanacağı programlar yazarken çevrim içi dokümantasyon üretmek için kullandığını bilin.

English: As a bit of practice, can you identify which type of comment each of the following six words is in? Is it a single-line or a multiline comment?

Türkçe: Biraz pratik yapmak için, aşağıdaki altı kelimenin her birinin hangi tür yorumda yer aldığını belirleyebilir misiniz? Tek satırlı mı yoksa çok satırlı bir yorum mu?

```
/*
 * // anteater
 */
// bear
// // cat
// /* dog */
/* elephant */
/*
 * /* ferret */
 */
```

English: Did you look closely? Some of these are tricky. Even though comments technically aren't on the exam, it is good to practice looking at code carefully.

Türkçe: Yakından baktın mı? Bunlardan bazıları çetrefilli. Her ne kadar teknik olarak sınavda yorumlar yer almasa da kodlara dikkatli bakma konusunda pratik yapmakta fayda var.

Kaynak PDF sayfası 7

English: Okay, on to the answers. The comment containing anteater is in a multiline comment.

Türkçe: Cevaplara geçelim. `anteater` içeren comment, multiline comment içindedir.

English: Everything between `/*` and `*/` is part of a multiline comment— even if it includes a single-line comment within it! The comment containing bear is your basic single-line comment. The comments containing cat and dog are also single-line comments. Everything from `//` to the end of the line is part of the comment, even if it is another type of comment. The comment containing elephant is your basic multiline comment, even though it only takes up one line.

Türkçe: `/*` ile `*/` arasındaki her şey, içinde single-line comment bulunsa bile multiline comment'in parçasıdır. `bear`, temel bir single-line comment içindedir. `cat` ve `dog` içeren comment'ler de single-line'dır; `//` ile satır sonu arasındaki her şey comment sayılır. `elephant` ise tek satır kaplasa da multiline comment içindedir.

English: The line with ferret is interesting in that it doesn't compile. Everything from the first `/*` to the first `*/` is part of the comment, which means the compiler sees something like this:

Türkçe: `ferret` bulunan satır derlenmediği için ilginçtir. İlk `/*` ile ilk `*/` arası comment'tir; dolayısıyla compiler yaklaşık olarak şunu görür:

```
/* */ */
```

English: We have a problem. There is an extra `*/`. That's not valid syntax— a fact the compiler is happy to inform you about.

Türkçe: Burada bir sorun vardır: fazladan bir `*/` bulunur. Bu geçerli syntax değildir ve compiler hata verir.

Classes and Source Files

Türkçe başlık: Class'lar ve Kaynak Dosyaları

English: Most of the time, each Java class is defined in its own .java file. In this chapter, the only top-level type is a class. A top-level type is a data structure that can be defined independently within a source file. For the majority of the book, we work with classes as the top-level type, but in Chapter 7, "Beyond Classes," we present other top-level types, as well as nested types.

Türkçe: Çoğu zaman her Java class'ı kendi `.java` file'ında tanımlanır. Bu bölümdeki tek top-level type class'tır. Top-level type, bir source file içinde bağımsız olarak tanımlanabilen data structure'dır. Kitabın büyük bölümünde top-level type olarak class'larla çalışacağız; Bölüm 7, "Beyond Classes" bölümünde ise nested type'ların yanı sıra diğer top-level type'ları da tanıtacağız.

English: A top-level class is often public, which means any code can call it. Interestingly, Java does not require that the type be public. For example, this class is just fine:

Türkçe: Top-level class çoğunlukla `public`dir (herkese açıktır); yani herhangi bir kod onu çağırabilir. Bununla birlikte Java, bir type'ın mutlaka `public` olmasını istemez. Örneğin aşağıdaki class geçerlidir:

```
1: class Animal {  
2: String name;  
3: }
```

English: You can even put two types in the same file. When you do so, at most one of the top-level types in the file is allowed to be public. That means a file containing the following is also fine:

Türkçe: Aynı file'a iki type da koyabilirsiniz. Bu durumda file'daki top-level type'lardan en fazla biri `public` (herkese açık) olabilir. Dolayısıyla aşağıdaki içeriğe sahip bir file da geçerlidir:

```
1: public class Animal {  
2: private String name;  
3: }  
4: class Animal2 {}
```

English: If you do have a public type, it needs to match the filename. The declaration `public class Animal2` would not compile in a file named `Animal.java`. In Chapter 5, "Methods," we discuss what access options are available other than public.

Türkçe: `public` bir top-level type varsa adı file adıyla eşleşmelidir. `Animal.java` adlı file içindeki `public class Animal2` declaration'ı derlenmez. Bölüm 5, "Method'lar"da `public` dışında hangi access seçeneklerinin bulunduğu ele alınır.

English: Noticing a pattern yet? This chapter includes numerous references to topics that we go into in more detail in later chapters. If you're an experienced Java developer, you'll notice we keep a lot of the examples and rules simple in this chapter. Don't worry; we have the rest of the book to present more rules and complicated edge cases!

Türkçe: Bir örüntü fark ettiniz mi? Bu bölüm, sonraki bölümlerde daha ayrıntılı ele alınacak konulara sık sık gönderme yapar. Deneyimli bir Java geliştiricisiyseniz buradaki örnek ve kuralların çoğunun basit tutulduğunu fark edeceksiniz. Endişelenmeyin; kitabın devamında daha çok kural ve karmaşık sınır durumuyla karşılaşacaksınız.

Kaynak PDF sayfası 8

Writing a main() Method

Türkçe başlık: Bir `main()` Method'u Yazmak

English: A Java program begins execution with its `main()` method. In this section, you learn how to create one, pass a parameter, and run a program. The `main()` method is often called an entry point into the program, because it is the starting point that the JVM looks for when it begins running a new program.

Türkçe: Bir Java programı `main()` method'uyla yürütülmeye başlar. Bu bölümde bir `main()` method'unun nasıl oluşturulacağını, ona nasıl parameter geçirileceğini ve programın nasıl çalıştırılacağını öğreneceksiniz. JVM yeni bir programı başlatırken ilk olarak bu noktayı aradığı için `main()` method'una programın entry point'i de denir.

Creating a main() Method

Türkçe başlık: Bir `main()` Method'u Oluşturmak

English: The `main()` method lets the JVM call our code. The simplest possible class with a `main()` method looks like this:

Türkçe: `main()` method'u JVM'nin kodumuzu çağırmasını sağlar. `main()` method'u içeren mümkün olan en basit class şöyledir:

```
1: public class Zoo {  
2: public static void main(String[] args) {  
3: System.out.println("Hello World");  
4: }  
5: }
```

English: This code prints Hello World. To compile and execute this code, type it into a file called `Zoo.java` and execute the following:

Türkçe: Bu kod `Hello World` yazdırır. Kodu derlemek ve çalıştırmak için onu `Zoo.java` adlı bir file'a yazın ve aşağıdaki komutları çalıştırın:

```
javac Zoo.java  
java Zoo
```

English: If it prints Hello World, you were successful. If you do get error messages, check that you've installed the Java 17 JDK, that you have added it to the PATH, and that you didn't make any typos in the example. If you have any of these problems and don't know what to do, post a question with the error message you received in the Beginning Java forum at CodeRanch:

Türkçe: `Hello World` çıktısını görürseniz başarılı oldunuz. Hata alırsanız Java 17 JDK'nın kurulu ve PATH üzerinde olduğundan, ayrıca örneği doğru yazdığınızdan emin olun. Sorun devam ederse aldığınız hata mesajıyla birlikte CodeRanch'in Beginning Java forumunda soru sorabilirsiniz:

English: www.coderanch.com/forums/f-33/java

Türkçe: www.coderanch.com/forums/f-33/java

English: To compile Java code with the `javac` command, the file must have the extension `.java`.

Türkçe: Java kodunu `javac` command'ıyla derleyebilmek için file'ın `.java` uzantısına sahip olması gerekir.

English: The name of the file must match the name of the public class. The result is a file of bytecode with the same name but with a `.class` filename extension. Remember that bytecode consists of instructions that the JVM knows how to execute. Notice that we must omit the `.class` extension to run `Zoo.class`.

Türkçe: File adı `public` class adıyla eşleşmelidir. Sonuç, aynı ada ve `.class` uzantısına sahip bir bytecode file'ıdır. Bytecode'un JVM'nin execute etmeyi bildiği instruction'lardan oluştuğunu unutmayın. `Zoo.class` çalıştırırken `.class` uzantısını yazmamamız gerektiğine dikkat edin.

English: The rules for what a Java file contains, and in what order, are more detailed than what we have explained so far (there is more on this topic later in the chapter). To keep things simple for now, we follow this subset of the rules:

Türkçe: Bir Java dosyasının neleri ve hangi sırayla içerdiğine ilişkin kurallar şu ana kadar açıkladıklarımızdan daha ayrıntılıdır (bölümün ilerleyen kısımlarında bu konu hakkında daha fazla bilgi bulunmaktadır). Şimdilik işleri basit tutmak için kuralların şu alt kümesini uyguluyoruz:

English: Each file can contain only one public class.

Türkçe: Her file yalnızca bir `public` class içerebilir.

English: The filename must match the class name, including case, and have a .java extension.

Türkçe: File adı, büyük/küçük harf kullanımı da dahil olmak üzere class adıyla eşleşmeli ve `.java` uzantısına sahip olmalıdır.

English: If the Java class is an entry point for the program, it must contain a valid `main()` method.

Türkçe: Java class'ı programın entry point'i ise geçerli bir `main()` method'u içermelidir.

English: Let's first review the words in the `main()` method's signature, one at a time. The keyword `public` is what's called an access modifier. It declares this method's level of exposure to potential callers in the program. Naturally, `public` means full access from anywhere in the program. You learn more about access modifiers in Chapter 5.

Türkçe: Önce `main()` method signature'ındaki sözcükleri tek tek inceleyelim. `public` keyword'ü bir access modifier'dır ve method'un programdaki olası caller'lara ne ölçüde açık olduğunu belirtir. `public`, programın her yerinden tam erişim anlamına gelir. Access modifier'lar Chapter 5'te ayrıntılandırılır.

Kaynak PDF sayfası 9

English: The keyword `static` binds a method to its class so it can be called by just the class name, as in, for example, `Zoo.main()`. Java doesn't need to create an object to call the `main()` method— which is good since you haven't learned about creating objects yet! In fact, the JVM does this, more or less, when loading the class name given to it. If a `main()` method doesn't have the right keywords, you'll get an error trying to run it. You see `static` again in Chapter 6, "Class Design."

Türkçe: `static` keyword'ü bir method'u class'a bağlar; böylece method `Zoo.main()` örneğinde olduğu gibi yalnızca class adıyla çağrılabilir. Java'nın `main()` method'unu çağırmak için object oluşturması gerekmez. JVM, kendisine verilen class adını yüklerken esasen bunu yapar. `main()` method'u gerekli keyword'lere sahip değilse programı çalıştırırken error alırsınız. `static` keyword'ünü Bölüm 6, "Class Design" içinde yeniden göreceksiniz.

English: The keyword `void` represents the return type. A method that returns no data returns control to the caller silently. In general, it's good practice to use `void` for methods that change an object's state. In that sense, the `main()` method changes the program state from started to finished. We explore return types in Chapter 5 as well. (Are you excited for Chapter 5 yet?) Finally, we arrive at the `main()` method's parameter list, represented as an array of `java.lang.String` objects. You can use any valid variable name along with any of these three formats:

Türkçe: `void` keyword'ü return type'ı belirtir. Veri döndürmeyen method, control'ü caller'a sessizce geri verir. Bir object'in state'ini değiştiren method'larda `void` kullanmak genellikle iyi bir pratiktir; bu bakımdan `main()` program state'ini başlamış durumdan tamamlanmış duruma getirir. Return type'lar Chapter 5'te ele alınır. Son olarak `main()` method'unun `java.lang.String` object'lerinden oluşan array biçimindeki parameter listesine geliriz. Aşağıdaki üç formatta herhangi bir geçerli variable adı kullanılabilir:

```
String[] args
```

```
String options[]
String... friends
```

English: The compiler accepts any of these. The variable name `args` is common because it hints that this list contains values that were read in (arguments) when the JVM started. The characters `[]` are brackets and represent an array. An array is a fixed-size list of items that are all of the same type. The characters `...` are called `varargs` (variable argument lists). You learn about `String` in this chapter. Arrays are in Chapter 4, "Core APIs," and `varargs` are in Chapter 5.

Türkçe: Compiler bu biçimlerin üçünü de kabul eder. `args` yaygın bir variable adıdır; JVM başlatılırken okunan argument'ları içerdiğini ima eder. `[]` karakterleri köşeli parantezdir ve bir array'i gösterir. Array, aynı type'taki elemanların sabit boyutlu listesidir. `...` karakterleri `varargs` (variable argument listesi) olarak adlandırılır. Bu bölümde `String`, Bölüm 4 "Core APIs"de array'ler ve Bölüm 5'te `varargs` ayrıntılı olarak ele alınır.

Optional Modifiers in `main()` Methods

Türkçe başlık: `main()` Method'larında Optional Modifier'lar

English: While most modifiers, such as `public` and `static`, are required for `main()` methods, there are some optional modifiers allowed.

Türkçe: `public` ve `static` gibi modifier'lar `main()` method'ları için zorunlu olsa da bazı optional modifier'lara izin verilir.

```
public final static void main(final String[] args) {}
```

English: In this example, both `final` modifiers are optional, and the `main()` method is a valid entry point with or without them. We cover the meaning of `final` methods and parameters in Chapter 6.

Türkçe: Bu örnekte iki `final` modifier da optional'dır; `main()` method'u bunlar bulunsun da bulunmasa da geçerli bir entry point'tir. `final` method ve parameter'ların anlamı Bölüm 6'da ele alınacaktır.

Passing Parameters to a Java Program

Türkçe başlık: Parametreleri Java Programına Aktarmak

English: Let's see how to send data to our program's `main()` method. First, we modify the Zoo program to print out the first two arguments passed in:

Türkçe: Programımızın `main()` method'una nasıl veri gönderildiğine bakalım. Önce Zoo programını, geçirilen ilk iki argument'ı yazdıracak şekilde değiştiriyoruz:

```
public class Zoo {  
    public static void main(String[] args) {
```

Kaynak PDF sayfası 10

```
        System.out.println(args[0]);  
        System.out.println(args[1]);  
    }  
}
```

English: The code `args[0]` accesses the first element of the array. That's right: array indexes begin with 0 in Java. To run it, type this:

Türkçe: `args[0]` kodu dizinin ilk öğesine erişir. Doğru: Java'da dizi indeksleri 0 ile başlar. Çalıştırmak için şunu yazın:

```
javac Zoo.java  
java Zoo Bronx Zoo
```

English: The output is what you might expect:

Türkçe: Çıktı beklediğiniz şeydir:

```
Bronx  
Zoo
```

English: The program correctly identifies the first two “words” as the arguments. Spaces are used to separate the arguments. If you want spaces inside an argument, you need to use quotes as in this example:

Türkçe: Program ilk iki “sözcüğü” ayrı argument olarak doğru biçimde tanır. Argument'lar boşlukla ayrılır. Bir argument içinde boşluk bulunmasını istiyorsanız aşağıdaki gibi tırnak işaretleri kullanmalısınız:

```
javac Zoo.java  
java Zoo "San Diego" Zoo
```

English: Now we have a space in the output:

Türkçe: Bu kez çıktı boşluk içeren tek argument'ı gösterir:

```
San Diego  
Zoo
```

English: Finally, what happens if you don't pass in enough arguments?

Türkçe: Son olarak, yeterli sayıda argument vermezseniz ne olur?

```
javac Zoo.java
java Zoo Zoo
```

English: Reading `args[0]` goes fine, and `Zoo` is printed out. Then Java panics. There's no second argument! What to do? Java prints out an exception telling you it has no idea what to do with this argument at position 1. (You learn about exceptions in Chapter 11, "Exceptions and Localization.")

Türkçe: `args[0]` okunur ve `Zoo` yazdırılır. Ardından ikinci argument bulunmadığı için Java exception fırlatır. Mesaj, index 1 konumunun array sınırları dışında olduğunu belirtir. Exception'lar Bölüm 11, "Exceptions and Localization" içinde ele alınır.

```
Zoo
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
Index 1 out of bounds for length 1
    at Zoo.main(Zoo.java:4)
```

English: To review, the JDK contains a compiler. Java class files run on the JVM and therefore run on any machine with Java rather than just the machine or operating system they happened to have been compiled on.

Türkçe: Tekrar etmek gerekirse JDK bir compiler içerir. Java class file'ları JVM üzerinde çalıştığı için yalnızca derlendikleri makine veya operating system'de değil, Java bulunan her makinede çalışabilir.

Kaynak PDF sayfası 11

Single-File Source-Code

Türkçe başlık: Tek Dosya Kaynak Kodu

English: If you get tired of typing both `javac` and `java` every time you want to try a code example, there's a shortcut. You can instead run

Türkçe: Her code example denemesinde hem `javac` hem de `java` yazmaktan sıkıldıysanız bir shortcut vardır. Bunun yerine şu command'ı çalıştırabilirsiniz:

```
java Zoo.java Bronx Zoo
```

English: There is a key difference here. When compiling first, you omitted the `.java` extension when running `java`. When skipping the explicit compilation step, you include this extension. This feature is called launching single-file source-code programs and is useful for testing or for small programs. The name cleverly tells you that it is designed for when your program is one file.

Türkçe: Burada önemli bir fark var. İlk önce derlerken, Java'yı çalıştırırken `.java` uzantısını çıkardınız. Açık derleme adımını atlarken bu uzantıyı dahil edersiniz. Bu özelliğe tek dosya kaynak kodlu programların başlatılması adı verilir ve test etme veya küçük programlar için kullanışlıdır. Bu ad, programınızın tek bir dosya olduğu durumlar için tasarlandığını akıllıca söyler.

Understanding Package Declarations and Imports

Türkçe başlık: Package Declaration'larını ve Import'ları Anlamak

English: Java comes with thousands of built-in classes, and there are countless more from developers like you. With all those classes, Java needs a way to organize them. It handles this in a way similar to a file cabinet. You put all your pieces of paper in folders. Java puts classes in packages. These are logical groupings for classes.

Türkçe: Java binlerce built-in class ile gelir; geliştiricilerin yazdığı sayısız class da vardır. Java bunları düzenlemek için bir file cabinet benzetmesine başvurur: Kâğıtları klasörlere koyduğunuz gibi Java da class'ları package'lara koyar. Package'lar class'lar için logical grouping sağlar.

English: We wouldn't put you in front of a file cabinet and tell you to find a specific paper. Instead, we'd tell you which folder to look in. Java works the same way. It needs you to tell it which packages to look in to find code.

Türkçe: Sizi bir dosya dolabının önüne koyup belirli bir kâğıt bulmanızı söylemeyiz. Bunun yerine size hangi klasöre bakmanız gerektiğini söyledik. Java da aynı şekilde çalışır. Kodu bulmak için hangi paketlere bakılacağını söylemeniz gerekiyor.

English: Suppose you try to compile this code:

Türkçe: Bu kodu derlemeye çalıştığınızı varsayalım:

```
public class NumberPicker {  
    public static void main(String[] args) {  
        Random r = new Random(); // DOES NOT COMPILE  
        System.out.println(r.nextInt(10));  
    }  
}
```

English: The Java compiler helpfully gives you an error that looks like this:

Türkçe: Java derleyicisi size aşağıdaki gibi görünen bir hata verir:

```
error: cannot find symbol
```

English: This error could mean you made a typo in the name of the class. You double-check and discover that you didn't. The other cause of this error is omitting a needed import statement.

Türkçe: Bu error, class adında typo yaptığınız anlamına gelebilir. Yeniden kontrol eder ve typo olmadığını görürsünüz. Bu error'ın diğer nedeni, gerekli bir import statement'ın eksik olmasıdır.

English: A statement is an instruction, and import statements tell Java which packages to look in for classes. Since you didn't tell Java where to look for Random, it has no clue.

Türkçe: Deyim bir talimattır ve import ifadeleri Java'ya class'lar için hangi paketlere bakacağını söyler. Java'ya Random'ı nerede arayacağını söylemediğiniz için hiçbir ipucu yok.

English: Trying this again with the import allows the code to compile.

Türkçe: Bunu içe aktarmayla tekrar denemek, kodun derlenmesine olanak tanır.

Kaynak PDF sayfası 12

```
import java.util.Random; // import tells us where to find Random
public class NumberPicker {
    public static void main(String[] args) {
        Random r = new Random();
        System.out.println(r.nextInt(10)); // a number 0-9
    }
}
```

English: Now the code runs; it prints out a random number between 0 and 9. Just like arrays, Java likes to begin counting with 0.

Türkçe: Artık kod çalışıyor; 0 ile 9 arasında rastgele bir sayı yazdırır. Tıpkı diziler gibi, Java da saymaya 0 ile başlamayı sever.

English: In Chapter 5, we cover another type of import referred to as a static import. It allows you to make static members of a class known, often so you can use variables and method names without having to keep specifying the class name.

Türkçe: Bölüm 5'te static import adı verilen başka bir import türünü ele alacağız. Static import, bir class'ın static member'larını görünür hâle getirir; böylece çoğunlukla class adını yazmadan variable ve method adlarını kullanabilirsiniz.

Packages

Türkçe başlık: Paketler

English: As you saw in the previous example, Java classes are grouped into packages. The import statement tells the compiler which package to look in to find a class. This is similar to how mailing a letter works. Imagine you are mailing a letter to 123 Main Street, Apartment 9.

Türkçe: Önceki örnekte gördüğünüz gibi Java class'ları package'lar halinde gruplandırılmıştır. Import deyimi, compiler'a bir class'ı bulmak için hangi pakete bakacağını söyler. Bu, mektup göndermenin işleyişine benzer. 123 Ana Cadde, Daire 9 adresine bir mektup gönderdiğiniz hayal edin.

English: The mail carrier first brings the letter to 123 Main Street. Then the carrier looks for the mailbox for apartment number 9. The address is like the package name in Java.

Türkçe: Postacı mektubu önce 123 Main Street'e getirir. Ardından 9 numaralı dairenin posta kutusunu arar. Adres, Java'daki package adına benzer.

English: The apartment number is like the class name in Java. Just as the mail carrier only looks at apartment numbers in the building, Java only looks for class names in the package.

Türkçe: Daire numarası Java'daki class adına benzer. Postacının yalnızca ilgili binadaki daire numaralarına bakması gibi, Java da yalnızca ilgili package içindeki class adlarını arar.

English: Package names are hierarchical like the mail as well. The postal service starts with the top level, looking at your country first. You start reading a package name at the beginning too. For example, if it begins with java, this means it came with the JDK. If it starts with something else, it likely shows where it came from using the website name in reverse. For example, com.wiley.javabook tells us the code is associated with the wiley.com website or organization. After the website name, you can add whatever you want. For example, com.wiley.java.my.name also came from wiley.com. Java calls more detailed packages child packages. The package com.wiley.javabook is a child package of com.wiley. You can tell because it's longer and thus more specific.

Türkçe: Package adları da posta adresleri gibi hiyerarşiktir. Posta hizmeti en üst düzeyden başlar; önce ülkeye bakar. Bir package adı da başından itibaren okunur. Örneğin ad `java` ile başlıyorsa package JDK ile birlikte gelmiştir. Başka bir ifadeyle başlıyorsa kaynağını çoğunlukla ters çevrilmiş bir web sitesi adıyla gösterir. Örneğin `com.wiley.javabook`, kodun `wiley.com` sitesi veya kuruluşuyla ilişkili olduğunu belirtir. Site adından sonra istenen bölümler eklenebilir; `com.wiley.java.my.name` de `wiley.com` ile ilişkilidir. Java, daha ayrıntılı package'lara child package (alt package) der. `com.wiley.javabook`, `com.wiley` package'ının child package'ıdır; daha uzun olduğu için daha özeldir.

English: You'll see package names on the exam that don't follow this convention. Don't be surprised to see package names like a.b.c. The rule for package names is that they are mostly letters or numbers separated by periods (.). Technically, you're allowed a couple of other characters between the periods (.). You can even use package names of websites you don't own if you want to, such as com.wiley, although people reading your code might be confused! The rules are the same as for variable names, which you see later in this chapter. The exam may try to trick you with invalid variable names. Luckily, it doesn't try to trick you by giving invalid package names.

Türkçe: Sınavda bu kurala uymayan package adlarını göreceksiniz. a.b.c. gibi package adlarını gördüğünüzde şaşmayın. Paket adları için kural, çoğunlukla nokta (.) ile ayrılmış harf veya rakamlardan oluşmasıdır. Teknik olarak noktalar (.) arasında birkaç karakter daha kullanmanıza izin verilir. İsterseniz com.wiley gibi size ait olmayan web sitelerinin package adlarını bile kullanabilirsiniz, ancak kodunuzu okuyan kişilerin kafası karışabilir! Kurallar, bu bölümün ilerleyen kısımlarında göreceğiniz variable adlarıyla aynıdır. Sınav sizi geçersiz variable adlarıyla kandırmaya çalışabilir. Neyse ki geçersiz package adları vererek sizi kandırmaya çalışmıyor.

Kaynak PDF sayfası 13

English: In the following sections, we look at imports with wildcards, naming conflicts with imports, how to create a package of your own, and how the exam formats code.

Türkçe: Aşağıdaki bölümlerde joker karakterlerle içe aktarmaya, içe aktarmalarla adlandırma çakışmalarına, kendi paketinizi nasıl oluşturacağınıza ve sınavın kodu nasıl biçimlendirdiğine bakacağız.

Wildcards

Türkçe başlık: Joker karakterler

English: Classes in the same package are often imported together. You can use a shortcut to import all the classes in a package.

Türkçe: Aynı paketteki class'lar genellikle birlikte içe aktarılır. Bir paketteki tüm class'ları içe aktarmak için bir kısayol kullanabilirsiniz.

```
import java.util.*; // imports java.util.Random among other things
public class NumberPicker {
    public static void main(String[] args) {
        Random r = new Random();
        System.out.println(r.nextInt(10));
    }
}
```

English: In this example, we imported `java.util.Random` and a pile of other classes. The `*` is a wildcard that matches all classes in the package. Every class in the `java.util` package is available to this program when Java compiles it. The import statement doesn't bring in child packages, fields, or methods; it imports only classes directly under the package. Let's say you wanted to use the class `AtomicInteger` (you learn about that one in Chapter 13, "Concurrency") in the `java.util.concurrent.atomic` package. Which import or imports support this?

Türkçe: Bu örnekte `java.util.Random` ile aynı package'taki birçok başka class'ı içe aktardık. `*`, package içindeki tüm class adlarıyla eşleşen bir wildcard'dır (joker karakter). Java derleme yaparken `java.util` package'ındaki her class bu program tarafından kullanılabilir. Ancak `import` deyimi child package'ları, field'ları veya method'ları içe aktarmaz; yalnızca doğrudan belirtilen package altında bulunan class'ları içe aktarır. `java.util.concurrent.atomic` package'ındaki `AtomicInteger` class'ını kullanmak istediğinizi varsayalım (bu class Bölüm 13, "Concurrency" konusunda ele alınır). Hangi `import` deyimi veya deyimleri bunu sağlar?

```
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
```

English: Only the last import allows the class to be recognized because child packages are not included with the first two.

Türkçe: Yalnızca son içe aktarma, class'ın tanınmasına izin verir çünkü alt package'lar ilk ikisine dahil değildir.

English: You might think that including so many classes slows down your program execution, but it doesn't. The compiler figures out what's actually needed. Which approach you choose is personal preference—or team preference, if you are working with others on a team. Listing the classes used makes the code easier to read, especially for new programmers. Using the wildcard can shorten the import list. You'll see both approaches on the exam.

Türkçe: Bu kadar çok class eklemenin programınızın yürütülmesini yavaşlatacağını düşünebilirsiniz, ancak öyle değil. compiler gerçekte neye ihtiyaç duyulduğunu bulur. Hangi yaklaşımı seçeceğinizi kişisel tercihtir veya başkalarıyla bir ekipte çalışıyorsanız ekip tercihidir. Kullanılan class'ların listelenmesi, özellikle yeni programcılar için kodun okunmasını kolaylaştırır. Joker karakterin kullanılması içe aktarma listesini kısaltabilir. Sınavda her iki yaklaşımı da göreceksiniz.

Redundant Imports

Türkçe başlık: Gereksiz Import'lar

English: Wait a minute! We've been referring to `System` without an import every time we printed text, and Java found it just fine. There's one special package in the Java world called `java.lang`. This package is special in that it is automatically imported. You can type this package in an import statement, but you don't have to. In the following code, how many of the imports do you think are redundant?

Türkçe: Bir dakika! Her metin yazdığımızda herhangi bir `import` kullanmadan `System`'a başvurduk ve Java onu sorunsuz buldu. Java'da `java.lang` adlı özel bir package vardır ve bu package otomatik olarak içe aktarılır. İsterseniz onu bir `import` deyiminde açıkça yazabilirsiniz, ancak buna gerek yoktur. Aşağıdaki kodda kaç `import` gereksizdir?

```
1: import java.lang.System;
2: import java.lang.*;
```

Kaynak PDF sayfası 14

```
3: import java.util.Random;
4: import java.util.*;
5: public class NumberPicker {
6:     public static void main(String[] args) {
7:         Random r = new Random();
8:         System.out.println(r.nextInt(10));
9:     }
10: }
```

English: The answer is that three of the imports are redundant. Lines 1 and 2 are redundant because everything in `java.lang` is automatically imported. Line 4 is also redundant in this example because `Random` is already imported from `java.util.Random`. If line 3 wasn't present, `java.util.*` wouldn't be redundant, though, since it would cover importing `Random`.

Türkçe: Üç `import` gereksizdir. `java.lang` içindeki her şey otomatik olarak içe aktarıldığı için 1. ve 2. satırlar gereksizdir. Bu örnekte 4. satır da gereksizdir; çünkü `Random` zaten `java.util.Random` ile açıkça içe aktarılmıştır. Ancak 3. satır bulunmasaydı `java.util.*`, `Random`'ı da kapsayacağından gereksiz olmazdı.

English: Another case of redundancy involves importing a class that is in the same package as the class importing it. Java automatically looks in the current package for other classes.

Türkçe: Başka bir redundant import durumu, import'u yapan class ile aynı package'taki bir class'ın import edilmesidir. Java diğer class'ları bulmak için current package'a zaten otomatik olarak bakar.

English: Let's take a look at one more example to make sure you understand the edge cases for imports. For this example, `Files` and `Paths` are both in the package `java.nio.file`. The exam may use packages you may never have seen before. The question will let you know which package the class is in if you need to know that in order to answer the question.

Türkçe: `import` ile ilgili sınır durumlarını anladığınızdan emin olmak için bir örneğe daha bakalım. Bu örnekte `Files` ve `Paths` class'larının ikisi de `java.nio.file` package'ındadır. Sınavda daha önce hiç görmediğiniz package'lar kullanılabilir. Soruyu yanıtlamak için gerekliyse class'ın hangi package'ta bulunduğu size verilir.

English: Which import statements do you think would work to get this code to compile?

Türkçe: Bu kodun derlenmesini sağlamak için hangi içe aktarma ifadelerinin işe yarayacağını düşünüyorsunuz?

```
public class InputImports {  
    public void read(Files files) {  
        Paths.get("name");  
    }  
}
```

English: There are two possible answers. The shorter one is to use a wildcard to import both at the same time.

Türkçe: İki olası cevap var. Daha kısa olanı, her ikisini de aynı anda içe aktarmak için joker karakter kullanmaktır.

```
import java.nio.file.*;
```

English: The other answer is to import both classes explicitly.

Türkçe: Diğer cevap ise her iki class'ı da açıkça içe aktarmaktır.

```
import java.nio.file.Files;  
import java.nio.file.Paths;
```

English: Now let's consider some imports that don't work.

Türkçe: Şimdi çalışmayan bazı `import` deyimlerini inceleyelim.

```
import java.nio.*; // NO GOOD - a wildcard only matches  
// class names, not "file.Files"  
import java.nio.*.*; // NO GOOD - you can only have one wildcard  
// and it must be at the end  
import java.nio.file.Paths.*; // NO GOOD - you cannot import methods  
// only class names
```

Kaynak PDF sayfası 15

Naming Conflicts

Türkçe başlık: Ad Çakışmaları

English: One of the reasons for using packages is so that class names don't have to be unique across all of Java. This means you'll sometimes want to import a class that can be found in multiple places. A common example of this is the Date class. Java provides implementations of `java.util.Date` and `java.sql.Date`. What import statement can we use if we want the `java.util.Date` version?

Türkçe: Package kullanmanın nedenlerinden biri, class adlarının Java'nın tamamında benzersiz olmak zorunda olmamasıdır. Bu nedenle bazen birden fazla package'ta bulunan bir class'ı içe aktarmak isteyebilirsiniz. Bunun yaygın bir örneği `Date` class'ıdır. Java hem `java.util.Date` hem de `java.sql.Date` implementasyonlarını sağlar. `java.util.Date` sürümünü kullanmak istiyorsak hangi `import` deyimini yazabiliriz?

```
public class Conflicts {  
    Date date;  
    // some more code  
}
```

English: The answer should be easy by now. You can write either `import java.util.*;` or

Türkçe: Cevap şu ana kadar kolay olmalı. `import java.util.*;` yazabilirsiniz. veya

```
import java.util.Date;
```

English: The tricky cases come about when other imports are present.

Türkçe: Zor durumlar, başka `import` deyimleri de bulunduğunda ortaya çıkar.

```
import java.util.*;  
import java.sql.*; // causes Date declaration to not compile
```

English: When the class name is found in multiple packages, Java gives you a compiler error. In our example, the solution is easy— remove the `import java.sql.*` that we don't need. But what do we do if we need a whole pile of other classes in the `java.sql` package?

Türkçe: Bir class adı birden fazla package'ta bulunduğunda Java compiler hatası verir. Örneğimizde çözüm kolaydır: gerek duymadığımız `import java.sql.*;` deyimini kaldırırız. Peki `java.sql` package'ındaki birçok başka class'a ihtiyacımız varsa ne yapacağız?

```
import java.util.Date;  
import java.sql.*;
```

English: Ah, now it works! If you explicitly import a class name, it takes precedence over any wildcards present. Java thinks, "The programmer really wants me to assume use of the `java.util.Date` class." One more example. What does Java do with "ties" for precedence?

Türkçe: Artık çalışır. Bir class adını açıkça içe aktarmak, wildcard ile yapılan `import` deyimlerine göre önceliklidir. Java, programcının özellikle `java.util.Date` class'ını kullanmak istediğini kabul eder. Peki aynı önceliğe sahip iki açık `import` arasında eşitlik olursa Java ne yapar?

```
import java.util.Date;  
import java.sql.Date;
```

English: Java is smart enough to detect that this code is no good. As a programmer, you've claimed to explicitly want the default to be both the `java.util.Date` and `java.sql.Date` implementations. Because there can't be two defaults, the compiler tells you the imports are ambiguous.

Türkçe: Java bu kodun geçersiz olduğunu algılar. Programcı hem `java.util.Date` hem de `java.sql.Date` implementasyonunu açıkça varsayılan yapmak istemiştir. İki varsayılan olamayacağı için compiler, `import` deyimlerinin ambiguous (belirsiz) olduğunu bildirir.

If You Really Need to Use Two Classes with the Same Name

Türkçe başlık: Aynı Ada Sahip İki Class'ı Gerçekten Kullanmanız Gerekirse

English: Sometimes you really do want to use `Date` from two different packages. When this happens, you can pick one to use in the import statement and use the other's fully qualified class name. Or you can drop both import statements and always use the fully qualified class name.

Türkçe: Bazen iki farklı package'taki `Date` class'larına birlikte ihtiyaç duyarsınız. Bu durumda birini import edip diğeri için fully qualified class name kullanabilirsiniz. Alternatif olarak iki import'u da kaldırıp her kullanımda fully qualified class name yazabilirsiniz.

```
public class Conflicts {  
    java.util.Date date;  
    java.sql.Date sqlDate;  
}
```

Kaynak PDF sayfası 16

Creating a New Package

Türkçe başlık: Yeni Paket Oluşturma

English: Up to now, all the code we've written in this chapter has been in the default package. This is a special unnamed package that you should use only for throwaway code. You can tell the code is in the default package, because there's no package name. On the exam, you'll see the default package used a lot to save space in code listings. In real life, always name your packages to avoid naming conflicts and to allow others to reuse your code.

Türkçe: Şu ana kadar bu bölümde yazdığımız tüm kodlar varsayılan paketteydi. Bu, yalnızca tek kullanımlık kod için kullanmanız gereken, adsız özel bir pakettir. Paket adı olmadığından kodun varsayılan pakette olduğunu anlayabilirsiniz. Sınavda, kod listelerinde yer kazanmak için varsayılan paketin çok kullanıldığını göreceksiniz. Gerçek hayatta, adlandırma çakışmalarını önlemek ve başkalarının kodunuzu yeniden kullanmasına izin vermek için paketlerinizi her zaman adlandırın.

English: Now it's time to create a new package. The directory structure on your computer is related to the package name. In this section, just read along. We cover how to compile and run the code in the next section.

Türkçe: Şimdi yeni bir package oluşturmanın zamanı geldi. Bilgisayarınızdaki dizin yapısı package adıyla ilgilidir. Bu bölümde sadece okuyun. Bir sonraki bölümde kodun nasıl derleneceğini ve çalıştırılacağını ele alacağız.

English: Suppose we have these two classes:

Türkçe: Diyelim ki bu iki class'a sahibiz:

```
package packagea;  
public class ClassA {}
```

```
package packageb;
import packagea.ClassA;
public class ClassB {
    public static void main(String[] args) {
        ClassA a;
        System.out.println("Got it");
    }
}
```

English: When you run a Java program, Java knows where to look for those package names.

Türkçe: Bir Java programını çalıştırdığınızda, Java bu package adlarını nerede arayacağını bilir.

English: In this case, running from C:\temp works because both packagea and packageb are underneath it.

Türkçe: Bu durumda C:\temp dizininden çalıştırmak işe yarar; çünkü hem packagea hem de packageb onun altındadır.

Compiling and Running Code with Packages

Türkçe başlık: Paketlerle Kod Derleme ve Çalıştırma

English: You'll learn Java much more easily by using the command line to compile and test your examples. Once you know the Java syntax well, you can switch to an IDE. But for the exam, your goal is to know details about the language and not have the IDE hide them for you.

Türkçe: Örneklerinizi derlemek ve test etmek için komut satırını kullanarak Java'yı çok daha kolay öğreneceksiniz. Java sözdizimini iyi öğrendikten sonra IDE'ye geçebilirsiniz. Ancak sınav için amacınız dil hakkındaki ayrıntıları bilmek ve IDE'nin bunları sizin için saklamasına izin vermemek.

English: Follow this example to make sure you know how to use the command line. If you have any problems following this procedure, post a question in the Beginning Java forum at CodeRanch. Describe what you tried and what the error said.

Türkçe: Command line'ı nasıl kullanacağınızı bildiğinizden emin olmak için bu örneği izleyin. Prosedürü uygularken sorun yaşarsanız CodeRanch'taki Java'ya Başlangıç forumuna bir soru gönderin. Ne denediğinizi ve error message'ın ne söylediğini açıklayın.

English: www.coderanch.com/forums/f-33/java

Türkçe: www.coderanch.com/forums/f-33/java

English: The first step is to create the two files from the previous section. Table 1.1 shows the expected fully qualified filenames and the command to get into the directory for the next steps.

Türkçe: İlk adım, önceki bölümdeki iki file'ı oluşturmaktır. Tablo 1.1, beklenen fully qualified file adlarını ve sonraki adımlar için ilgili directory'ye geçme command'ını gösterir.

Kaynak PDF sayfası 17

English table caption: TABLE 1.1 Setup procedure by operating system

Türkçe tablo başlığı: TABLO 1.1 İşletim sistemine göre kurulum prosedürü

Step / Adım	Windows	Mac/Linux
1. Create first class / İlk class'ı oluştur	<code>C:\temp\packagea\ClassA.java</code>	<code>/tmp/packagea/ClassA.java</code>
2. Create second class / İkinci class'ı oluştur	<code>C:\temp\packageb\ClassB.java</code>	<code>/tmp/packageb/ClassB.java</code>
3. Go to directory / Dizine git	<code>cd C:\temp</code>	<code>cd /tmp</code>

English: Now it is time to compile the code. Luckily, this is the same regardless of the operating system. To compile, type the following command:

Türkçe: Artık kodu derleme zamanıdır. Neyse ki komut, işletim sisteminden bağımsız olarak aynıdır. Derlemek için şu komutu yazın:

```
javac packagea/ClassA.java packageb/ClassB.java
```

English: If this command doesn't work, you'll get an error message. Check your files carefully for typos against the provided files. If the command does work, two new files will be created:

Türkçe: Bu komut işe yaramazsa bir hata mesajı alırsınız. Sağlanan dosyalarda yazım hataları olup olmadığını görmek için dosyalarınızı dikkatlice kontrol edin. Komut işe yararsa iki yeni dosya oluşturulacaktır:

English: packagea/ClassA.class and packageb/ClassB.class.

Türkçe: `packagea/ClassA.class` ve `packageb/ClassB.class`.

Compiling with Wildcards

Türkçe başlık: Wildcard ile Derleme

English: You can use an asterisk to specify that you'd like to include all Java files in a directory. This is convenient when you have a lot of files in a package. We can rewrite the previous javac command like this:

Türkçe: Bir dizindeki bütün Java file'larını dahil etmek istediğinizi belirtmek için asterisk (*) kullanabilirsiniz. Bir package'ta çok sayıda file olduğunda bu kullanışlıdır. Önceki `javac` komutunu şöyle yeniden yazabiliriz:

```
javac packagea/*.java packageb/*.java
```

English: However, you cannot use a wildcard to include subdirectories. If you were to write `javac *.java`, the code in the packages would not be picked up.

Türkçe: Ancak alt dizinleri eklemek için joker karakter kullanamazsınız. `javac *.java` yazarsanız paketlerin içindeki kodlar alınmazdı.

English: Now that your code has compiled, you can run it by typing the following command:

Türkçe: Artık kodunuz derlendiğine göre, aşağıdaki komutu yazarak çalıştırabilirsiniz:

```
java packageb.ClassB
```

English: If it works, you'll see `Got it` printed. You might have noticed that we typed `ClassB` rather than `ClassB.class`. As discussed earlier, you don't pass the extension when running a program.

Türkçe: Komut çalışırsa `Got it` çıktısını görürsünüz. `ClassB.class` yerine `ClassB` yazdığımıza dikkat edin. Daha önce belirtildiği gibi programı çalıştırırken `.class` uzantısı verilmez.

English: Figure 1.1 shows where the `.class` files were created in the directory structure.

Türkçe: Şekil 1.1, `.class` dosyalarının dizin yapısında nerede oluşturulduğunu göstermektedir.

Kaynak PDF sayfası 18

English figure caption: FIGURE 1.1 Compiling with packages

Türkçe şekil başlığı: ŞEKİL 1.1 Paketlerle derleme

English: `packagea ClassA.java ClassA.class packageb ClassB.java ClassB.class`

Türkçe: `packagea` altında `ClassA.java` ve `ClassA.class`; `packageb` altında `ClassB.java` ve `ClassB.class` bulunur.

Compiling to Another Directory

Türkçe başlık: Başka Dizine Derleme

English: By default, the `javac` command places the compiled classes in the same directory as the source code. It also provides an option to place the class files into a different directory. The `-d` option specifies this target directory.

Türkçe: `javac` komutu varsayılan olarak derlenmiş class'ları source code ile aynı dizine yerleştirir. Ayrıca class file'larını farklı bir dizine yerleştirme seçeneği de sunar. `-d` seçeneği bu target directory'yi belirtir.

English: Java options are case sensitive. This means you cannot pass `-D` instead of `-d`.

Türkçe: Java option'ları case-sensitive'dir. Bu nedenle `-d` yerine `-D` kullanamazsınız.

English: If you are following along, delete the ClassA.class and ClassB.class files that were created in the previous section. Where do you think this command will create the file ClassA.class?

Türkçe: Devam ediyorsanız, önceki bölümde oluşturulan ClassA.class ve ClassB.class dosyalarını silin. Bu komutun ClassA.class dosyasını nerede oluşturacağını düşünüyorsunuz?

```
javac -d classes packagea/ClassA.java packageb/ClassB.java
```

English: The correct answer is in classes/packagea/ClassA.class. The package structure is preserved under the requested target directory. Figure 1.2 shows this new structure.

Türkçe: Doğru konum `classes/packagea/ClassA.class` yoludur. Package yapısı, istenen hedef dizinin altında korunur. Şekil 1.2 bu yeni yapıyı gösterir.

English figure caption: FIGURE 1.2 Compiling with packages and directories

Türkçe şekil başlığı: ŞEKİL 1.2 Paketler ve dizinlerle derleme

English: packagea ClassA.java packageb ClassB.java classes packagea ClassA.class packageb ClassB.class

Türkçe: Kaynaklar `packagea/ClassA.java` ve `packageb/ClassB.java`; üretilen dosyalar ise `classes/packagea/ClassA.class` ve `classes/packageb/ClassB.class` yollarındadır.

Kaynak PDF sayfası 19

English: To run the program, you specify the classpath so Java knows where to find the classes.

Türkçe: Programı çalıştırırken classpath'i belirtirsiniz; böylece Java class'ları nerede arayacağını bilir.

English: There are three options you can use. All three of these do the same thing:

Türkçe: Kullanabileceğiniz üç seçenek var. Bunların üçü de aynı şeyi yapıyor:

```
java -cp classes packageb.ClassB
java -classpath classes packageb.ClassB
java --class-path classes packageb.ClassB
```

English: Notice that the last one requires two dashes (--), while the first two require one dash (-).

Türkçe: Sonuncunun iki tire (--) gerektirdiğine, ilk ikisinin ise bir tire (-) gerektirdiğine dikkat edin.

English: If you have the wrong number of dashes, the program will not run.

Türkçe: Yanlış sayıda tire varsa program çalışmayacaktır.

Three Classpath Options

Türkçe başlık: Üç Classpath Seçeneği

English: You might wonder why there are three options for the classpath. The `-cp` option is the short form. Developers frequently choose the short form because we are lazy typists. The `-classpath` and `--class-path` versions can be clearer to read but require more typing.

Türkçe: Classpath için neden üç seçenek bulunduğunu merak edebilirsiniz. `-cp` kısa biçimdir. Geliştiriciler daha az yazım gerektirdiği için çoğunlukla bunu tercih eder. `-classpath` ve `--class-path` biçimleri daha açıklayıcıdır, ancak daha uzundur.

English: Table 1.2 and Table 1.3 review the options you need to know for the exam. There are many other options available! And in Chapter 12, “Modules,” you learn additional options specific to modules.

Türkçe: Tablo 1.2 ve Tablo 1.3 sınav için bilmeniz gereken seçenekleri gözden geçirmektedir. Başka birçok seçenek mevcut! Ve Bölüm 12, “Modüller”de, modüllere özel ek seçenekleri öğreneceksiniz.

English table caption: TABLE 1.2 Important javac options

Türkçe tablo başlığı: TABLO 1.2 Önemli javac seçenekleri

Option / Seçenek	Description / Açıklama
<code>-cp <classpath></code> , <code>-classpath <classpath></code> , <code>--class-path <classpath></code>	Location of classes needed to compile the program / Programı derlemek için gereken class'ların konumu
<code>-d <dir></code>	Directory in which to place generated class files / Üretilen class dosyalarının yerleştirileceği dizin

English table caption: TABLE 1.3 Important java options

Türkçe tablo başlığı: TABLO 1.3 Önemli Java seçenekleri

Option / Seçenek	Description / Açıklama
<code>-cp <classpath></code> , <code>-classpath <classpath></code> , <code>--class-path <classpath></code>	Location of classes needed to run the program / Programı çalıştırmak için gereken class'ların konumu

Kaynak PDF sayfası 20

Compiling with JAR Files

Türkçe başlık: JAR Dosyalarıyla Derleme

English: Just like the classes directory in the previous example, you can also specify the location of the other files explicitly using a classpath. This technique is useful when the class files are located elsewhere or in special JAR files. A Java archive (JAR) file is like a ZIP file of mainly Java class files.

Türkçe: Önceki örnekteki `classes` dizininde olduğu gibi başka file'ların konumunu da classpath üzerinden açıkça belirtebilirsiniz. Bu teknik, class file'ları başka bir yerde veya özel JAR file'larında bulunduğu kullanışlıdır. Java archive (JAR) file'ı, çoğunlukla Java class file'ları içeren bir ZIP file'ına benzer.

English: On Windows, you type the following:

Türkçe: Windows'ta aşağıdakileri yazarsınız:

```
java -cp ".;C:\temp\someOtherLocation;c:\temp\myJar.jar" myPackage.MyClass
```

English: And on macOS/Linux, you type this:

Türkçe: Ve macOS / Linux'ta şunu yazarsınız:

```
java -cp ".:tmp/someOtherLocation:tmp/myJar.jar" myPackage.MyClass
```

English: The period (.) indicates that you want to include the current directory in the classpath. The rest of the command says to look for loose class files (or packages) in someOtherLocation and within myJar.jar. Windows uses semicolons (;) to separate parts of the classpath; other operating systems use colons.

Türkçe: Nokta (.), current directory'nin classpath'e eklenmesini belirtir. Komutun kalan kısmı Java'ya `someOtherLocation` içindeki bağımsız class file'larını (veya package'ları) ve `myJar.jar` içeriğini aramasını söyler. Windows classpath parçalarını noktalı virgülle (;), diğer işletim sistemleri iki noktayla (:) ayırır.

English: Just like when you're compiling, you can use a wildcard (*) to match all the JARs in a directory. Here's an example:

Türkçe: Tıpkı derleme yaparken olduğu gibi, bir dizindeki tüm JAR'ları eşleştirmek için joker karakter (*) kullanabilirsiniz. İşte bir örnek:

```
java -cp "C:\temp\directoryWithJars\*" myPackage.MyClass
```

English: This command will add to the classpath all the JARs that are in directoryWithJars. It won't include any JARs in the classpath that are in a subdirectory of directoryWithJars.

Türkçe: Bu komut, `directoryWithJars` içindeki bütün JAR'ları classpath'e ekler. `directoryWithJars` altındaki subdirectory'lerde bulunan JAR'ları classpath'e eklemeyiz.

Creating a JAR File

Türkçe başlık: JAR Dosyası Oluşturma

English: Some JARs are created by others, such as those downloaded from the Internet or created by a teammate. Alternatively, you can create a JAR file yourself. To do so, you use the `jar` command. The simplest commands create a jar containing the files in the current directory.

Türkçe: İnternette indirilen veya bir ekip arkadaşı tarafından hazırlananlar gibi bazı JAR file'ları başkalarının oluşturulur. İsterseniz `jar` komutuyla kendi JAR file'ınızı da oluşturabilirsiniz. En basit komutlar current directory'deki file'ları içeren bir JAR üretir.

English: You can use the short or long form for each option.

Türkçe: Her seçenek için kısa veya uzun formu kullanabilirsiniz.

```
jar -cvf myNewFile.jar .
jar --create --verbose --file myNewFile.jar .
```

English: Alternatively, you can specify a directory instead of using the current directory.

Türkçe: Alternatif olarak, geçerli dizini kullanmak yerine bir dizin belirtebilirsiniz.

```
jar -cvf myNewFile.jar -C dir .
```

English: There is no long form of the `-C` option. Table 1.4 lists the options you need to use the `jar` command to create a JAR file. In Chapter 12, you see `jar` again for modules.

Türkçe: `-C` seçeneğinin uzun bir biçimi yoktur. Tablo 1.4, JAR dosyası oluşturmak için `jar` komutunu kullanmak için ihtiyaç duyduğunuz seçenekleri listelemektedir. Bölüm 12'de modüller için yine `jar` ifadesini göreceksiniz.

Kaynak PDF sayfası 21

English table caption: TABLE 1.4 Important jar options

Türkçe tablo başlığı: TABLO 1.4 Önemli `jar` seçenekleri

Option / Seçenek	Description / Açıklama
<code>-c, --create</code>	Creates a new JAR file / Yeni bir JAR dosyası oluşturur
<code>-v, --verbose</code>	Prints details when working with JAR files / JAR dosyalarıyla çalışırken ayrıntıları yazdırır
<code>-f <fileName>, --file <fileName></code>	JAR filename / JAR dosyasının adı
<code>-C <directory></code>	Directory containing files to be used to create the JAR / JAR oluşturulurken kullanılacak dosyaları içeren dizin

Ordering Elements in a Class

Türkçe başlık: Bir Class İçindeki Element'leri Sıralama

English: Now that you've seen the most common parts of a class, let's take a look at the correct order to type them into a file. Comments can go anywhere in the code. Beyond that, you need to memorize the rules in Table 1.5.

Türkçe: Artık bir class'ın en yaygın bölümlerini gördüğünüze göre, bunları bir dosyaya yazmanın doğru sırasına bir göz atalım. Yorumlar kodun herhangi bir yerine gidebilir. Bunun ötesinde Tablo 1.5'teki kuralları ezberlemeniz gerekmektedir.

English table caption: TABLE 1.5 Order for declaring a class

Türkçe tablo başlığı: TABLO 1.5 class bildirme sırası

Element / Öğe	Example / Örnek	Required? / Zorunlu mu?	Where does it go? / Konumu
Package declaration	<code>package abc;</code>	No / Hayır	First line in the file, excluding comments or blank lines / Comment ve boş satırlar hariç dosyanın ilk satırı
Import statements	<code>import java.util.*;</code>	No / Hayır	Immediately after the package, if present / Varsa package declaration'dan hemen sonra
Top-level type declaration	<code>public class C</code>	Yes / Evet	Immediately after imports, if any / Varsa import'lardan hemen sonra
Field declarations	<code>int value;</code>	No / Hayır	Any top-level element within a class / Class içinde herhangi bir top-level element
Method declarations	<code>void method()</code>	No / Hayır	Any top-level element within a class / Class içinde herhangi bir top-level element

Kaynak PDF sayfası 22

English: Let's look at a few examples to help you remember this. The first example contains one of each element:

Türkçe: Bunu hatırlamanıza yardımcı olacak birkaç örneğe bakalım. İlk örnek her öğeden birini içerir:

```
package structure; // package must be first non-comment
import java.util.*; // import must come after package
public class Meerkat { // then comes the class
    double weight; // fields and methods can go in either order
    public double getWeight() {
        return weight;
    }
    double height; // another field - they don't need to be together
}
```

English: So far, so good. This is a common pattern that you should be familiar with. How about this one?

Türkçe: Şu ana kadar çok iyi. Bu, aşına olmanız gereken yaygın bir kalıptır. Buna ne dersiniz?

```
/* header */
package structure;
// class Meerkat
public class Meerkat { }
```

English: Still good. We can put comments anywhere, blank lines are ignored, and imports are optional. In the next example, we have a problem:

Türkçe: Hala iyi. Yorumları herhangi bir yere koyabiliriz, boş satırlar dikkate alınmaz ve içe aktarmalar isteğe bağlıdır. Bir sonraki örnekte bir sorunuz var:

```
import java.util.*;
package structure; // DOES NOT COMPILE
String name; // DOES NOT COMPILE
public class Meerkat { } // DOES NOT COMPILE
```

English: There are two problems here. One is that the package and import statements are reversed. Although both are optional, package must come before import if present. The other issue is that a field attempts a declaration outside a class. This is not allowed. Fields and methods must be within a class.

Türkçe: Burada iki sorun vardır. İlk olarak `package` ve `import` declaration'larının sırası terstir. İkisi de optional olsa da mevcutlarsa `package`, `import`tan önce gelmelidir. İkinci olarak bir field class dışında declare edilmeye çalışılmıştır; buna izin verilmez. Field'lar ve method'lar bir class'ın içinde olmalıdır.

English: Got all that? Think of the acronym PIC (picture): package, import, and class. Fields and methods are easier to remember because they merely have to be inside a class.

Türkçe: Hepsini kavradınız mı? PIC mnemonic'ini düşünün: package, import ve class. Field ve method'ları hatırlamak daha kolaydır; çünkü bunlar yalnızca bir class'ın içinde bulunabilir.

English: Throughout this book, if you see two public classes in a code snippet or question, you can assume they are in different files unless it specifically says they are in the same .java file.

Türkçe: Bu kitap boyunca bir code snippet veya soruda iki `public class` görürseniz, özellikle aynı `.java` file'ında oldukları belirtilmedikçe farklı file'larda bulunduklarını varsayabilirsiniz.

English: Now you know how to create and arrange a class. Later chapters show you how to create classes with more powerful operations.

Türkçe: Artık bir class'ın nasıl oluşturulacağını ve düzenleneceğini biliyorsunuz. Daha sonraki bölümler size daha güçlü işlemlerle class'ların nasıl oluşturulacağını gösterecektir.

Kaynak PDF sayfası 23

Creating Objects

Türkçe başlık: Object'ler Oluşturma

English: Our programs wouldn't be able to do anything useful if we didn't have the ability to create new objects. Remember that an object is an instance of a class. In the following sections, we look at constructors, object fields, instance initializers, and the order in which values are initialized.

Türkçe: Yeni object'ler oluşturamıyorduk programlarımız pek işe yaramazdı. Bir object'in, bir class'ın instance'ı olduğunu hatırlayın. Aşağıdaki bölümlerde constructor'ları, object field'larını, instance initializer'ları ve initialization order'ı inceleyeceğiz.

Calling Constructors

Türkçe başlık: Constructor'ları Çağırma

English: To create an instance of a class, all you have to do is write new before the class name and add parentheses after it. Here's an example:

Türkçe: Bir class'tan instance oluşturmak için class adının önüne **new**, ardına parentheses **()** yazmanız yeterlidir. Örnek:

```
Park p = new Park();
```

English: First you declare the type that you'll be creating (Park) and give the variable a name (p).

Türkçe: Öncelikle oluşturacağınız türü (Park) bildirirsiniz ve variable'a bir ad verirsiniz (p).

English: This gives Java a place to store a reference to the object. Then you write new Park() to actually create the object.

Türkçe: Bu, Java'ya object reference'ını saklayacağı bir yer sağlar. Ardından object'i gerçekten oluşturmak için **new Park()** yazarsınız.

English: Park() looks like a method since it is followed by parentheses. It's called a constructor, which is a special type of method that creates a new object. Now it's time to define a constructor of your own:

Türkçe: Ardından parantez geldiği için **Park()** bir method'a benzer. Ancak bu, yeni object oluşturan özel yapı olan constructor'dır. Şimdi kendi constructor'ınızı declare edelim:

```
public class Chick {  
    public Chick() {  
        System.out.println("in constructor");  
    }  
}
```

English: There are two key points to note about the constructor: the name of the constructor matches the name of the class, and there's no return type. You may see a method like this on the exam:

Türkçe: Constructor hakkında iki temel nokta vardır: Constructor adı class adıyla aynıdır ve return type yoktur. Sınavda şöyle bir method görebilirsiniz:

```
public class Chick {  
    public void Chick() { } // NOT A CONSTRUCTOR  
}
```

English: When you see a method name beginning with a capital letter and having a return type, pay special attention to it. It is not a constructor since there's a return type. It's a regular method that does compile but will not be called when you write `new Chick()`.

Türkçe: Büyük harfle başlayan ve return type'ı bulunan bir method adı gördüğünüzde dikkat edin. Return type bulunduğu için bu bir constructor değildir. Kod derlenir; ancak `new Chick()` yazıldığında bu normal method çağrılmaz.

English: The purpose of a constructor is to initialize fields, although you can put any code in there.

Türkçe: Constructor'ın temel amacı field'ları initialize etmektir; ancak gövdesine başka kodlar da yazılabilir.

English: Another way to initialize fields is to do so directly on the line on which they're declared. This example shows both approaches:

Türkçe: Field'ları initialize etmenin başka bir yolu, değeri doğrudan declare edildikleri satırda vermektir. Bu örnek iki yaklaşımı da gösterir:

```
public class Chicken {  
    int numEggs = 12; // initialize on line  
    String name;
```

Kaynak PDF sayfası 24

```
public Chicken() {  
    name = "Duke"; // initialize in constructor  
}
```

English: For most classes, you don't have to code a constructor— the compiler will supply a “do nothing” default constructor for you. There are some scenarios that do require you to declare a constructor. You learn all about them in Chapter 6.

Türkçe: Çoğu class için constructor yazmanız gerekmez; compiler sizin için hiçbir işlem yapmayan bir default constructor sağlar. Bazı durumlarda constructor'ı açıkça declare etmek gerekir. Bunlar Bölüm 6'da ele alınacaktır.

Reading and Writing Member Fields

Türkçe başlık: Member Field'ları Okuma ve Yazma

English: It's possible to read and write instance variables directly from the caller. In this example, a mother swan lays eggs:

Türkçe: Instance variable'ları çağırarak koddan doğrudan okumak ve onlara yazmak mümkündür. Bu örnekte bir anne kuğu yumurtlar:

```
public class Swan {
    int numberEggs; // instance variable
    public static void main(String[] args) {
        Swan mother = new Swan();
        mother.numberEggs = 1; // set variable
        System.out.println(mother.numberEggs); // read variable
    }
}
```

English: The “caller” in this case is the main() method, which could be in the same class or in another class. This class sets numberEggs to 1 and then reads numberEggs directly to print it out. In Chapter 5, you learn how to use encapsulation to protect the Swan class from having someone set a negative number of eggs.

Türkçe: Bu durumda caller (çağırıcı kod), aynı class'ta veya başka bir class'ta bulunabilen `main()` method'udur. Kod `numberEggs` değerini `1` yapar, ardından yazdırmak için `numberEggs` field'ını doğrudan okur. Bölüm 5'te, birinin negatif yumurta sayısı atamasını önlemek amacıyla `Swan` class'ını encapsulation ile nasıl koruyacağınızı öğreneceksiniz.

English: You can even read values of already initialized fields on a line initializing a new field:

Türkçe: Yeni bir field'ı initialize eden satırda, daha önce initialize edilmiş field'ların değerleri de okunabilir:

```
1: public class Name {
2:     String first = "Theodore";
3:     String last = "Moose";
4:     String full = first + last;
5: }
```

English: Lines 2 and 3 both write to fields. Line 4 both reads and writes data. It reads the fields first and last. It then writes the field full.

Türkçe: 2. ve 3. satırlar field'lara değer yazar. 4. satır ise hem veri okur hem de yazar: önce `first` ve `last` field'larını okur, ardından sonucu `full` field'ına yazar.

Executing Instance Initializer Blocks

Türkçe başlık: Instance Initializer Block'larını Yürütme

English: When you learned about methods, you saw braces (`{}`). The code between the braces (sometimes called “inside the braces”) is called a code block. Anywhere you see braces is a code block.

Türkçe: Method'ları öğrenirken braces, yani süslü parantezleri (`{ }`) gördünüz. Süslü parantezler arasındaki koda code block denir. Süslü parantez gördüğünüz her yerde bir code block vardır.

Kaynak PDF sayfası 25

English: Sometimes code blocks are inside a method. These are run when the method is called.

Türkçe: Bazen code block'lar bir method'un içindedir ve method çağrıldığında yürütülür.

English: Other times, code blocks appear outside a method. These are called instance initializers. In Chapter 6, you learn how to use a static initializer.

Türkçe: Bazen de code block'lar method dışında bulunur; bunlara instance initializer denir. Bölüm 6'da static initializer kullanımı ele alınacaktır.

English: How many blocks do you see in the following example? How many instance initializers do you see?

Türkçe: Aşağıdaki örnekte kaç block görüyorsunuz? Kaç tane instance initializer görüyorsunuz?

```
1: public class Bird {  
2: public static void main(String[] args) {  
3: { System.out.println("Feathers"); }  
4: }  
5: { System.out.println("Snowy"); }  
6: }
```

English: There are four code blocks in this example: a class definition, a method declaration, an inner block, and an instance initializer. Counting code blocks is easy: you just count the number of pairs of braces. If there aren't the same number of open ({) and close (}) braces or they aren't defined in the proper order, the code doesn't compile. For example, you cannot use a closed brace (}) if there's no corresponding open brace ({) that it matches written earlier in the code. In programming, this is referred to as the balanced parentheses problem, and it often comes up in job interview questions.

Türkçe: Bu örnekte dört code block vardır: class declaration, method declaration, inner block ve instance initializer. Block'ları saymak için braces ({ }), yani süslü parantez çiftlerini sayarsınız. Açılan { ile kapanan } sayıları aynı değilse veya doğru sırada eşleşmiyorlarsa kod derlenmez. Daha önce eşleşen bir { olmadan } kullanılamaz. Programlamada buna balanced parentheses problem (dengeli parantez problemi) denir ve iş görüşmelerinde sık sorulur.

English: When you're counting instance initializers, keep in mind that they cannot exist inside of a method. Line 5 is an instance initializer, with its braces outside a method. On the other hand, line 3 is not an instance initializer, as it is only called when the main() method is executed.

Türkçe: Instance initializer'ları sayarken bunların method içinde bulunamayacağını unutmayın. 5. satırdaki braces bir method'un dışında olduğu için bu block bir instance initializer'dır. Buna karşılık 3. satır instance initializer değildir; yalnızca main() method'u yürütüldüğünde çalışır.

English: There is one additional set of braces on lines 1 and 6 that constitute the class declaration.

Türkçe: 1. ve 6. satırlarda class declaration'ını saran ek bir braces ({ }) çifti vardır.

Following the Order of Initialization

Türkçe başlık: Başlatma Sırasını Takip Etmek

English: When writing code that initializes fields in multiple places, you have to keep track of the order of initialization. This is simply the order in which different methods, constructors, or blocks are called when an instance of the class is created. We add some more rules to the order of initialization in Chapter 6. In the meantime, you need to remember:

Türkçe: Field'ları birden fazla yerde initialize eden kod yazarken initialization order'ı izlemelisiniz. Bu, class'tan bir instance oluşturulduğunda farklı method, constructor veya block'ların çağrılma sırasidir. Chapter 6'da bu sıraya yeni kurallar eklenecektir. Şimdilik şunu hatırlayın:

English: - Fields and instance initializer blocks are run in the order in which they appear in the file.

Türkçe: - Field'lar ve instance initializer block'ları, file içinde göründükleri sırayla çalışır.

English: - The constructor runs after all fields and instance initializer blocks have run.

Türkçe: - Constructor, bütün field'lar ve instance initializer block'ları çalıştıktan sonra çalışır.

English: Let's look at an example:

Türkçe: Bir örneğe bakalım:

```
1: public class Chick {
2:     private String name = "Fluffy";
3:     { System.out.println("setting field"); }
4:     public Chick() {
5:         name = "Tiny";
6:         System.out.println("setting constructor");
7:     }
```

Kaynak PDF sayfası 26

```
8: public static void main(String[] args) {
9:     Chick chick = new Chick();
10:    System.out.println(chick.name); } }
```

English: Running this example prints this:

Türkçe: Bu örnek şunları yazdırır:

```
setting field
setting constructor
Tiny
```

English: Let's look at what's happening here. We start with the `main()` method because that's where Java starts execution. On line 9, we call the constructor of `Chick`. Java creates a new object. First it initializes `name` to "Fluffy" on line 2. Next it executes the `println()` statement in the instance initializer on line 3. Once all the fields and instance initializers have run, Java returns to the constructor. Line 5 changes the value of `name` to "Tiny", and line 6 prints another statement. At this point, the constructor is done, and then the execution goes back to the `println()` statement on line 10.

Türkçe: Ne olduğuna bakalım. Execution, Java'nın başladığı `main()` method'unda başlar. 9. satırda `Chick` constructor'ı çağrılır ve Java yeni bir object oluşturur. Önce 2. satırda `name` field'ı "Fluffy" ile initialize edilir. Ardından 3. satırdaki instance initializer içindeki `println()` çalışır. Bütün field'lar ve instance initializer'lar tamamlandıncaya Java constructor'a döner. 5. satır `name` value'sunu "Tiny" yapar, 6. satır ikinci mesajı yazdırır. Constructor bitince execution 10. satırdaki `println()` statement'ına döner.

English: Order matters for the fields and blocks of code. You can't refer to a variable before it has been defined:

Türkçe: Field'lar ve code block'lar için sıra önemlidir. Bir variable'a declare edilmeden önce başvuramazsınız:

```
{ System.out.println(name); } // DOES NOT COMPILE
private String name = "Fluffy";
```

English: You should expect to see a question about initialization on the exam. Let's try one more.

Türkçe: Sınavda başlatmayla ilgili bir soru görmeyi beklemelisiniz. Bir tane daha deneyelim.

English: What do you think this code prints out?

Türkçe: Sizce bu kod ne yazdırıyor?

```
public class Egg {
    public Egg() {
        number = 5;
    }
    public static void main(String[] args) {
        Egg egg = new Egg();
        System.out.println(egg.number);
    }
    private int number = 3;
    { number = 4; } }
```

English: If you answered 5, you got it right. Fields and blocks are run first in order, setting `number` to 3 and then 4. Then the constructor runs, setting `number` to 5. You see a lot more rules and examples covering order of initialization in Chapter 6. We only cover the basics here so you can follow the order of initialization for simple programs.

Türkçe: Çıktı 5'tir. Önce field'lar ve block'lar sırayla çalışır; `number` önce 3, sonra 4 olur. Ardından constructor çalışır ve `number` değerini 5 yapar. Bölüm 6'da initialization order hakkında daha fazla kural ve örnek göreceksiniz. Burada basit programların sırasını izlemeye yetecek temeller ele alınmaktadır.

Understanding Data Types

Türkçe başlık: Veri Türlerini Anlamak

English: Java applications contain two types of data: primitive types and reference types. In this section, we discuss the differences between a primitive type and a reference type.

Türkçe: Java uygulamaları iki tür veri içerir: primitive type'lar ve referans türleri. Bu bölümde primitive tür ile referans türü arasındaki farkları tartışıyoruz.

Kaynak PDF sayfası 27

Using Primitive Types

Türkçe başlık: Primitive Türleri Kullanma

English: Java has eight built-in data types, referred to as the Java primitive types. These eight data types represent the building blocks for Java objects, because all Java objects are just a complex collection of these primitive data types. That said, a primitive is not an object in Java, nor does it represent an object. A primitive is just a single value in memory, such as a number or character.

Türkçe: Java'da Java primitive type'ları denenen sekiz built-in data type vardır. Bütün Java object'leri bu primitive data type'ların karmaşık birleşimlerinden oluştuğu için bu sekiz type, object'lerin yapı taşlarıdır. Bununla birlikte primitive, Java'da bir object değildir ve bir object'i temsil etmez; bellekteki sayı veya karakter gibi tek bir value'dur.

The Primitive Types

Türkçe başlık: Primitive Türler

English: The exam assumes you are well versed in the eight primitive data types, their relative sizes, and what can be stored in them. Table 1.6 shows the Java primitive types together with their size in bits and the range of values that each holds.

Türkçe: Sınav, sekiz temel veri türü, bunların göreceli boyutları ve bunlarda nelerin saklanabileceği konusunda bilgili olduğunuzu varsayar. Tablo 1.6, Java temel türlerini, bit cinsinden boyutları ve her birinin tuttuğu değer aralığıyla birlikte gösterir.

English table caption: TABLE 1.6 Primitive types

Türkçe tablo başlığı: TABLO 1.6 Primitive type'lar

Keyword	Type / Tür	Min value	Max value	Default value	Example
<code>boolean</code>	true or false / true veya false	n/a	n/a	<code>false</code>	<code>true</code>
<code>byte</code>	8-bit integral value	-128	127	0	123
<code>short</code>	16-bit integral value	-32,768	32,767	0	123
<code>int</code>	32-bit integral value	-2,147,483,648	2,147,483,647	0	123

Keyword	Type / Tür	Min value	Max value	Default value	Example
<code>long</code>	64-bit integral value	-2^{63}	$2^{63} - 1$	<code>0L</code>	<code>123L</code>
<code>float</code>	32-bit floating-point value	n/a	n/a	<code>0.0f</code>	<code>123.45f</code>
<code>double</code>	64-bit floating-point value	n/a	n/a	<code>0.0</code>	<code>123.456</code>
<code>char</code>	16-bit Unicode value	0	65,535	<code>\u0000</code>	<code>'a'</code>

Is String a Primitive?

Türkçe başlık: String Primitive midir?

English: No, it is not. That said, String is often mistaken for a ninth primitive because Java includes built-in support for String literals and operators. You learn more about String in Chapter 4, but for now, just remember it's an object, not a primitive.

Türkçe: Hayır. Java, `String` literal'ları ve operator'ları için yerleşik destek sağladığından `String` bazen yanlışlıkla dokuzuncu primitive type sanılır. Bölüm 4'te `String` hakkında daha fazla bilgi edineceksiniz; şimdilik onun primitive değil, bir object olduğunu unutmayın.

Kaynak PDF sayfası 28

English: There's a lot of information in Table 1.6. Let's look at some key points:

Türkçe: Tablo 1.6'da pek çok bilgi var. Bazı önemli noktalara bakalım:

English: - The byte, short, int, and long types are used for integer values without decimal points.

Türkçe: - Ondalık noktaları olmayan tamsayı değerleri için byte, short, int ve long türleri kullanılır.

English: - Each numeric type uses twice as many bits as the smaller similar type. For example, short uses twice as many bits as byte does.

Türkçe: - Her numeric type, kendisinden küçük benzer type'ın iki katı bit kullanır. Örneğin `short`, `byte`in iki katı bit kullanır.

English: - All of the numeric types are signed and reserve one of their bits to cover a negative range. For example, instead of byte covering 0 to 255 (or even 1 to 256) it actually covers -128 to 127.

Türkçe: - Bütün numeric type'lar signed'dır ve bitlerinden birini negatif aralık için ayırır. Örneğin `byte`, 0–255 (veya 1–256) yerine -128–127 aralığını kapsar.

English: - A float requires the letter f or F following the number so Java knows it is a float.

Türkçe: - Bir `float` literal'ın sonuna `f` veya `F` yazılmalıdır; böylece Java değer `float` olduğunu bilir.

English: Without an f or F, Java interprets a decimal value as a double.

Türkçe: `f` veya `F` bulunmazsa Java ondalıklı literal'ı `double` olarak yorumlar.

English: - A long requires the letter l or L following the number so Java knows it is a long.

Türkçe: - Bir `long` literal'ın sonuna `l` veya `L` yazılmalıdır; böylece Java değer `long` olduğunu bilir.

English: Without an I or L, Java interprets a number without a decimal point as an int in most scenarios.

Türkçe: `l` veya `L` bulunmazsa Java, ondalık noktası olmayan literal'ı çoğu durumda `int` olarak yorumlar.

English: You won't be asked about the exact sizes of these types, although you should have a general idea of the size of smaller types like byte and short. A common question among newer Java developers is, what is the bit size of boolean? The answer is, it is not specified and is dependent on the JVM where the code is being executed.

Türkçe: `byte` ve `short` gibi daha küçük type'ların boyutları hakkında genel bir fikriniz bulunmalı; ancak bu type'ların exact size değerleri sınavda sorulmaz. Yeni Java geliştiricilerinin sık sorduğu sorulardan biri `boolean`'ın bit size'dır. Bunun kesin bir değeri belirtilmemiştir ve kodun çalıştığı JVM'ye bağlıdır.

Signed and Unsigned: short and char

Türkçe başlık: Signed ve Unsigned: `short` ve `char`

English: For the exam, you should be aware that short and char are closely related, as both are stored as integral types with the same 16-bit length. The primary difference is that short is signed, which means it splits its range across the positive and negative integers. Alternatively, char is unsigned, which means its range is strictly positive, including 0.

Türkçe: Sınav için `short` ve `char` type'larının yakından ilişkili olduğunu bilmelisiniz; ikisi de 16 bit uzunluğa sahip integral type olarak saklanır. Temel fark, `short`'un signed olması ve range'ini positive ile negative integer'lar arasında bölmeleridir. `char` ise unsigned'dır; range'i 0 dâhil negatif olmayan değerlerden oluşur.

English: Often, short and char values can be cast to one another because the underlying data size is the same. You learn more about casting in Chapter 2, "Operators."

Türkçe: Altteki data size aynı olduğundan `short` ve `char` value'lar çoğu zaman birbirine cast edilebilir. Casting hakkında daha fazlasını Chapter 2 "Operators" içinde öğreneceksiniz.

Writing Literals

Türkçe başlık: Literal Yazmak

English: There are a few more things you should know about numeric primitives. When a number is present in the code, it is called a literal. By default, Java assumes you are defining an int value with a numeric literal. In the following example, the number listed is bigger than what fits in an int. Remember, you aren't expected to memorize the maximum value for an int. The exam will include it in the question if it comes up.

Türkçe: Numeric primitive'ler hakkında birkaç ayrıntı daha vardır. Kodda doğrudan yazılan sayıya literal denir. Java, numeric literal'ın varsayılan olarak `int` olduğunu kabul eder. Aşağıdaki sayı `int` aralığına sığmaz. Bir `int`'in maksimum değerini ezberlemeniz beklenmez; gerekirse sınav sorusunda verilir.

```
long max = 3123456789; // DOES NOT COMPILE
```

Kaynak PDF sayfası 29

English: Java complains the number is out of range. And it is— for an int. However, we don't have an int. The solution is to add the character L to the number:

Türkçe: Java, sayının aralık dışında olduğundan şikayet ediyor. Ve bu — bir int için. Ancak int'imiz yok. Çözüm, L karakterini sayıya eklemektir:

```
long max = 3123456789L; // Now Java knows it is a long
```

English: Alternatively, you could add a lowercase l to the number. But please use the uppercase L.

Türkçe: Alternatif olarak sayıya küçük l harfi ekleyebilirsiniz. Ama lütfen büyük L harfini kullanın.

English: The lowercase l looks like the number 1.

Türkçe: Küçük l harfi 1 rakamına benziyor.

English: Another way to specify numbers is to change the “base.” When you learned how to count, you studied the digits 0–9. This numbering system is called base 10 since there are 10 possible values for each digit. It is also known as the decimal number system. Java allows you to specify digits in several other formats:

Türkçe: Sayıları belirtmenin başka bir yolu da “tabanı” değiştirmektir. Saymayı öğrendiğinizde 0'dan 9'a kadar olan rakamları incelediniz. Her rakam için 10 olası değer olduğundan bu numaralandırma sistemine 10 tabanı denir. Ondalık sayı sistemi olarak da bilinir. Java, rakamları başka formatlarda belirtmenize olanak tanır:

English: - Octal (digits 0–7), which uses the number 0 as a prefix— for example, 017.

Türkçe: - Sekizli (0-7 arasındaki rakamlar), örnek olarak 0 sayısını kullanır; örneğin, 017.

English: - Hexadecimal (digits 0–9 and letters A–F/a–f), which uses 0x or 0X as a prefix— for example, 0xFF, 0xff, 0xFF. Hexadecimal is case insensitive, so all of these examples mean the same value.

Türkçe: - Ön ek olarak 0x veya 0X kullanan onaltılık sistem (0-9 arasındaki rakamlar ve A–F harfleri / a–f), örneğin 0xFF, 0xff, 0xFF. Onaltılık sayı büyük/küçük harfe duyarlı değildir, dolayısıyla bu örneklerin tümü aynı değeri ifade eder.

English: - Binary (digits 0–1), which uses the number 0 followed by b or B as a prefix— for example, 0b10, 0B10.

Türkçe: - Ön ek olarak 0 sayısını ve ardından b veya B'yi kullanan ikili (0–1 rakamları) — örneğin, 0b10, 0B10.

English: You won't need to convert between number systems on the exam. You'll have to recognize valid literal values that can be assigned to numbers.

Türkçe: Sınavda number system'lar arasında conversion yapmanız gerekmez. Number'lara atanabilen geçerli literal value'ları tanımanız gerekir.

Literals and the Underscore Character

Türkçe başlık: Literal'lar ve Alt Çizgi Karakteri

English: The last thing you need to know about numeric literals is that you can have underscores in numbers to make them easier to read:

Türkçe: Numeric literal'ların okunmasını kolaylaştırmak için alt çizgi (_) kullanabilirsiniz:

```
int million1 = 1000000;  
int million2 = 1_000_000;
```

English: We'd rather be reading the latter one because the zeros don't run together. You can add underscores anywhere except at the beginning of a literal, the end of a literal, right before a decimal point, or right after a decimal point. You can even place multiple underscore characters next to each other, although we don't recommend it.

Türkçe: Sıfırlar birlikte gitmediği için ikincisini okumayı tercih ederiz. Bir sabit değerin başlangıcı, bir sabit değerin sonu, ondalık ayırıcının hemen öncesi veya ondalık ayırıcının hemen sonrası dışında herhangi bir yere alt çizgi ekleyebilirsiniz. Hatta tavsiye etmesek de birden fazla alt çizgi karakterini yan yana bile yerleştirebilirsiniz.

English: Let's look at a few examples:

Türkçe: Birkaç örneğe bakalım:

```
double notAtStart = _1000.00; // DOES NOT COMPILE  
double notAtEnd = 1000.00_; // DOES NOT COMPILE  
double notByDecimal = 1000_.00; // DOES NOT COMPILE  
double annoyingButLegal = 1_00_0.0_0; // Ugly, but compiles  
double reallyUgly = 1_____2; // Also compiles
```

Using Reference Types

Türkçe başlık: Referans Türlerini Kullanma

English: A reference type refers to an object (an instance of a class). Unlike primitive types that hold their values in the memory where the variable is allocated, references do not hold the value of the object they refer to. Instead, a reference "points" to an object by storing the memory

Türkçe: Referans türü bir object'i (bir class'ın örneği) ifade eder. Değerlerini variable'ın tahsis edildiği bellekte tutan primitive türlerin aksine referanslar, atıfta bulundukları object'in değerini tutmaz. Bunun yerine, bir referans, hafızayı saklayarak bir object'e "işaret eder"

Kaynak PDF sayfası 30

English: address where the object is located, a concept referred to as a pointer. Unlike other languages, Java does not allow you to learn what the physical memory address is. You can only use the reference to refer to the object.

Türkçe: Reference, object'in bulunduğu bellek adresini saklar; bu kavram pointer olarak adlandırılır. Diğer bazı dillerden farklı olarak Java fiziksel bellek adresini öğrenmenize izin vermez. Reference yalnızca object'e erişmek için kullanılır.

English: Let's take a look at some examples that declare and initialize reference types. Suppose we declare a reference of type String:

Türkçe: Referans türlerini bildiren ve başlatan bazı örneklere göz atalım. String tipinde bir referans bildirdiğimizi varsayalım:

```
String greeting;
```

English: The greeting variable is a reference that can only point to a String object. A value is assigned to a reference in one of two ways:

Türkçe: `greeting` variable'ı yalnızca bir `String` object'ine işaret edebilen bir reference'tır. Bir reference'a iki yoldan biriyle value atanır:

English: - A reference can be assigned to another object of the same or compatible type.

Türkçe: - Aynı veya uyumlu türdeki başka bir object'e bir referans atanabilir.

English: - A reference can be assigned to a new object using the new keyword.

Türkçe: - `new` keyword'ü kullanılarak yeni bir object'e ait reference atanabilir.

English: For example, the following statement assigns this reference to a new object:

Türkçe: Örneğin, aşağıdaki ifade bu referansı yeni bir object'e atar:

```
greeting = new String("How are you?");
```

English: The greeting reference points to a new String object, "How are you?". The String object does not have a name and can be accessed only via a corresponding reference.

Türkçe: `greeting` reference'ı yeni bir `String` object'ine, yani `"How are you?"` değerine işaret eder. `String` object'inin kendi adı yoktur; ona yalnızca karşılık gelen reference üzerinden erişilebilir.

Distinguishing between Primitives and Reference Types

Türkçe başlık: İlkeleri ve Referans Türlerini Ayırmak

English: There are a few important differences you should know between primitives and reference types. First, notice that all the primitive types have lowercase type names. All classes that come with Java begin with uppercase. Although not required, it is a standard practice, and you should follow this convention for classes you create as well.

Türkçe: Primitive ve referans türleri arasında bilmeniz gereken birkaç önemli fark vardır. İlk olarak, tüm primitive türlerin küçük harfli tür adlarına sahip olduğuna dikkat edin. Java ile gelen tüm class'lar büyük harfle başlar. Zorunlu olmasa da standart bir uygulamadır ve oluşturduğunuz class'lar için de bu kurala uymalısınız.

English: Next, reference types can be used to call methods, assuming the reference is not null.

Türkçe: Ardından reference type'lar, reference'ın `null` olmadığı varsayımıyla method çağırmak için kullanılabilir.

English: Primitives do not have methods declared on them. In this example, we can call a method on reference since it is of a reference type. You can tell `length` is a method because it has `()` after it. See if you can understand why the following snippet does not compile:

Türkçe: Primitive type'larda tanımlı method bulunmaz. Bu örnekte `reference`, bir reference type olduğu için onun üzerinde method çağırabiliriz. `length()` ifadesindeki parantezler, `length` adının bir method olduğunu gösterir. Aşağıdaki kod parçasının neden derlenmediğini bulmaya çalışın:

```
4: String reference = "hello";
5: int len = reference.length();
6: int bad = len.length(); // DOES NOT COMPILE
```

English: Line 6 is gibberish. No methods exist on `len` because it is an `int` primitive. Primitives do not have methods. Remember, a `String` is not a primitive, so you can call methods like `length()` on a `String` reference, as we did on line 5.

Türkçe: 6. satır geçersizdir. `len`, `int` primitive type'ında olduğundan üzerinde çağırılacak hiçbir method yoktur. Primitive type'ların method'ları bulunmaz. `String` ise primitive değildir; bu nedenle 5. satırdaki gibi bir `String` referansı üzerinde `length()` çağırılabilir.

English: Finally, reference types can be assigned null, which means they do not currently refer to an object. Primitive types will give you a compiler error if you attempt to assign them null. In this example, `value` cannot point to null because it is of type `int`:

Türkçe: Son olarak reference type'lara `null` atanabilir; bu, o anda hiçbir object'e işaret etmedikleri anlamına gelir. Primitive type'a `null` atanmaya çalışılırsa compiler hatası oluşur. Bu örnekte `value`, `int` type'ında olduğu için `null` alamaz:

```
int value = null; // DOES NOT COMPILE
String name = null;
```

English: But what if you don't know the value of an int and want to assign it to null? In that case, you should use a numeric wrapper class, such as Integer, instead of int.

Türkçe: Peki ya bir int'in değerini bilmiyorsanız ve onu null değerine atamak istiyorsanız? Bu durumda int yerine Integer gibi sayısal bir wrapper class'ı kullanmalısınız.

Kaynak PDF sayfası 31

Creating Wrapper Classes

Türkçe başlık: Wrapper Class'ları Oluşturma

English: Each primitive type has a wrapper class, which is an object type that corresponds to the primitive. Table 1.7 lists all the wrapper classes along with how to create them.

Türkçe: Her temel türün, primitive öğeye karşılık gelen bir object türü olan bir wrapper class'ı vardır. Tablo 1.7'de tüm wrapper class'ları ve bunların nasıl oluşturulacağı listelenmektedir.

English table caption: TABLE 1.7 Wrapper classes

Türkçe tablo başlığı: TABLO 1.7 Wrapper class'ları

Primitive type	Wrapper class	Inherits <i>Number</i> ? / <i>Number</i> dan kalıtım alır mı?	Example of creating / Oluşturma örneği
<code>boolean</code>	<code>Boolean</code>	No / Hayır	<code>Boolean.valueOf(true)</code>
<code>byte</code>	<code>Byte</code>	Yes / Evet	<code>Byte.valueOf((byte) 1)</code>
<code>short</code>	<code>Short</code>	Yes / Evet	<code>Short.valueOf((short) 1)</code>
<code>int</code>	<code>Integer</code>	Yes / Evet	<code>Integer.valueOf(1)</code>
<code>long</code>	<code>Long</code>	Yes / Evet	<code>Long.valueOf(1)</code>
<code>float</code>	<code>Float</code>	Yes / Evet	<code>Float.valueOf((float) 1.0)</code>
<code>double</code>	<code>Double</code>	Yes / Evet	<code>Double.valueOf(1.0)</code>
<code>char</code>	<code>Character</code>	No / Hayır	<code>Character.valueOf('c')</code>

English: There is also a `valueOf()` variant that converts a `String` into the wrapper class.

Türkçe: Ayrıca bir `String`i wrapper class'a dönüştüren bir `valueOf()` variant'ı da vardır.

English: For example:

Türkçe: Örneğin:

```
int primitive = Integer.parseInt("123");
Integer wrapper = Integer.valueOf("123");
```

English: The first line converts a String to an int primitive. The second converts a String to an Integer wrapper class.

Türkçe: İlk satır bir `String` değerini `int` primitive'ine dönüştürür. İkinci satır ise bir `String` değerini `Integer` wrapper class'ına dönüştürür.

English: All of the numeric classes in Table 1.7 extend the Number class, which means they all come with some useful helper methods: `byteValue()`, `shortValue()`, `intValue()`, `longValue()`, `floatValue()`, and `doubleValue()`. The Boolean and Character wrapper classes include `booleanValue()` and `charValue()`, respectively.

Türkçe: Tablo 1.7'deki bütün numeric class'lar `Number` class'ını extend eder; dolayısıyla `byteValue()`, `shortValue()`, `intValue()`, `longValue()`, `floatValue()` ve `doubleValue()` helper method'larına sahiptir. `Boolean` ve `Character` wrapper class'ları ise sırasıyla `booleanValue()` ve `charValue()` method'larını içerir.

English: As you probably guessed, these methods return the primitive value of a wrapper instance, in the type requested.

Türkçe: Muhtemelen tahmin ettiğiniz gibi, bu method'lar, istenen türde bir wrapper örneğinin primitive değerini döndürür.

```
Double apple = Double.valueOf("200.99");
System.out.println(apple.byteValue()); // -56
System.out.println(apple.intValue()); // 200
System.out.println(apple.doubleValue()); // 200.99
```

Kaynak PDF sayfası 32

English: These helper methods do their best to convert values but can result in a loss of precision. In the first example, there is no 200 in byte, so it wraps around to -56. In the second example, the value is truncated, which means all of the numbers after the decimal are dropped. In Chapter 5, we apply autoboxing and unboxing to show how easy Java makes it to work with primitive and wrapper values.

Türkçe: Bu helper method'lar value'ları dönüştürmeye çalışır; ancak precision loss oluşabilir. İlk örnekte `byte` aralığı 200'ü içermez, bu nedenle value wraparound ile -56 olur. İkinci örnekte value truncate edilir; yani decimal point'ten sonraki bütün basamaklar atılır. Bölüm 5'te autoboxing ve unboxing ile Java'nın primitive ve wrapper value'larla çalışmayı nasıl kolaylaştırdığı gösterilecektir.

English: Some of the wrapper classes contain additional helper methods for working with numbers. You don't need to memorize these; you can assume any you are given are valid. For example, Integer has:

Türkçe: Bazı wrapper class'lar sayılarla çalışmak için ek yardımcı method'lar içerir. Bunları ezberlemeniz gerekmez; soruda verilenlerin geçerli olduğunu varsayabilirsiniz. Örneğin `Integer` şu method'ları içerir:

English: - `max(int num1, int num2)`, which returns the largest of the two numbers

Türkçe: - `max(int num1, int num2)`, iki sayıdan büyük olanı döndürür

English: - `min(int num1, int num2)`, which returns the smallest of the two numbers

Türkçe: - `min(int num1, int num2)`, iki sayıdan küçük olanı döndürür

English: - `sum(int num1, int num2)`, which adds the two numbers

Türkçe: - `sum(int num1, int num2)`, iki sayının toplamını döndürür

Defining Text Blocks

Türkçe başlık: Metin Bloklarını Tanımlama

English: Earlier we saw a simple String with the value "hello". What if we want to have a String with something more complicated? For example, let's figure out how to create a String with this value:

Türkçe: Daha önce "hello" value'suna sahip basit bir `String` görmüştük. Daha karmaşık bir `String` oluşturmak istersek ne olur? Örneğin şu value'yu nasıl oluşturacağımıza bakalım:

```
"Java Study Guide"
by Scott & Jeanne
```

English: Building this as a String requires two things you haven't learned yet. The syntax `"` lets you say you want a `"` rather than to end the String, and `\n` says you want a new line. Both of these are called escape characters because the backslash provides a special meaning. With these two new skills, we can write

Türkçe: Bunu bir `String` olarak oluşturmak, henüz öğrenmediğiniz iki şeyi gerektirir. `"` syntax'ı, `String`i sonlandırmak yerine `"` karakterini istediğinizi belirtir; `\n` ise yeni bir satır istediğinizi söyler. Backslash özel bir anlam kazandırdığı için bunların ikisine de escape character denir. Bu iki yeni bilgiyle şunu yazabiliriz:

```
String eyeTest = "\"Java Study Guide\"\n by Scott & Jeanne";
```

English: While this does work, it is hard to read. Luckily, Java has text blocks, also known as multiline strings. See Figure 1.3 for the text block equivalent.

Türkçe: Bu işe yarasa da okunması zordur. Neyse ki Java'da çok satırlı dizeler olarak da bilinen metin blokları vardır. Metin bloğu eşdeğeri için Şekil 1.3'e bakınız.

English figure caption: FIGURE 1.3 Text block

Türkçe şekil başlığı: ŞEKİL 1.3 Metin bloğu

```
String textBlock = """
    Java Study Guide
    by Scott & Jeanne""";
```

English: The figure labels the opening delimiter as “Start text block” and the closing delimiter as “End text block.” It also distinguishes incidental whitespace from essential whitespace.

Türkçe: Şekil, opening delimiter'ı “Start text block”, closing delimiter'ı ise “End text block” olarak işaretler. Ayrıca incidental whitespace ile essential whitespace'i ayırır.

English: A text block starts and ends with three double quotes ("""), and the contents don't need to be escaped. This is much easier to read. Notice how the type is still String. This means the methods you learn about in Chapter 4 for String work for both a regular String and a text block.

Türkçe: Text block üç double quote (""") ile başlar ve biter; içeriğin escape edilmesi gerekmez. Bu biçimi okumak çok daha kolaydır. Type'ın hâlâ String olduğuna dikkat edin. Bu nedenle Chapter 4'te String için öğreneceğiniz method'lar hem normal String hem de text block ile çalışır.

Kaynak PDF sayfası 33

English: You might have noticed the words incidental and essential whitespace in the figure.

Türkçe: Şekilde incidental whitespace (biçimsel boşluk) ve essential whitespace (değerin parçası olan boşluk) terimlerini fark etmiş olabilirsiniz.

English: What's that? Essential whitespace is part of your String and is important to you. Incidental whitespace just happens to be there to make the code easier to read. You can reformat your code and change the amount of incidental whitespace without any impact on your String value.

Türkçe: Essential whitespace, String value'sunun parçasıdır ve korunur. Incidental whitespace ise source code'u okunaklı kılmak için bulunur. String value'sunu değiştirmeden kodu yeniden biçimlendirip incidental whitespace miktarını değiştirebilirsiniz.

English: Imagine a vertical line drawn on the leftmost non-whitespace character in your text block. Everything to the left of it is incidental whitespace, and everything to the right is essential whitespace. Let's try an example. How many lines does this output, and how many incidental and essential whitespace characters begin each line?

Türkçe: Text block'taki en soldaki whitespace olmayan karakterden dikey bir çizgi geçtiğini düşünün. Çizginin solundaki whitespace incidental, sağındaki whitespace essential kabul edilir. Şu örnekte output kaç satırdır ve her satırın başında kaç incidental ve essential whitespace karakteri vardır?

```
14: String pyramid = ""
15:  *
16:  *
17:  *
18:  """;
19: System.out.print(pyramid);
```

English: There are four lines of output. Lines 15–17 have stars. Line 18 is a line without any characters. The closing triple " would have needed to be on line 17 if we didn't want that blank line. There are no incidental whitespace characters here. The closing "" on line 18 are the leftmost characters, so the line is drawn at the leftmost position. Line 15 has two essential whitespace characters to begin the line, and line 16 has one. That whitespace fills in the line drawn to match line 18.

Türkçe: Output dört satırdır. 15–17. satırlarda yıldız bulunur; 18. satır ise karakter içermeyen boş satırdır. Bu boş satır istenmeseydi kapanış delimiter'ı (" ") 17. satırda olmalıydı. Burada incidental whitespace yoktur; çünkü 18. satırdaki kapanış delimiter'ı en soldaki öğedir ve sınır çizgisi en sol konumdadır. 15. satır iki, 16. satır bir essential whitespace karakteriyle başlar. Bu whitespace, 18. satıra göre çizilen hizaya kadar olan alanı doldurur.

English: Table 1.8 shows some special formatting sequences and compares how they work in a regular String and a text block.

Türkçe: Tablo 1.8 bazı özel biçimlendirme sıralarını gösterir ve bunların normal bir String ve bir metin bloğunda nasıl çalıştıklarını karşılaştırır.

English table caption: TABLE 1.8 Text block formatting

Türkçe tablo başlığı: TABLO 1.8 Metin bloğu formatlaması

Formatting / Biçimlendirme	Meaning in regular <code>String</code> / Normal <code>String</code> içindeki anlamı	Meaning in text block / Text block içindeki anlamı
<code>\ "</code>	"	"
<code>\ " " "</code>	n/a – Invalid / Geçersiz	" " " "
<code>\ " \" \" "</code>	" " " "	" " " "
Space at end of line / Satır sonundaki space	Space	Ignored / Yok sayılır
<code>\s</code>	Two spaces (<code>\s</code> is a space and preserves leading space on the line) / İki space	Two spaces / İki space
<code>\</code> at end of line / Satır sonunda <code>\</code>	n/a – Invalid / Geçersiz	Omits new line on that line / O satırdaki newline'ı kaldırır

Kaynak PDF sayfası 34

English: Let's try a few examples. First, do you see why this doesn't compile?

Türkçe: Birkaç örnek deneyelim. Öncelikle bunun neden derlenmediğini anlıyor musunuz?

```
String block = """"doe""""; // DOES NOT COMPILE
```

English: Text blocks require a line break after the opening """, making this one invalid. Now let's try a valid one. How many lines do you think are in this text block?

Türkçe: Metin blokları """" açılışından sonra satır sonu gerektirir, bu da bunu geçersiz kılar. Şimdi geçerli bir tane deneyelim. Sizce bu metin bloğunda kaç satır var?

```
String block = ""
    doe \
    deer"";
```

English: Just one. The output is doe deer since the \ tells Java not to add a new line before deer.

Türkçe: Yalnızca bir satır vardır. Satır sonundaki \, Java'ya deer öncesinde yeni satır eklememesini söylediği için çıktı doe deer olur.

English: Let's try determining the number of lines in another text block:

Türkçe: Başka bir metin bloğundaki satır sayısını belirlemeyi deneyelim:

```
String block = ""
    doe \n
    deer
    "";
```

English: This time we have four lines. Since the text block has the closing "" on a separate line, we have three lines for the lines in the text block plus the explicit \n. Let's try one more.

Türkçe: Bu kez dört satır vardır. Text block'un closing "" delimiter'ı ayrı bir satırda bulunduğundan block için üç satır, explicit \n için de bir satır vardır. Bir örnek daha deneyelim.

English: What do you think this outputs?

Türkçe: Sizce bu ne sonuç verir?

```
String block = ""
    "doe\\"
    \n"deer"
    ""
    "";
System.out.print("*"+ block + "*");
```

English: The answer is:

Türkçe: Çıktı şöyledir:

```
* "doe"
  "deer"
*
```

English: All of the \ escape the ". There is one space of essential whitespace on the doe and deer lines. All the other leading whitespace is incidental whitespace.

Türkçe: Bütün \ " dizileri " karakterini escape eder. doe ve deer satırlarında birer essential whitespace vardır; satır başındaki diğer space'lerin tümü incidental whitespace'tir.

Declaring Variables

Türkçe başlık: Variable Declaration

English: You've seen some variables already. A variable is a name for a piece of memory that stores data. When you declare a variable, you need to state the variable type along with giving it a name. Giving a variable a value is called initializing a variable. To initialize a variable, you just type the variable name followed by an equal sign, followed by the desired value. This example shows declaring and initializing a variable in one line:

Türkçe: Daha önce bazı variable'lar gördünüz. Variable, veriyi saklayan bir bellek parçasına verilen addır. Bir variable declare ederken type'ını ve adını belirtmeniz gerekir. Variable'a value verme işlemine initialization denir. Initialize etmek için variable adını, eşittir işaretini ve istenen value'yu yazarsınız. Bu örnek bir variable'ın tek satırda declaration ve initialization işlemlerini gösterir:

```
String zooName = "The Best Zoo";
```

Kaynak PDF sayfası 35

English: In the following sections, we look at how to properly define variables in one or multiple lines.

Türkçe: Aşağıdaki bölümlerde variable'ların bir veya birden fazla satırda nasıl doğru şekilde tanımlanabileceğine bakacağız.

Identifying Identifiers

Türkçe başlık: Tanımlayıcıları Tanımlama

English: It probably comes as no surprise to you that Java has precise rules about identifier names.

Türkçe: Java'nın tanımlayıcı adlarla ilgili kesin kurallara sahip olması muhtemelen size sürpriz olmayacaktır.

English: An identifier is the name of a variable, method, class, interface, or package. Luckily, the rules for identifiers for variables apply to all of the other types that you are free to name.

Türkçe: Identifier; bir variable, method, class, interface (arayüz) veya package (paket) adıdır. Variable identifier'ları için geçerli kurallar, adını seçebildiğiniz diğer bütün type'lar için de geçerlidir.

English: There are only four rules to remember for legal identifiers:

Türkçe: Geçerli identifier'lar için hatırlanması gereken yalnızca dört kural vardır:

English: - Identifiers must begin with a letter, a currency symbol, or a _ symbol. Currency symbols include dollar (\$), yuan (¥), euro (€), and so on.

Türkçe: - Tanımlayıcılar bir harfle, para birimi simgesiyle veya _ simgesiyle başlamalıdır. Para birimi simgeleri arasında dolar (\$), yuan (¥), euro (€) vb. bulunur.

English: - Identifiers can include numbers but not start with them.

Türkçe: - Identifier'lar sayı içerebilir; ancak sayıyla başlayamaz.

English: - A single underscore `_` is not allowed as an identifier.

Türkçe: - Tanımlayıcı olarak tek bir alt çizgiye `_` izin verilmez.

English: - You cannot use the same name as a Java reserved word. A reserved word is a special word that Java has held aside so that you are not allowed to use it. Remember that Java is case sensitive, so you can use versions of the keywords that only differ in case. Please don't, though.

Türkçe: - Bir Java reserved word ile aynı adı kullanamazsınız. Reserved word, Java'nın identifier olarak kullanılmaması için ayırdığı özel bir sözcüktür. Java case-sensitive olduğundan keyword'lerin yalnızca harf büyüklüğü farklı olan biçimleri kullanılabilir; yine de bunu yapmayın.

English: Don't worry— you won't need to memorize the full list of reserved words. The exam will only ask you about ones that are commonly used, such as `class` and `for`. Table 1.9 lists all of the reserved words in Java.

Türkçe: Endişelenmeyin, ayrılmış kelimelerin tam listesini ezberlemenize gerek kalmayacak. Sınavda size yalnızca `class` ve `for` gibi yaygın olarak kullanılanlar sorulacaktır. Tablo 1.9 Java'da ayrılmış tüm kelimeleri listeler.

English table caption: TABLE 1.9 Reserved words

Türkçe tablo başlığı: TABLO 1.9 Ayrılmış kelimeler

<code>abstract</code>	<code>assert</code>	<code>boolean</code>	<code>break</code>	<code>byte</code>
<code>case</code>	<code>catch</code>	<code>char</code>	<code>class</code>	<code>const</code> [*]
<code>continue</code>	<code>default</code>	<code>do</code>	<code>double</code>	<code>else</code>
<code>enum</code>	<code>extends</code>	<code>final</code>	<code>finally</code>	<code>float</code>
<code>for</code>	<code>goto</code> [*]	<code>if</code>	<code>implements</code>	<code>import</code>
<code>instanceof</code>	<code>int</code>	<code>interface</code>	<code>long</code>	<code>native</code>
<code>new</code>	<code>package</code>	<code>private</code>	<code>protected</code>	<code>public</code>
<code>return</code>	<code>short</code>	<code>static</code>	<code>strictfp</code>	<code>super</code>
<code>switch</code>	<code>synchronized</code>	<code>this</code>	<code>throw</code>	<code>throws</code>
<code>transient</code>	<code>try</code>	<code>void</code>	<code>volatile</code>	<code>while</code>

English: ^{*} The reserved words `const` and `goto` aren't actually used in Java. They are reserved so that people coming from other programming languages don't use them by accident—and, in theory, in case Java wants to use them one day.

Türkçe: ^{*} `const` ve `goto` reserved word'leri Java'da fiilen kullanılmaz. Başka programming language'lardan gelenlerin bunları yanlışlıkla kullanmaması ve teorik olarak Java'nın bir gün kullanmak istemesi ihtimaline karşı ayrılmışlardır.

Kaynak PDF sayfası 36

English: There are other names that you can't use. For example, true, false, and null are literal values, so they can't be variable names. Additionally, there are contextual keywords like module in Chapter 12.

Prepare to be tested on these rules. The following examples are legal:

Türkçe: Kullanamayacağınız başka adlar da vardır. Örneğin true, false ve null literal value oldukları için variable adı olamaz. Ayrıca Bölüm 12'deki module gibi contextual keyword'ler vardır. Bu kuralların sınavda sorulmasına hazırlıklı olun. Aşağıdaki örnekler geçerlidir:

```
long okIdentifier;
float $OK2Identifier;
boolean _alsoOK1d3ntifi3r;
char __SStilloKbutKnotsonice$;
```

English: These examples are not legal:

Türkçe: Bu örnekler geçerli değildir:

```
int 3DPointClass; // identifiers cannot begin with a number
byte hollywood@vine; // @ is not a letter, digit, $ or _
String *$coffee; // * is not a letter, digit, $ or _
double public; // public is a reserved word
short _; // a single underscore is not allowed
```

camelCase and snake_case

Türkçe başlık: camelCase ve snake_case

English: Although you can do crazy things with identifier names, please don't. Java has conventions so that code is readable and consistent. For example, camel case has the first letter of each word capitalized. Method and variable names are typically written in camel case with the first letter lowercase, such as toUpper(). Class and interface names are also written in camel case, with the first letter uppercase, such as ArrayList.

Türkçe: Identifier adlarında çok farklı biçimler kullanabilseniz de bunu yapmayın. Java, kodu okunabilir ve tutarlı tutan adlandırma kurallarına sahiptir. camelCase'te her sözcüğün ilk harfi büyüktür. Method ve variable adları genellikle toUpper() gibi ilk harfi küçük olan camelCase ile; class ve interface adları ise ArrayList gibi ilk harfi büyük olan camelCase (PascalCase) ile yazılır.

English: Another style is called snake case. It simply uses an underscore (_) to separate words.

Türkçe: Başka bir style snake_case olarak adlandırılır. Sözcükleri ayırmak için underscore (_) kullanılır.

English: Java generally uses uppercase snake case for constants and enum values, such as NUMBER_FLAGS.

Türkçe: Java, constant ve enum value'lar için genellikle NUMBER_FLAGS örneğindeki gibi uppercase snake case kullanır.

English: The exam will not always follow these conventions to make questions about identifiers trickier. By contrast, questions on other topics generally do follow standard conventions. We recommend you follow these conventions on the job.

Türkçe: Sınav, tanımlayıcılarla ilgili soruları daha karmaşık hale getirmek için her zaman bu kurallara uymayacaktır. Bunun tersine, diğer konulardaki sorular genellikle standart kurallara uygundur. İşyerinde bu kurallara uymanızı öneririz.

Declaring Multiple Variables

Türkçe başlık: Birden Çok Variable Declare Etme

English: You can also declare and initialize multiple variables in the same statement. How many variables do you think are declared and initialized in the following example?

Türkçe: Aynı ifadede birden fazla variable'ı de bildirebilir ve başlatabilirsiniz. Aşağıdaki örnekte kaç variable'ın bildirildiğini ve başlatıldığını düşünüyorsunuz?

```
void sandFence() {  
    String s1, s2;  
    String s3 = "yes", s4 = "no";  
}
```

Kaynak PDF sayfası 37

English: Four String variables were declared: s1, s2, s3, and s4. You can declare many variables in the same declaration as long as they are all of the same type. You can also initialize any or all of those values inline. In the previous example, we have two initialized variables: s3 and s4. The other two variables remain declared but not yet initialized.

Türkçe: Dört `String` variable declare edilmiştir: `s1`, `s2`, `s3` ve `s4`. Aynı type'ta oldukları sürece birden çok variable'ı aynı declaration içinde declare edebilirsiniz. Bunların herhangi birini veya tümünü aynı satırda initialize edebilirsiniz. Önceki örnekte `s3` ve `s4` initialize edilmiştir; diğer iki variable declare edilmiş fakat henüz initialize edilmemiştir.

English: This is where it gets tricky. Pay attention to tricky things! The exam will attempt to trick you. Again, how many variables do you think are declared and initialized in the following code?

Türkçe: Burası işin zorlaştığı yer. Zor şeylere dikkat edin! Sınav sizi kandırmaya çalışacaktır. Yine, sizce aşağıdaki kodda kaç variable bildirilmiş ve başlatılmıştır?

```
void paintFence() {  
    int i1, i2, i3 = 0;  
}
```

English: As you should expect, three variables were declared: i1, i2, and i3. However, only one of those values was initialized: i3. The other two remain declared but not yet initialized.

Türkçe: Beklediğiniz gibi üç variable bildirildi: `i1`, `i2` ve `i3`. Ancak bu değerlerden yalnızca biri başlatıldı: `i3`. Diğer ikisi bildirilmiş durumda ancak henüz başlatılmadı.

English: That's the trick. Each snippet separated by a comma is a little declaration of its own. The initialization of i3 only applies to i3. It doesn't have anything to do with i1 or i2 despite being in the same statement. As you will see in the next section, you can't actually use i1 or i2 until they have been initialized.

Türkçe: İşin püf noktası bu. Virgülle ayrılmış her parça, kendine ait küçük bir beyandır. i3'ün başlatılması yalnızca i3 için geçerlidir. Aynı açıklamada olmasına rağmen i1 veya i2 ile hiçbir ilgisi yoktur. Bir sonraki bölümde göreceğiniz gibi, aslında başlatılana kadar i1 veya i2'yi kullanamazsınız.

English: Another way the exam could try to trick you is to show you code like this line:

Türkçe: Sınavın sizi kandırmaya çalışmasının bir başka yolu da size şu satırdaki gibi bir kod göstermektir:

```
int num, String value; // DOES NOT COMPILE
```

English: This code doesn't compile because it tries to declare multiple variables of different types in the same statement. The shortcut to declare multiple variables in the same statement is legal only when they share a type.

Türkçe: Bu kod derlenmiyor çünkü aynı ifadede farklı türden birden fazla variable'ı bildirmeye çalışıyor. Aynı ifadede birden fazla variable bildiriminin kısayolu, yalnızca bir türü paylaştıklarında yasaldır.

English: Legal, valid, and compiles are all synonyms in the Java exam world. We try to use all the terminology you could encounter on the exam.

Türkçe: Java sınavı bağlamında *legal*, *valid* ve *compiles* (derlenir) aynı anlamda kullanılır. Sınavda karşılaşılabileceğiniz bütün terminolojiyi kullanmaya çalışıyoruz.

English: To make sure you understand this, see if you can figure out which of the following are legal declarations:

Türkçe: Konuyu anladığınızdan emin olmak için aşağıdaki declaration'lardan hangilerinin geçerli olduğunu belirleyin:

```
4: boolean b1, b2;
5: String s1 = "1", s2;
6: double d1, double d2;
7: int i1; int i2;
8: int i3; i4;
```

English: Lines 4 and 5 are legal. They each declare two variables. Line 4 doesn't initialize either variable, and line 5 initializes only one. Line 7 is also legal. Although int does appear twice, each one is in a separate statement. A semicolon (;) separates statements in Java. It just so happens there are two completely different statements on the same line.

Türkçe: 4. ve 5. satırlar geçerlidir; her biri iki variable declare eder. 4. satırdaki iki variable da initialize edilmez, 5. satırda ise yalnızca biri initialize edilir. 7. satır da geçerlidir. *int* iki kez görünür, ancak her biri ayrı bir statement içindedir. Noktalı virgül (;) Java statement'larını ayırır; dolayısıyla aynı satırda birbirinden bağımsız iki statement vardır.

English: Line 6 is not legal. Java does not allow you to declare two different types in the same statement. Wait a minute! Variables d1 and d2 are the same type. They are both of type double. Although that's true, it still isn't allowed. If you want to declare multiple variables in the same statement, they must share the same type declaration and not repeat it.

Türkçe: 6. satır geçersizdir. Java aynı statement içinde iki farklı type declaration'ına izin vermez. d1 ile d2 aynı double type'ında olsa bile type'ı ikinci kez yazmak geçersizdir. Aynı statement'ta birden fazla variable declare edilecekse tek bir ortak type declaration'ı kullanılmalıdır.

Kaynak PDF sayfası 38

English: Line 8 is not legal. Again, we have two completely different statements on the same line.

Türkçe: 8. satır yasal değil. Yine aynı satırda tamamen farklı iki ifademiz var.

English: The second one on line 8 is not a valid declaration because it omits the type. When you see an oddly placed semicolon on the exam, pretend the code is on separate lines and think about whether the code compiles that way. In this case, the last two lines of code could be rewritten as follows:

Türkçe: 8. satırdaki ikincisi geçerli bir bildirim değil çünkü türü atlıyor. Sınavda garip bir şekilde yerleştirilmiş noktalı virgül gördüğünüzde, kodun ayrı satırlarda olduğunu düşünün ve kodun bu şekilde derlenip derlenmediğini düşünün. Bu durumda kodun son iki satırı şu şekilde yeniden yazılabilir:

```
int i1;
int i2;
int i3;
i4;
```

English: Looking at the last line on its own, you can easily see that the declaration is invalid. And yes, the exam really does cram multiple statements onto the same line— partly to try to trick you and partly to fit more code on the screen. In the real world, please limit yourself to one declaration per statement and line. Your teammates will thank you for the readable code.

Türkçe: Son satıra tek başına bakıldığında beyanın geçersiz olduğunu rahatlıkla görebilirsiniz. Ve evet, sınav gerçekten de birden fazla ifadeyi aynı satıra sığdırıyor; kısmen sizi kandırmaya çalışmak, kısmen de ekrana daha fazla kod sığdırmak için. Gerçek dünyada lütfen kendinizi ifade ve satır başına bir bildirimle sınırlayın. Takım arkadaşlarınız okunabilir kod için size teşekkür edecek.

Initializing Variables

Türkçe başlık: Variable'ları Initialize Etme

English: Before you can use a variable, it needs a value. Some types of variables get this value set automatically, and others require the programmer to specify it. In the following sections, we look at the differences between the defaults for local, instance, and class variables.

Türkçe: Bir variable kullanılmadan önce bir value'ya sahip olmalıdır. Bazı variable type'larında bu value otomatik atanır; diğerlerinde programcı belirtmelidir. Aşağıdaki bölümlerde local, instance ve class variable'ların default value kuralları arasındaki farkları inceleyeceğiz.

Creating Local Variables

Türkçe başlık: Yerel Variable'lar Oluşturma

English: A local variable is a variable defined within a constructor, method, or initializer block. For simplicity, we focus primarily on local variables within methods in this section, although the rules for the others are the same.

Türkçe: Local variable, constructor, method veya initializer block içinde define edilen variable'dır. Kurallar diğer bağlamlarda da aynı olmakla birlikte burada kolaylık için öncelikle method içindeki local variable'lara odaklanıyoruz.

Final Local Variables

Türkçe başlık: `final` Local Variable'lar

English: The final keyword can be applied to local variables and is equivalent to declaring constants in other languages. Consider this example:

Türkçe: `final` keyword'ü local variable'lara uygulanabilir; bu, diğer dillerde constant declare etmeye benzer. Şu örneği inceleyin:

```
5: final int y = 10;
6: int x = 20;
7: y = x + 10; // DOES NOT COMPILE
```

English: Both variables are set, but y uses the final keyword. For this reason, line 7 triggers a compiler error since the value cannot be modified.

Türkçe: İki variable da initialize edilmiştir; ancak y, `final` keyword'ünü kullanır. Bu nedenle value değiştirilemediğinden 7. satır compiler error oluşturur.

English: The final modifier can also be applied to local variable references. The following example uses an `int[]` array object, which you learn about in Chapter 4.

Türkçe: `final` modifier local variable reference'larına da uygulanabilir. Aşağıdaki örnekte Bölüm 4'te ele alınacak bir `int[]` array object'i kullanılır.

```
5: final int[] favoriteNumbers = new int[10];
6: favoriteNumbers[0] = 10;
```

Kaynak PDF sayfası 39

```
7: favoriteNumbers[1] = 20;
8: favoriteNumbers = null; // DOES NOT COMPILE
```

English: Notice that we can modify the content, or data, in the array. The compiler error isn't until line 8, when we try to change the value of the reference favoriteNumbers.

Türkçe: Array'in içeriğini değiştirebildiğimize dikkat edin. Compiler error ancak `favoriteNumbers` reference'ının value'sunu değiştirmeye çalıştığımız 8. satırda oluşur.

Uninitialized Local Variables

Türkçe başlık: Başlatılmamış Yerel Variable'lar

English: Local variables do not have a default value and must be initialized before use. Furthermore, the compiler will report an error if you try to read an uninitialized value. For example, the following code generates a compiler error:

Türkçe: Local variable'ların default value'su yoktur; kullanılmadan önce initialize edilmeleri gerekir. Initialize edilmemiş bir local variable okunmaya çalışılırsa compiler error verir. Örnek:

```
4: public int notValid() {
5:   int y = 10;
6:   int x;
7:   int reply = x + y; // DOES NOT COMPILE
8:   return reply;
9: }
```

English: The y variable is initialized to 10. By contrast, x is not initialized before it is used in the expression on line 7, and the compiler generates an error. The compiler is smart enough to recognize variables that have been initialized after their declaration but before they are used.

Türkçe: y variable'ı 10 ile initialize edilmiştir. Buna karşılık x, 7. satırdaki statement'ta kullanılmadan önce initialize edilmediği için compiler hata verir. Compiler, declaration'dan sonra fakat kullanımdan önce initialize edilen variable'ları tanıyabilir.

English: Here's an example:

Türkçe: İşte bir örnek:

```
public int valid() {
  int y = 10;
  int x; // x is declared here
  x = 3; // x is initialized here
  int z; // z is declared here but never initialized or used
  int reply = x + y;
  return reply;
}
```

English: In this example, x is declared, initialized, and used in separate lines. Also, z is declared but never used, so it is not required to be initialized.

Türkçe: Bu örnekte x ayrı satırlarda declare edilir, initialize edilir ve kullanılır. z de declare edilir; ancak hiç kullanılmadığı için initialize edilmesi gerekmez.

English: The compiler is also smart enough to recognize initializations that are more complex. In this example, there are two branches of code:

Türkçe: compiler ayrıca daha karmaşık başlatmaları tanıyacak kadar akıllıdır. Bu örnekte iki kod dalı vardır:

```
public void findAnswer(boolean check) {  
    int answer;  
    int otherAnswer;  
    int onlyOneBranch;  
    if (check) {  
        onlyOneBranch = 1;  
        answer = 1;  
    }
```

Kaynak PDF sayfası 40

```
    } else {  
        answer = 2;  
    }  
    System.out.println(answer);  
    System.out.println(onlyOneBranch); // DOES NOT COMPILE  
}
```

English: The answer variable is initialized in both branches of the if statement, so the compiler is perfectly happy. It knows that regardless of whether check is true or false, the value answer will be set to something before it is used. The otherAnswer variable is not initialized but never used, and the compiler is equally as happy. Remember, the compiler is only concerned if you try to use uninitialized local variables; it doesn't mind the ones you never use.

Türkçe: answer variable'ı if statement'ının iki branch'inde de initialize edildiğinden compiler açısından sorun yoktur. Compiler, check ister true ister false olsun answer kullanılmadan önce ona bir value atanacağını bilir. otherAnswer initialize edilmez ama hiç kullanılmadığı için bu da sorun oluşturmaz. Compiler yalnızca initialize edilmemiş local variable kullanılmaya çalışıldığında hata verir; hiç kullanılmayanlarla ilgilenmez.

English: The onlyOneBranch variable is initialized only if check happens to be true. The compiler knows there is the possibility for check to be false, resulting in uninitialized code, and gives a compiler error. You learn more about the if statement in Chapter 3, "Making Decisions."

Türkçe: onlyOneBranch variable'ı yalnızca check value'su true olursa initialize edilir. Compiler, checkin false olabileceğini ve bu durumda variable'ın initialize edilmeden kalacağını bilir; bu nedenle compiler error verir. if statement'ı Chapter 3 "Making Decisions" içinde ayrıntılandırılır.

English exam tip: On the exam, be wary of any local variable that is declared but not initialized in a single line. This is a common place on the exam that could result in a "Does not compile" answer. Be sure to check to make sure it's initialized before it's used on the exam.

Türkçe sınav ipucu: Sınavda declare edildiği satırda initialize edilmeyen local variable'lara dikkat edin; bunlar sıkça Does not compile sonucuna yol açar. Variable'ın kullanılmadan önce kesinlikle initialize edildiğini kontrol edin.

Passing Constructor and Method Parameters

Türkçe başlık: Constructor ve Method Parameter'larını Geçirmek

English: Variables passed to a constructor or method are called constructor parameters or method parameters, respectively. These parameters are like local variables that have been pre-initialized.

Türkçe: Constructor'a veya method'a geçirilen variable'lara sırasıyla constructor parameter ve method parameter denir. Bu parameter'lar, önceden initialize edilmiş local variable'lar gibidir.

English: The rules for initializing constructor and method parameters are the same, so we focus primarily on method parameters.

Türkçe: Constructor ve method parameter'larını initialize etme kuralları aynıdır; bu nedenle öncelikle method parameter'larına odaklanıyoruz.

English: In the previous example, check is a method parameter.

Türkçe: Önceki örnekte check bir method parametresidir.

```
public void findAnswer(boolean check) {}
```

English: Take a look at the following method checkAnswer() in the same class:

Türkçe: Aynı class'taki aşağıdaki `checkAnswer()` method'unu inceleyin:

```
public void checkAnswer() {  
    boolean value;  
    findAnswer(value); // DOES NOT COMPILE  
}
```

English: The call to findAnswer() does not compile because it tries to use a variable that is not initialized. While the caller of a method checkAnswer() needs to be concerned about the variable being initialized, once inside the method findAnswer(), we can assume the local variable has been initialized to some value.

Türkçe: `findAnswer()` çağırısı, initialize edilmemiş bir variable kullanmaya çalıştığı için derlenmez. `checkAnswer()` method'unu çağırarak kod, geçirilen variable'ın initialize edilmiş olmasını sağlamalıdır. Buna karşılık `findAnswer()` method'una girildiğinde parameter'ın bir value ile initialize edilmiş olduğu kabul edilebilir.

Kaynak PDF sayfası 41

Defining Instance and Class Variables

Türkçe başlık: Instance ve Class Variable'ları Tanımlama

English: Variables that are not local variables are defined either as instance variables or as class variables. An instance variable, often called a field, is a value defined within a specific instance of an object. Let's say we have a Person class with an instance variable name of type String.

Türkçe: Local olmayan variable'lar instance variable veya class variable olarak tanımlanır. Genellikle field olarak da anılan instance variable, belirli bir object instance'ına ait value'dur. Örneğin `String` type'ında `name` instance variable'ı bulunan bir `Person` class'ı düşünün.

English: Each instance of the class would have its own value for name, such as Elysia or Sarah.

Türkçe: Class'ın her instance'ı `Elysia` veya `Sarah` gibi kendi `name` value'suna sahip olur.

English: Two instances could have the same value for name, but changing the value for one does not modify the other.

Türkçe: İki instance aynı `name` value'suna sahip olabilir; ancak birinin value'sunu değiştirmek diğerini etkilemez.

English: On the other hand, a class variable is one that is defined on the class level and shared among all instances of the class. It can even be publicly accessible to classes outside the class and doesn't require an instance to use. In our previous Person example, a shared class variable could be used to represent the list of people at the zoo today. You can tell a variable is a class variable because it has the keyword `static` before it. You learn about this in Chapter 5. For now, just know that a variable is a class variable if it has the `static` keyword in its declaration.

Türkçe: Class variable ise class düzeyinde define edilir ve class'ın bütün instance'ları arasında paylaşılır. `public` ise başka class'lar tarafından da erişilebilir; kullanmak için instance gerekmez. Önceki `Person` örneğinde, o gün hayvanat bahçesinde bulunan kişilerin listesini ortak bir class variable temsil edebilir. Declaration'ın önündeki `static` keyword'ü bunun class variable olduğunu gösterir. Ayrıntılar Bölüm 5'te ele alınacaktır.

English: Instance and class variables do not require you to initialize them. As soon as you declare these variables, they are given a default value. The compiler doesn't know what value to use and so wants the simplest value it can give the type: null for an object, zero for the numeric types, and false for a boolean. You don't need to know the default value for char, but in case you are curious, it is `'\u0000'` (NUL).

Türkçe: Instance ve class variable'larını sizin initialize etmeniz gerekmez. Bu variable'lar declare edildiği anda default value alır: object reference için `null`, numeric type'lar için sıfır ve `boolean` için `false`. `char` için default value'yu bilmeniz gerekmez; merak ederseniz değeri `'\u0000'` (NUL) karakteridir.

Inferring the Type with var

Türkçe başlık: Var ile Tür Çıkarımı

English: You have the option of using the keyword `var` instead of the type when declaring local variables under certain conditions. To use this feature, you just type `var` instead of the primitive or reference type. Here's an example:

Türkçe: Belirli koşullarda local variable declare ederken explicit type yerine `var` keyword'ü kullanılabilir. Bunun için primitive veya reference type adı yerine `var` yazılır. Örnek:

```
public class Zoo {
    public void whatTypeAmI() {
        var name = "Hello";
        var size = 7;
    }
}
```

English: The formal name of this feature is local variable type inference. Let's take that apart. First comes local variable. This means just what it sounds like. You can only use this feature for local variables. The exam may try to trick you with code like this:

Türkçe: Bu özelliğin resmi adı yerel variable türü çıkarımıdır. Bunu ayıralım. İlk önce yerel variable gelir. Bu tam olarak kulağa nasıl geldiği anlamına geliyor. Bu özelliği yalnızca yerel variable'lar için kullanabilirsiniz. Sınav sizi şöyle bir kodla kandırmaya çalışabilir:

```
public class VarKeyword {  
    var tricky = "Hello"; // DOES NOT COMPILE  
}
```

Kaynak PDF sayfası 42

English: Wait a minute! We just learned the difference between instance and local variables. The variable tricky is an instance variable. Local variable type inference works with local variables and not instance variables.

Türkçe: Buradaki `tricky`, instance variable'dır. Local variable type inference yalnızca local variable'larla çalışır; instance variable'larda kullanılamaz.

Type Inference of var

Türkçe başlık: `var` Type Inference

English: Now that you understand the local variable part, it is time to go on to what type inference means. The good news is that this also means what it sounds like. When you type var, you are instructing the compiler to determine the type for you. The compiler looks at the code on the line of the declaration and uses it to infer the type. Take a look at this example:

Türkçe: Local variable bölümünü anladığınıza göre şimdi type inference'ın anlamına geçebiliriz. `var` yazdığınızda compiler'a type'ı sizin yerinize belirlemesini söylersiniz. Compiler declaration satırındaki kodu inceler ve type'ı buradan infer eder. Şu örneğe bakın:

```
7: public void reassignment() {  
8:     var number = 7;  
9:     number = 4;  
10:    number = "five"; // DOES NOT COMPILE  
11: }
```

English: On line 8, the compiler determines that we want an int variable. On line 9, we have no trouble assigning a different int to it. On line 10, Java has a problem. We've asked it to assign a String to an int variable. This is not allowed. It is equivalent to typing this:

Türkçe: 8. satırda compiler, `int` variable istediğimizi belirler. 9. satırda ona başka bir `int` atamak sorun değildir. 10. satırdaysa bir `int` variable'a `String` assign etmeye çalışırız; buna izin verilmez. Bu, şunu yazmaya eşdeğerdir:

```
int number = "five";
```

English memory tip: If you know a language like JavaScript, you might be expecting `var` to mean a variable that can take on any type at runtime. In Java, `var` is still a specific type defined at compile time. It does not change type at runtime.

Türkçe hafıza ipucu: JavaScript gibi bir dil biliyorsanız `var` ile declare edilen variable'ın runtime'da herhangi bir type alabileceğini düşünebilirsiniz. Java'da ise `var`, compile time'da belirlenen specific bir type'a sahiptir ve bu type runtime'da değişmez.

English: For simplicity when discussing `var`, we are going to assume a variable declaration statement is completed in a single line. You could insert a line break between the variable name and its initialization value, as in the following example:

Türkçe: Var'ı tartışırken basitlik sağlamak için, bir variable bildirim ifadesinin tek bir satırda tamamlandığını varsayacağız. Aşağıdaki örnekte olduğu gibi, variable adı ile başlangıç değeri arasına bir satır sonu ekleyebilirsiniz:

```
7: public void breakingDeclaration() {  
8:   var silly  
9:   = 1;  
10: }
```

English: This example is valid and does compile, but we consider the declaration and initialization of `silly` to be happening on the same line.

Türkçe: Bu örnek geçerlidir ve derlenir; `silly` variable'ının declaration ve initialization işlemlerini tek bir statement olarak kabul ederiz.

Examples with var

Türkçe başlık: `var` ile Örnekler

English: Let's go through some more scenarios so the exam doesn't trick you on this topic! Do you think the following compiles?

Türkçe: Sınavın bu konuda sizi yanıltmaması için birkaç senaryo daha inceleyelim. Sizce aşağıdaki kod derlenir mi?

```
3: public void doesThisCompile(boolean check) {  
4:   var question;  
5:   question = 1;  
6:   var answer;
```

Kaynak PDF sayfası 43

```
7: if (check) {  
8:   answer = 2;  
9: } else {  
10:  answer = 3;  
11: }  
12: System.out.println(answer);  
13: }
```

English: The code does not compile. Remember that for local variable type inference, the compiler looks only at the line with the declaration. Since question and answer are not assigned values on the lines where they are defined, the compiler does not know what to make of them. For this reason, both lines 4 and 6 do not compile.

Türkçe: Kod derlenmiyor. Yerel variable türü çıkarımı için compiler'ın yalnızca bildirimin bulunduğu satıra baktığını unutmayın. Soru ve cevaba tanımlandıkları satırlarda değer atanmadığından compiler bunları ne yapacağını bilemez. Bu nedenle 4. ve 6. satırların ikisi de derlenmiyor.

English: You might find that strange since both branches of the if/else do assign a value. Alas, it is not on the same line as the declaration, so it does not count for var. Contrast this behavior with what we saw a short while ago when we discussed branching and initializing a local variable in our findAnswer() method.

Türkçe: if/else yapısının iki branch'i de value atadığı için bu sonuç şaşırtıcı gelebilir. Ancak initialization, declaration statement'ının parçası değildir; dolayısıyla var için yeterli değildir. Bu davranışı, az önce findAnswer() örneğinde branch'ler üzerinden local variable initialization'ını tartıştığımız durumla karşılaştırın.

English: Now we know the initial value used to determine the type needs to be part of the same statement. Can you figure out why these two statements don't compile?

Türkçe: Artık türü belirlemek için kullanılan başlangıç değerinin aynı ifadenin parçası olması gerektiğini biliyoruz. Bu iki ifadenin neden derlenmediğini anlayabiliyor musunuz?

```
4: public void twoTypes() {
5:   int a, var b = 3; // DOES NOT COMPILE
6:   var n = null; // DOES NOT COMPILE
7: }
```

English: Line 5 wouldn't work even if you replaced var with a real type. All the types declared on a single line must be the same type and share the same declaration. We couldn't write int a, int v = 3; either.

Türkçe: varı gerçek bir type ile değiştirseniz bile 5. satır çalışmaz. Tek satırda declare edilen bütün variable'lar aynı type'ı ve aynı declaration'ı paylaşmalıdır. int a, int v = 3; de yazamazdık.

English: Line 6 is a single line. The compiler is being asked to infer the type of null. This could be any reference type. The only choice the compiler could make is Object. However, that is almost certainly not what the author of the code intended. The designers of Java decided it would be better not to allow var for null than to have to guess at intent.

Türkçe: 6. satır tek satırdır. Compiler'dan null'ın type'ını infer etmesi istenir. null herhangi bir reference type'a ait olabilir. Compiler'ın seçebileceği tek type Object'tir; ancak kodun yazarının amaçladığı büyük olasılıkla bu değildir. Java tasarımcıları, niyeti tahmin etmek yerine var'ın null ile initialize edilmesine izin vermemeyi seçmiştir.

English memory tip: While a var cannot be initialized with a null value without a type, it can be reassigned a null value after it is declared, provided that the underlying data type is a reference type.

Türkçe hafıza ipucu: Bir var, explicit type olmadan null value ile initialize edilemez; ancak underlying data type bir reference type ise declaration'dan sonra ona yeniden null atanabilir.

English: Let's try another example. Do you see why this does not compile?

Türkçe: Başka bir örnek deneyelim. Bunun neden derlenmediğini anlıyor musunuz?

```
public int addition(var a, var b) { // DOES NOT COMPILE
    return a + b;
}
```

English: In this example, `a` and `b` are method parameters. These are not local variables. Be on the lookout for `var` used with constructors, method parameters, or instance variables. Using `var` in one of these places is a good exam trick to see if you are paying attention. Remember that `var` is only used for local variable type inference!

Türkçe: Bu örnekte `a` ve `b` method parameter'larıdır; local variable değildir. Constructor parameter'ında, method parameter'ında veya instance variable declaration'ında kullanılan `var`a dikkat edin. Bunlar yaygın sınav tuzaklarıdır. `var` yalnızca local variable type inference için kullanılabilir.

Kaynak PDF sayfası 44

English: There's one last rule you should be aware of: `var` is not a reserved word and allowed to be used as an identifier. It is considered a reserved type name. A reserved type name means it cannot be used to define a type, such as a class, interface, or enum. Do you think this is legal?

Türkçe: Son bir kural daha vardır: `var` bir reserved word değildir ve identifier olarak kullanılabilir. Bunun yerine reserved type name kabul edilir; yani class, interface veya enum gibi bir type tanımlamak için kullanılamaz. Sizce aşağıdaki kullanım geçerli midir?

```
package var;
public class Var {
    public void var() {
        var var = "var";
    }
    public void Var() {
        Var var = new Var();
    }
}
```

English: Believe it or not, this code does compile. Java is case sensitive, so `Var` doesn't introduce any conflicts as a class name. Naming a local variable `var` is legal. Please don't write code that looks like this at your job! But understanding why it works will help get you ready for any tricky exam questions the exam creators could throw at you.

Türkçe: İster inanın ister inanmayın, bu kod derleniyor. Java büyük/küçük harfe duyarlı olduğundan `Var`, class adı olarak herhangi bir çakışmaya neden olmaz. Yerel bir variable'a `var` adını vermek yasaldır. Lütfen iş yerinizde buna benzer kodlar yazmayın! Ancak bunun neden işe yaradığını anlamak, sınavı hazırlayanların size yöneltebileceği zorlu sınav sorularına hazırlanmanıza yardımcı olacaktır.

var in the Real World

Türkçe başlık: Gerçek Dünyada `var`

English: The `var` keyword is great for exam authors because it makes it easier to write tricky code.

Türkçe: `var` keyword'ü, karmaşık kod yazmayı kolaylaştırdığı için sınav yazarları açısından çok kullanışlıdır.

English: When you work on a real project, you want the code to be easy to read.

Türkçe: Gerçek bir proje üzerinde çalışırken kodun okunması kolay olmasını istersiniz.

English: Once you start having code that looks like the following, it is time to consider using `var`:

Türkçe: Aşağıdaki gibi görünen bir koda sahip olmaya başladığınızda, `var` kullanmayı düşünmenin zamanı gelmiştir:

```
PileOfPapersToFileInFilingCabinet pileOfPapersToFile =
new PileOfPapersToFileInFilingCabinet();
```

English: You can see how shortening this would be an improvement without losing any information:

Türkçe: Hiçbir bilgiyi kaybetmeden bunun kısaltılmasının ne kadar bir gelişme olacağını görebilirsiniz:

```
var pileOfPapersToFile = new PileOfPapersToFileInFilingCabinet();
```

English: If you are ever unsure whether it is appropriate to use `var`, we recommend “Local Variable Type Inference: Style Guidelines,” which is available at the following location.

Türkçe: `var` kullanımının uygun olup olmadığından emin değilseniz “Local Variable Type Inference: Style Guidelines” belgesine bakmanızı öneririz.

English: <https://openjdk.java.net/projects/amber/LVTIstyle.html>

Türkçe: <https://openjdk.java.net/projects/amber/LVTIstyle.html>

Kaynak PDF sayfası 45

Managing Variable Scope

Türkçe başlık: Variable Scope'unu Yönetme

English: You've learned that local variables are declared within a code block. How many variables do you see that are scoped to this method?

Türkçe: Yerel variable'ların bir kod bloğu içerisinde bildirildiğini öğrendiniz. Bu method'un kapsamına giren kaç variable görüyorsunuz?

```
public void eat(int piecesOfCheese) {
    int bitesOfCheese = 1;
}
```

English: There are two variables with local scope. The `bitesOfCheese` variable is declared inside the method. The `piecesOfCheese` variable is a method parameter. Neither variable can be used outside of where it is defined.

Türkçe: Local scope'a sahip iki variable vardır. `bitesOfCheese` method içinde declare edilir; `piecesOfCheese` ise bir method parameter'dır. Hiçbiri tanımlandığı scope'un dışında kullanılamaz.

Limiting Scope

Türkçe başlık: Kapsamın Sınırlandırılması

English: Local variables can never have a scope larger than the method they are defined in. However, they can have a smaller scope. Consider this example:

Türkçe: Local variable'ların scope'u hiçbir zaman define edildikleri method'dan geniş olamaz; ancak daha dar olabilir. Şu örneği inceleyin:

```
3: public void eatIfHungry(boolean hungry) {  
4:   if (hungry) {  
5:     int bitesOfCheese = 1;  
6:   } // bitesOfCheese goes out of scope here  
7:   System.out.println(bitesOfCheese); // DOES NOT COMPILE  
8: }
```

English: The variable `hungry` has a scope of the entire method, while the variable `bitesOfCheese` has a smaller scope. It is only available for use in the `if` statement because it is declared inside of it. When you see a set of braces (`{}`) in the code, it means you have entered a new block of code. Each block of code has its own scope. When there are multiple blocks, you match them from the inside out. In our case, the `if` statement block begins at line 4 and ends at line 6. The method's block begins at line 3 and ends at line 8.

Türkçe: `hungry` variable'ının scope'u method'un tamamını kapsarken `bitesOfCheese` daha dar bir scope'a sahiptir. `bitesOfCheese`, `if` statement'ının içinde declare edildiği için yalnızca orada kullanılabilir. Kodda braces (`{}`), yani süslü parantez gördüğünüzde yeni bir code block'a girmiş olursunuz. Her code block'un kendi scope'u vardır. Birden fazla block varsa bunlar içten dışa eşleştirilir. Burada `if` block'u 4–6. satırları, method block'u ise 3–8. satırları kapsar.

English: Since `bitesOfCheese` is declared in an `if` statement block, the scope is limited to that block. When the compiler gets to line 7, it complains that it doesn't know anything about this `bitesOfCheese` thing and gives an error.

Türkçe: `bitesOfCheese`, `if` block'unda declare edildiği için scope'u bu block ile sınırlıdır. Compiler 7. satıra ulaştığında `bitesOfCheese` artık scope dışında olduğundan hata verir.

English: Remember that blocks can contain other blocks. These smaller contained blocks can reference variables defined in the larger scoped blocks, but not vice versa. Here's an example:

Türkçe: Block'ların başka block'lar içerebileceğini unutmayın. İçteki küçük block'lar, daha geniş scope'a sahip dış block'larda tanımlanan variable'lara erişebilir; tersi mümkün değildir. Örnek:

```

16: public void eatIfHungry(boolean hungry) {
17: if (hungry) {
18: int bitesOfCheese = 1;
19: {
20: var teenyBit = true;
21: System.out.println(bitesOfCheese);

```

Kaynak PDF sayfası 46

```

22: }
23: }
24: System.out.println(teenyBit); // DOES NOT COMPILE
25: }

```

English: The variable defined on line 18 is in scope until the block ends on line 23. Using it in the smaller block from lines 19 to 22 is fine. The variable defined on line 20 goes out of scope on line 22. Using it on line 24 is not allowed.

Türkçe: 18. satırda tanımlanan variable, blok 23. satırda bitene kadar kapsam dahilindedir. Bunu 19. satırdan 22. satıra kadar olan daha küçük blokta kullanmak uygundur. 20. satırda tanımlanan variable 22. satırda kapsam dışına çıkıyor. 24. satırda kullanılmasına izin verilmiyor.

Tracing Scope

Türkçe başlık: İzleme Kapsamı

English: The exam will attempt to trick you with various questions on scope. You'll probably see a question that appears to be about something complex and fails to compile because one of the variables is out of scope.

Türkçe: Sınavda scope ile ilgili çeşitli tuzaklar bulunur. Karmaşık bir konuyu ölçüyor gibi görünen bir kod, variable'lardan biri scope dışında olduğu için derlenmeyebilir.

English: Let's try one. Don't worry if you aren't familiar with if statements or while loops yet. It doesn't matter what the code does since we are talking about scope. See if you can figure out on which line each of the five local variables goes into and out of scope:

Türkçe: Bir tane deneyelim. Henüz if ifadelerine veya while döngülerine aşina değilseniz endişelenmeyin. Kapsamdan bahsettiğimiz için kodun ne yaptığı önemli değil. Bakalım beş yerel variable'ın her birinin hangi satırda kapsam içine girip çıkacağını bulabilecek misiniz:

```

11: public void eatMore(boolean hungry, int amountOfFood) {
12: int roomInBelly = 5;
13: if (hungry) {
14: var timeToEat = true;
15: while (amountOfFood > 0) {
16: int amountEaten = 2;
17: roomInBelly = roomInBelly - amountEaten;
18: amountOfFood = amountOfFood - amountEaten;
19: }
20: }
21: System.out.println(amountOfFood);
22: }

```

English: This method does compile. The first step in figuring out the scope is to identify the blocks of code. In this case, there are three blocks. You can tell this because there are three sets of braces. Starting from the innermost set, we can see where the while loop's block starts and ends. Repeat this process as we go on for the if statement block and method block.

Türkçe: Bu method derlenir. Scope belirlemenin ilk adımı code block'ları saptamaktır. Burada üç braces (`{ }`), yani süslü parantez çifti bulunduğu için üç block vardır. En içteki çiftten başlayarak `while` loop block'unun başlangıç ve bitişi buluruz; ardından aynı işlemi `if` block'u ve method block'u için tekrarlarız.

English: Table 1.10 shows the line numbers that each block starts and ends on.

Türkçe: Tablo 1.10 her bloğun başladığı ve bittiği satır numaralarını göstermektedir.

English table caption: TABLE 1.10 Tracking scope by block

Türkçe tablo başlığı: TABLO 1.10 Blok bazında izleme kapsamı

Block / Blok	First line in block / İlk satır	Last line in block / Son satır
<code>while</code>	15	19
<code>if</code>	13	20
Method	11	22

Kaynak PDF sayfası 47

English: Now that we know where the blocks are, we can look at the scope of each variable.

Türkçe: Artık blokların nerede olduğunu bildiğimize göre her variable'ın kapsamına bakabiliriz.

English: `hungry` and `amountOfFood` are method parameters, so they are available for the entire method. This means their scope is lines 11 to 22. The variable `roomInBelly` goes into scope on line 12 because that is where it is declared. It stays in scope for the rest of the method and goes out of scope on line 22. The variable `timeToEat` goes into scope on line 14 where it is declared. It goes out of scope on line 20 where the if block ends. Finally, the variable `amountEaten` goes into scope on line 16 where it is declared. It goes out of scope on line 19 where the while block ends.

Türkçe: `hungry` ve `amountOfFood` method parameter'ları olduğundan method'un tamamında kullanılabilir; scope'ları 11–22. satırlardır. `roomInBelly`, declare edildiği 12. satırda scope'a girer, method'un kalanı boyunca scope'ta kalır ve 22. satırda scope'tan çıkar. `timeToEat`, declare edildiği 14. satırda scope'a girer ve `if` block'unun bittiği 20. satırda scope'tan çıkar. Son olarak `amountEaten`, declare edildiği 16. satırda scope'a girer ve `while` block'unun bittiği 19. satırda scope'tan çıkar.

English: You'll want to practice this skill a lot! Identifying blocks and variable scope needs to be second nature for the exam. The good news is that there are lots of code examples to practice on. You can look at any code example on any topic in this book and match up braces.

Türkçe: Bu beceriyi bolca çalışmalısınız. Block'ları ve variable scope'unu belirlemek sınavda otomatikleşmiş bir beceri olmalıdır. Neyse ki kitapta pratik yapabileceğiniz çok sayıda code example vardır; herhangi bir örnekte braces'i, yani süslü parantezleri eşleştirebilirsiniz.

Applying Scope to Classes

Türkçe başlık: Scope'u Class'lara Uygulama

English: All of that was for local variables. Luckily, the rule for instance variables is easier: they are available as soon as they are defined and last for the entire lifetime of the object itself. The rule for class, aka static, variables is even easier: they go into scope when declared like the other variable types.

However, they stay in scope for the entire life of the program.

Türkçe: Şimdiye kadarki kurallar local variable'lar içindi. Instance variable'larda kural daha basittir: declare edildikleri andan object'in lifetime'ı sona erene kadar kullanılabilirler. Static variable olarak da adlandırılan class variable'lar ise declaration anında scope'a girer ve programın tüm lifetime'ı boyunca scope'ta kalır.

English: Let's do one more example to make sure you have a handle on this. Again, try to figure out the type of the four variables and when they go into and out of scope.

Türkçe: Bu konuya hakim olduğunuzdan emin olmak için bir örnek daha yapalım. Yine dört variable'ın türünü ve ne zaman kapsam dışına çıkıp çıktıklarını anlamaya çalışın.

```
1: public class Mouse {
2:   final static int MAX_LENGTH = 5;
3:   int length;
4:   public void grow(int inches) {
5:     if (length < MAX_LENGTH) {
6:       int newSize = length + inches;
7:       length = newSize;
8:     }
9:   }
10: }
```

English: In this class, we have one class variable, MAX_LENGTH; one instance variable, length; and two local variables, inches and newSize. The MAX_LENGTH variable is a class variable because it has the static keyword in its declaration. In this case, MAX_LENGTH goes into scope on line 2 where it is declared. It stays in scope until the program ends.

Türkçe: Bu class'ta MAX_LENGTH adlı bir class variable, length adlı bir instance variable ve inches ile newSize adlı iki local variable vardır. MAX_LENGTH, declaration'ında static keyword'ü bulunduğu için class variable'dır. Declare edildiği 2. satırda scope'a girer ve program sona erene kadar scope'ta kalır.

English: Next, length goes into scope on line 3 where it is declared. It stays in scope as long as this Mouse object exists. inches goes into scope where it is declared on line 4. It goes out of scope at the end of the method on line 9. newSize goes into scope where it is declared on line 6. Since it is defined inside the if statement block, it goes out of scope when that block ends on line 8.

Türkçe: Ardından length, declare edildiği 3. satırda scope'a girer ve bu Mouse object'i var olduğu sürece scope'ta kalır. inches, declare edildiği 4. satırda scope'a girer ve 9. satırda method sona erdiğinde scope'tan çıkar. newSize, declare edildiği 6. satırda scope'a girer; if block'u içinde define edildiği için block'un bittiği 8. satırda scope'tan çıkar.

Kaynak PDF sayfası 48

Reviewing Scope

Türkçe başlık: Kapsamın İncelenmesi

English: Got all that? Let's review the rules on scope:

Türkçe: Bunların hepsini anladın mı? Kapsamla ilgili kuralları gözden geçirelim:

English: - Local variables: In scope from declaration to the end of the block

Türkçe: - Local variable'lar: Declaration'dan block'un sonuna kadar scope'tadır.

English: - Method parameters: In scope for the duration of the method

Türkçe: - Method parameter'ları: Method boyunca scope'tadır.

English: - Instance variables: In scope from declaration until the object is eligible for garbage collection

Türkçe: - Instance variable'lar: Declaration'dan object garbage collection için eligible olana kadar scope'tadır.

English: - Class variables: In scope from declaration until the program ends

Türkçe: - Class variable'lar: Declaration'dan program sona erene kadar scope'tadır.

English: Not sure what garbage collection is? Relax: that's our next and final section for this chapter.

Türkçe: Garbage collection'ın ne olduğundan emin değilseniz endişelenmeyin; sıradaki son bölüm bunu açıklıyor.

Destroying Objects

Türkçe başlık: Object'leri Yok Etmek

English: Now that we've played with our objects, it is time to put them away. Luckily, the JVM takes care of that for you. Java provides a garbage collector to automatically look for objects that aren't needed anymore.

Türkçe: Object'lerle işimiz bittiğinde onları kaldırma görevini JVM üstlenir. Java, artık gerekmeyen object'leri otomatik olarak belirleyen bir garbage collector sağlar.

English: Remember, your code isn't the only process running in your Java program. Java code exists inside of a JVM, which includes numerous processes independent from your application code. One of the most important of those is a built-in garbage collector.

Türkçe: Java programınızda çalışan tek işlemin kodunuz olmadığını unutmayın. Java kodu, uygulama kodunuzdan bağımsız çok sayıda işlemi içeren JVM'nin içinde bulunur. Bunlardan en önemlilerinden biri yerleşik çöp toplayıcıdır.

English: All Java objects are stored in your program memory's heap. The heap, which is also referred to as the free store, represents a large pool of unused memory allocated to your Java application. If your program keeps instantiating objects and leaving them on the heap, eventually it will run out of memory and crash. Oh, no! Luckily, garbage collection solves this problem. In the following sections, we look at garbage collection.

Türkçe: Bütün Java object'leri program belleğinin heap bölümünde tutulur. Free store olarak da adlandırılan heap, Java application'ına ayrılan büyük bir kullanılabilir bellek havuzudur. Program object oluşturmaya devam edip onları heap'te bırakırsa sonunda bellek tükenir ve program çöker. Garbage collection bu sorunu önler. Aşağıdaki bölümlerde bu süreci inceleyeceğiz.

Understanding Garbage Collection

Türkçe başlık: Çöp Toplama İşlemini Anlamak

English: Garbage collection refers to the process of automatically freeing memory on the heap by deleting objects that are no longer reachable in your program. There are many different algorithms for garbage collection, but you don't need to know any of them for the exam.

Türkçe: Garbage collection, programın artık erişemediği object'leri kaldırarak heap belleğinin otomatik biçimde geri kazanılmasıdır. Birçok farklı garbage collection algorithm'i vardır; ancak sınav için bunların hiçbirini bilmeniz gerekmez.

English: As a developer, the most interesting part of garbage collection is determining when the memory belonging to an object can be reclaimed. In Java and other languages, eligible for garbage collection refers to an object's state of no longer being accessible in a program and therefore able to be garbage collected.

Türkçe: Bir geliştirici olarak garbage collection işleminin en ilginç kısmı, bir object'e ait belleğin ne zaman geri alınabileceğinin belirlenmesidir. Java ve diğer dillerde, çöp toplamaya uygun olma durumu, bir object'in artık bir programda erişilebilir olmaması ve dolayısıyla çöp toplanabilme durumu anlamına gelir.

English: Does this mean an object that's eligible for garbage collection will be immediately garbage collected? Definitely not. When the object actually is discarded is not under your control, but for the exam, you will need to know at any given moment which objects are eligible for garbage collection.

Türkçe: Bu, eligible olan bir object'in hemen garbage collection işlemine alınacağı anlamına gelmez. Object'in gerçekten ne zaman kaldırılacağı sizin kontrolünüzde değildir; ancak sınavda belirli bir anda hangi object'lerin eligible olduğunu belirlemeniz gerekir.

English: Think of garbage-collection eligibility like shipping a package. You can take an item, seal it in a labeled box, and put it in your mailbox. This is analogous to making an item eligible for garbage collection. When the mail carrier comes by to pick it up, though, is not in your

Türkçe: Garbage collection eligibility durumunu bir paketin gönderilmesine benzetin. Bir eşyayı etiketli kutuya koyup posta kutunuza bıraktığınızda onu alınmaya hazır hâle getirirsiniz; bu, object'in garbage collection için eligible olmasına benzer. Ancak postacının onu ne zaman alacağı sizin

Kaynak PDF sayfası 49

English: control. For example, it may be a postal holiday, or there could be a severe weather event.

Türkçe: kontrolünüzde değildir. Örneğin resmî tatil veya şiddetli hava koşulları nedeniyle teslim alma gecikebilir.

English: You can even call the post office and ask them to come pick it up right away, but there's no way to guarantee when and if this will actually happen. Hopefully, they will come by before your mailbox fills with packages!

Türkçe: Hatta postaneyi arayabilir ve hemen gelip almalarını isteyebilirsiniz, ancak bunun gerçekten ne zaman olacağını ve gerçekleşip gerçekleşmeyeceğini garanti etmenin bir yolu yoktur. Umarım posta kutunuz paketlerle dolmadan gelirler!

English: Java includes a built-in method to help support garbage collection where you can suggest that garbage collection run.

Türkçe: Java, garbage collection çalıştırılmasını önermek için çağrılabilen yerleşik bir method sağlar.

```
System.gc();
```

English: Just like the post office, Java is free to ignore you. This method is not guaranteed to do anything.

Türkçe: Tıpkı postane gibi, Java da sizi görmezden gelmekte özgürdür. Bu method'un hiçbir şey yapacağı garanti edilmez.

Tracing Eligibility

Türkçe başlık: Uygunluğun İzlenmesi

English: How does the JVM know when an object is eligible for garbage collection? The JVM waits patiently and monitors each object until it determines that the code no longer needs that memory. An object will remain on the heap until it is no longer reachable. An object is no longer reachable when one of two situations occurs:

Türkçe: JVM bir object'in çöp toplamaya uygun olduğunu nasıl biliyor? JVM sabırla bekler ve kodun artık o belleğe ihtiyaç duymadığını belirleyene kadar her object'i izler. Bir object artık erişilemeyece kadar yığında kalacaktır. İki durumdan biri meydana geldiğinde bir object'e artık ulaşılamaz:

English: - The object no longer has any references pointing to it.

Türkçe: - Object'e işaret eden hiçbir reference kalmamıştır.

English: - All references to the object have gone out of scope.

Türkçe: - Object'e yönelik bütün reference'lar scope dışına çıkmıştır.

Objects vs. References

Türkçe başlık: Object'ler ve Referanslar

English: Do not confuse a reference with the object that it refers to; they are two different entities.

Türkçe: Bir reference ile onun işaret ettiği object'i karıştırmayın; bunlar iki farklı varlıktır.

English: The reference is a variable that has a name and can be used to access the contents of an object. A reference can be assigned to another reference, passed to a method, or returned from a method. All references are the same size, no matter what their type is.

Türkçe: Reference, adı bulunan ve bir object'in içeriğine erişmek için kullanılan variable'dır. Bir reference başka bir reference'a atanabilir, method'a geçirilebilir veya method'dan döndürülebilir. Type'larından bağımsız olarak bütün reference'lar aynı boyuttadır.

English: An object sits on the heap and does not have a name. Therefore, you have no way to access an object except through a reference. Objects come in all different shapes and sizes and consume varying amounts of memory. An object cannot be assigned to another object, and an object cannot be passed to a method or returned from a method. It is the object that gets garbage collected, not its reference.

Türkçe: Object heap üzerinde bulunur ve kendi adı yoktur; bu nedenle ona yalnızca bir reference üzerinden erişilebilir. Object'lerin boyutları ve tükettikleri bellek miktarı değişebilir. Doğrudan bir object başka bir object'e atanmaz, method'a geçirilmez veya method'dan döndürülmez; bunları yapan reference'tır. Garbage collection'a alınan da reference değil object'tir.

English figure labels: The heap · A reference name · An object

Türkçe şekil etiketleri: Heap · Bir reference adı · Bir object

```
A reference name          The heap
                           +-----+
[name] -----> | [An object] |
                           +-----+
```

Kaynak PDF sayfası 50

English: Realizing the difference between a reference and an object goes a long way toward understanding garbage collection, the new operator, and many other facets of the Java language.

Türkçe: Reference ile object arasındaki farkı anlamak; garbage collection'ı, new operator'ını ve Java dilinin pek çok başka yönünü kavramayı büyük ölçüde kolaylaştırır.

English: Look at this code and see whether you can figure out when each object first becomes eligible for garbage collection:

Türkçe: Bu koda bakın ve her object'in ilk kez ne zaman çöp toplamaya uygun hale geldiğini anlayıp çözemeyeceğinizi görün:

```

1: public class Scope {
2: public static void main(String[] args) {
3: String one, two;
4: one = new String("a");
5: two = new String("b");
6: one = two;
7: String three = one;
8: one = null;
9: } }

```

English: When you are asked a question about garbage collection on the exam, we recommend that you draw what's going on. There's a lot to keep track of in your head, and it's easy to make a silly mistake trying to hold it all in your memory. Let's try it together now. Really.

Türkçe: Garbage collection sorularında reference'lar ile object'ler arasındaki bağlantıları çizmenizi öneririz. Zihinde aynı anda çok sayıda bağlantıyı izlemek kolayca hataya yol açabilir. Şimdi bunu birlikte deneyelim.

English: Get a pencil and paper. We'll wait.

Türkçe: Bir kalem ve kağıt alın. Bekleyeceğiz.

English: Got that paper? Okay, let's get started. On line 3, write one and two (just the words— no need for boxes or arrows since no objects have gone on the heap yet). On line 4, we have our first object. Draw a box with the string "a" in it, and draw an arrow from the word one to that box. Line 5 is similar. Draw another box with the string "b" in it this time and an arrow from the word two. At this point, your work should look like Figure 1.4.

Türkçe: 3. satır için *one* ve *two* adlarını yazın; henüz heap'te object olmadığı için kutu veya ok gerekmez. 4. satırda ilk object oluşturulur: içinde "*a*" bulunan bir kutu ve *one*dan bu kutuya uzanan bir ok çizin. 5. satırda içinde "*b*" bulunan ikinci kutuyu ve *two*dan ona uzanan oku ekleyin. Çiziminiz Şekil 1.4'e benzemelidir.

English figure caption: FIGURE 1.4 Your drawing after line 5

Türkçe şekil başlığı: ŞEKİL 1.4 5. satırdan sonraki çiziminiz

English: one "a" two "b" On line 6, the variable one changes to point to "b". Either erase or cross out the arrow from one and draw a new arrow from one to "b". On line 7, we have a new variable, so write the word three and draw an arrow from three to "b". Notice that three points to what one is pointing to right now and not what it was pointing to at the beginning. This is why you are drawing pictures. It's easy to forget something like that. At this point, your work should look like Figure 1.5.

Türkçe: Şekilde *one* → "*a*" ve *two* → "*b*" bağlantıları vardır. 6. satırda *one*, "*b*" object'ini gösterecek şekilde yeniden atanır; eski oku silip *onedan* "*b*"ye yeni bir ok çizin. 7. satırda *three* variable'ını ekleyip ondan da "*b*"ye ok çizin. *three*, *one*in başlangıçta gösterdiği object'i değil, o anda gösterdiği object'i gösterir. Çiziminiz Şekil 1.5'e benzemelidir.

English: Finally, cross out the line between one and "b" since line 8 sets this variable to null.

Türkçe: Son olarak 8. satır *one* variable'ına *null* atar; bu nedenle *one* ile "*b*" arasındaki oku silin.

English: Now, we were trying to find out when the objects were first eligible for garbage collection.

Türkçe: Şimdi object'lerin çöp toplamaya ilk kez ne zaman uygun olduğunu bulmaya çalışıyorduk.

English: On line 6, we got rid of the only arrow pointing to "a", making that object eligible for garbage collection. "b" has arrows pointing to it until it goes out of scope. This means "b" doesn't go out of scope until the end of the method on line 9.

Türkçe: 6. satırda "a"yı gösteren tek reference kaldırılır ve bu object garbage collection için eligible olur. "b" ise method sona erene kadar kendisini gösteren reference'lara sahiptir; dolayısıyla 9. satırdaki method sonuna kadar eligible olmaz.

Kaynak PDF sayfası 51

English figure caption: FIGURE 1.5 Your drawing after line 7

Türkçe şekil başlığı: ŞEKİL 1.5 7. satırdan sonraki çiziminiz

English: The figure shows *one*, *two*, and *three* all pointing to "b"; "a" has no incoming reference.

Türkçe: Şekilde *one*, *two* ve *three* reference'larının tümü "b" object'ini gösterir; "a" object'ine ulaşan reference kalmamıştır.

Code Formatting on the Exam

Türkçe başlık: Sınavda Kod Formatlama

English: Not all questions will include package declarations and imports. Don't worry about missing package statements or imports unless you are asked about them. The following are common cases where you don't need to check the imports:

Türkçe: Tüm sorular package bildirimlerini ve içe aktarmaları içermeyecektir. Size sorulmadıkça eksik package bildirimleri veya içe aktarmalar konusunda endişelenmeyin. Aşağıdakiler, içe aktarmaları kontrol etmenize gerek olmayan yaygın durumlardır:

English: - Code that begins with a class name

Türkçe: - class adıyla başlayan kod

English: - Code that begins with a method declaration

Türkçe: - Method declaration ile başlayan kod

English: - Code that begins with a code snippet that would normally be inside a class or method

Türkçe: - Normalde bir class veya method'un içinde yer alan bir kod parçacığıyla başlayan kod

English: - Code that has line numbers that don't begin with 1

Türkçe: - Satır numaraları 1'den başlamayan kod

English: You'll see code that doesn't have a method. When this happens, assume any necessary plumbing code like the `main()` method and class definition were written correctly. You're just being asked if the part of the code you're shown compiles when dropped into valid surrounding code. Finally, remember that extra whitespace doesn't matter in Java syntax. The exam may use varying amounts of whitespace to trick you.

Türkçe: Çevreleyen method'u gösterilmeyen code snippet'lerle karşılaşabilirsiniz. Bu durumda `main()` method'u ve class definition gibi gerekli plumbing code'un doğru yazıldığını varsayın. Yalnızca size gösterilen bölümün geçerli surrounding code içine yerleştirildiğinde derlenip derlenmediği sorulur. Ayrıca extra whitespace'in Java syntax'ında önemli olmadığını unutmayın; sınav sizi yanıltmak için farklı miktarlarda whitespace kullanabilir.

Summary

Türkçe başlık: Özet

English: Java begins program execution with a `main()` method. The most common signature for this method run from the command line is `public static void main(String[] args)`.

Türkçe: Java program execution'ını `main()` method'unda başlatır. Command line'dan çalıştırılan en yaygın signature `public static void main(String[] args)` biçimindedir.

English: Arguments are passed in after the class name, as in `java NameOfClass firstArgument`.

Türkçe: Argument'lar class adından sonra geçirilir: `java NameOfClass firstArgument`.

English: Arguments are indexed starting with 0.

Türkçe: Argümanlar 0'dan başlayarak indekslenir.

English: Java code is organized into folders called packages. To reference classes in other packages, you use an import statement. A wildcard ending an import statement means you want to import all classes in that package. It does not include packages that are inside that one. The package `java.lang` is special in that it does not need to be imported.

Türkçe: Java kodu package adı verilen klasörlerde düzenlenir. Başka package'lardaki class'lara başvurmak için `import statement` kullanılır. `import statement`ın sonundaki wildcard, o package'taki bütün class'ların içe aktarılmasını sağlar; ancak içindeki child package'ları kapsamaz. `java.lang` package'ı otomatik olarak içe aktarıldığı için özeldir.

Kaynak PDF sayfası 52

English: For some class elements, order matters within the file. The package statement comes first if present. Then come the import statements if present. Then comes the class declaration.

Türkçe: Bazı class member'ları için file içindeki sıra önemlidir. Varsa önce `package statement`, ardından `import statement`lar ve son olarak class declaration gelir.

English: Fields and methods are allowed to be in any order within the class.

Türkçe: Field'lar ve method'lar class içinde herhangi bir sırada bulunabilir.

English: Primitive types are the basic building blocks of Java types. They are assembled into reference types. Reference types can have methods and be assigned a null value. Numeric literals are allowed to contain underscores (_) as long as they do not start or end the literal and are not next to a decimal point (.). Wrapper classes are reference types, and there is one for each primitive. Text blocks allow creating a String on multiple lines using "" .

Türkçe: Primitive type'lar Java type'larının temel yapı taşlarıdır ve reference type'ları oluşturmak üzere bir araya getirilir. Reference type'ların method'ları olabilir ve kendilerine null atanabilir. Numeric literal'lar, başta veya sonda bulunmamak ve decimal point'in (.) hemen yanında yer almamak koşuluyla underscore (_) içerebilir. Wrapper class'lar reference type'tır ve her primitive için bir tane bulunur. Text block'lar "" kullanarak birden çok satırlı String oluşturmayı sağlar.

English: Declaring a variable involves stating the data type and giving the variable a name. Variables that represent fields in a class are automatically initialized to their corresponding 0, null, or false values during object instantiation. Local variables must be specifically initialized before they can be used. Identifiers may contain letters, numbers, currency symbols, or _. Identifiers may not begin with numbers. Local variable declarations may use the var keyword instead of the actual type. When using var, the type is set once at compile time and does not change.

Türkçe: Variable declaration, data type belirtmeyi ve variable'a ad vermeyi içerir. Class field'ları object instantiate edilirken type'larına göre otomatik olarak 0, null veya false ile initialize edilir. Local variable'lar ise kullanılmadan önce açıkça initialize edilmelidir. Identifier'lar harf, rakam, currency symbol veya _ içerebilir; ancak rakamla başlayamaz. Local variable declaration'larında explicit type yerine var kullanılabilir. var ile inferred type compile time'da bir kez belirlenir ve sonradan değişmez.

English: Scope refers to that portion of code where a variable can be accessed. There are three kinds of variables in Java, depending on their scope: instance variables, class variables, and local variables. Instance variables are the non-static fields of your class. Class variables are the static fields within a class. Local variables are declared within a constructor, method, or initializer block.

Türkçe: Scope, kodun bir variable'a erişilebildiği bölümüdür. Java'da scope'a göre üç variable türü vardır: instance variable, class variable ve local variable. Instance variable'lar class'ın non-static field'ları; class variable'lar class içindeki static field'lardır. Local variable'lar constructor, method veya initializer block içinde declare edilir.

English: Constructors create Java objects. A constructor is a method matching the class name and omitting the return type. When an object is instantiated, fields and blocks of code are initialized first. Then the constructor is run. Finally, garbage collection is responsible for removing objects from memory when they can never be used again. An object becomes eligible for garbage collection when there are no more references to it or its references have all gone out of scope.

Türkçe: Constructor'lar Java object'leri oluşturur. Constructor, class adıyla aynı ada sahip olan ve return type içermeyen özel bir yapıdır. Bir object instantiate edildiğinde önce field'lar ve code block'lar initialize edilir, ardından constructor çalışır. Son olarak garbage collection, artık hiç kullanılamayacak object'leri memory'den kaldırır. Object'e hiç reference kalmadığında veya bütün reference'lar scope dışına çıktığında object garbage collection için eligible olur.

Exam Essentials

Türkçe başlık: Sınav Esasları

English: Be able to write code using a `main()` method. A `main()` method is usually written as `public static void main(String[] args)`. Arguments are referenced starting with `args[0]`. Accessing an argument that wasn't passed in will cause the code to throw an exception.

Türkçe: `main()` method'unu kullanarak kod yazabilme. Bir `main()` method'u genellikle `public static void main(String[] args)` biçimindedir. Argument'lara `args[0]` ile başlayarak erişilir. Geçirilmemiş bir argument'a erişmek runtime'da exception fırlatılmasına neden olur.

English: Understand the effect of using packages and imports. Packages contain Java classes.

Türkçe: Package ve import kullanımının etkisini anlayın. Package'lar Java class'larını içerir.

English: Classes can be imported by class name or wildcard. Wildcards do not look at subdirectories. In the event of a conflict, class name imports take precedence. Package and import statements are optional. If they are present, they both go before the class declaration in that order.

Türkçe: Class'lar, class adı veya joker karakterle içe aktarılabilir. Joker karakterler alt dizinlere bakmaz. Bir çakışma durumunda class adının içe aktarılması önceliklidir. Paket ve içe aktarma ifadeleri isteğe bağlıdır. Eğer mevcutlarsa, ikisi de class bildiriminin önüne bu sırayla giderler.

English: Be able to recognize a constructor. A constructor has the same name as the class. It looks like a method without a return type.

Türkçe: Bir constructor'ı tanıyabilmelisiniz. Constructor, class ile aynı adı taşır ve return type'i bulunmayan bir method'a benzer görünür.

Kaynak PDF sayfası 53

English: Be able to identify legal and illegal declarations and initialization. Multiple variables can be declared and initialized in the same statement when they share a type. Local variables require an explicit initialization; others use the default value for that type. Identifiers may contain letters, numbers, currency symbols, or `_`, although they may not begin with numbers.

Türkçe: Yasal ve yasa dışı bildirimleri ve başlatmaları tanımlayabilme. Bir türü paylaştıklarında birden fazla variable aynı ifadede bildirilebilir ve başlatılabilir. Yerel variable'lar açık bir başlatma gerektirir; diğerleri bu tür için varsayılan değeri kullanır. Tanımlayıcılar harfler, sayılar, para birimi simgeleri veya `_` içerebilir, ancak sayılarla başlamayabilirler.

English: Also, you cannot define an identifier that is just a single underscore character `_`. Numeric literals may contain underscores between two digits, such as `1_000`, but not in other places, such as `_100_0_`.

Türkçe: Ayrıca, yalnızca tek bir alt çizgi karakterinden (`_`) oluşan bir tanımlayıcı tanımlanamazsınız. Sayısal sabit değerler, `1_000` gibi iki rakam arasında alt çizgi içerebilir, ancak `_100_0_` gibi diğer yerlerde alt çizgi içeremez.

English: Understand how to create text blocks. A text block begins with `"""` on the first line. On the next line begins the content. The last line ends with `"""`. If `"""` is on its own line, a trailing line break is included.

Türkçe: Metin bloklarının nasıl oluşturulacağını anlayın. Bir metin bloğu ilk satırında `"""` ile başlar. Sonraki satırda içerik başlar ve son satır `"""` ile biter. `"""` kendi satırındaysa sondaki satır sonu eklenir.

English: Be able to use `var` correctly. A `var` is used for a local variable. A `var` is initialized on the same line where it is declared, and while it can change value, it cannot change type. A `var` cannot be initialized with a null value without a type, nor can it be used in multiple variable declarations.

Türkçe: `var`ı doğru kullanabilmek. `var`, local variable için kullanılır. Declare edildiği satırda initialize edilir; value'su değişebilse de type'ı değişemez. Explicit type olmadan `null` ile initialize edilemez ve multiple variable declaration'da kullanılamaz.

English: Be able to determine where variables go into and out of scope. All variables go into scope when they are declared. Local variables go out of scope when the block they are declared in ends. Instance variables go out of scope when the object is eligible for garbage collection.

Türkçe: Variable'ların ne zaman scope'a girip çıktığını belirleyebilmelisiniz. Bütün variable'lar declare edildiklerinde scope'a girer. Local variable'lar declare edildikleri block bittiğinde; instance variable'lar ise object garbage collection için eligible olduğunda scope'tan çıkar.

English: Class variables remain in scope as long as the program is running.

Türkçe: Program çalıştığı sürece class variable'lar scope'ta kalır.

English: Know how to identify when an object is eligible for garbage collection. Draw a diagram to keep track of references and objects as you trace the code. When no arrows point to a box (object), it is eligible for garbage collection.

Türkçe: Bir object'in garbage collection için eligible olup olmadığını belirlemeyi öğrenin. Kodu izlerken reference ve object'leri bir diyagramla takip edin. Bir kutuya, yani object'e hiçbir ok işaret etmiyorsa object garbage collection için eligible'dır.

Review Questions

Türkçe başlık: İnceleme Soruları

English: The answers to the chapter review questions can be found in the Appendix.

Türkçe: Bölüm inceleme sorularının yanıtlarını Ek'te bulabilirsiniz.

Question 1 / Soru 1

English: 1. Which of the following are legal entry point methods that can be run from the command line? (Choose all that apply.)

Türkçe: 1. Aşağıdakilerden hangileri command line'dan çalıştırılabilen geçerli entry point method'larıdır? (Uygun olanların tümünü seçin.)

```
A. private static void main(String[] args)
B. public static final main(String[] args)
C. public void main(String[] args)
D. public static final void main(String[] args)
E. public static void main(String[] args)
F. public static main(String[] args)
```

Question 2 / Soru 2

English: 2. Which answer options represent the order in which the following statements can be assembled into a program that will compile successfully? (Choose all that apply.)

Türkçe: 2. Aşağıdaki statement'ların başarıyla derlenen bir program oluşturacak sırasını hangi seçenekler gösterir? (Uygun olanların tümünü seçin.)

```
X: class Rabbit {}
Y: import java.util.*;
Z: package animals;
```

```
A. X, Y, Z
B. Y, Z, X
C. Z, Y, X
D. Y, X
E. Z, X
F. X, Z
```

English: G. None of the above

Türkçe: G. Yukarıdakilerin hiçbiri.

Question 3 / Soru 3

English: 3. Which of the following are true? (Choose all that apply.)

Türkçe: 3. Aşağıdakilerden hangileri doğrudur? (Uygun olanların tümünü seçin.)

```
public class Bunny {
    public static void main(String[] x) {
        Bunny bun = new Bunny();
    } }
```

English: A. Bunny is a class.

Türkçe: A. **Bunny** bir class'tır.

English: B. bun is a class.

Türkçe: B. **bun** bir class'tır.

English: C. main is a class.

Türkçe: C. **main** bir class'tır.

English: D. Bunny is a reference to an object.
Türkçe: D. **Bunny**, bir object'e yönelik reference'tır.

English: E. bun is a reference to an object.
Türkçe: E. **bun**, bir object'e yönelik reference'tır.

English: F. main is a reference to an object.
Türkçe: F. **main**, bir object'e yönelik reference'tır.

English: G. The main() method doesn't run because the parameter name is incorrect.
Türkçe: G. Parameter adı yanlış olduğu için **main()** method'u çalışmaz.

Question 4 / Soru 4

English: 4. Which of the following are valid Java identifiers? (Choose all that apply.)
Türkçe: 4. Aşağıdakilerden hangileri geçerli Java identifier'larıdır? (Uygun olanların tümünü seçin.)

A. _
B. _helloWorld\$
C. true
D. java.lang
E. Public
F. 1980_s
G. _Q2_

Question 5 / Soru 5

English: 5. Which statements about the following program are correct? (Choose all that apply.)
Türkçe: 5. Aşağıdaki programla ilgili hangi statement'lar doğrudur? (Uygun olanların tümünü seçin.)

```
2: public class Bear {  
3:     private Bear pandaBear;  
4:     private void roar(Bear b) {  
5:         System.out.println("Roar!");  
6:         pandaBear = b;  
7:     }  
8:     public static void main(String[] args) {  
9:         Bear brownBear = new Bear();  
10:        Bear polarBear = new Bear();  
11:        brownBear.roar(polarBear);  
12:        polarBear = null;  
13:        brownBear = null;  
14:        System.gc(); } }
```

English: A. The object created on line 9 is eligible for garbage collection after line 13.
Türkçe: A. 9. satırda oluşturulan object, 13. satırdan sonra garbage collection için eligible olur.

English: B. The object created on line 9 is eligible for garbage collection after line 14.
Türkçe: B. 9. satırda oluşturulan object, 14. satırdan sonra garbage collection için eligible olur.

English: C. The object created on line 10 is eligible for garbage collection after line 12.

Türkçe: C. 10. satırda oluşturulan object, 12. satırdan sonra garbage collection için eligible olur.

English: D. The object created on line 10 is eligible for garbage collection after line 13.

Türkçe: D. 10. satırda oluşturulan object, 13. satırdan sonra garbage collection için eligible olur.

English: E. Garbage collection is guaranteed to run.

Türkçe: E. Garbage collection'ın çalışması garantidir.

English: F. Garbage collection might or might not run.

Türkçe: F. Garbage collection çalışabilir de çalışmayabilir de.

English: G. The code does not compile.

Türkçe: G. Kod derlenmez.

Question 6 / Soru 6

English: 6. Assuming the following class compiles, how many variables defined in the class or method are in scope on the line marked on line 14?

Türkçe: 6. Aşağıdaki class'ın derlendiğini varsayarsak 14. satırda class veya method içinde declare edilmiş kaç variable scope'tadır?

```
1: public class Camel {
2:     { int hairs = 3_000_0; }
3:     long water, air=2;
4:     boolean twoHumps = true;
5:     public void spit(float distance) {
6:         var path = "";
7:         { double teeth = 32 + distance++; }
8:         while(water > 0) {
9:             int age = twoHumps ? 1 : 2;
10:            short i=-1;
11:            for(i=0; i<10; i++) {
12:                var Private = 2;
13:            }
14:            // SCOPE
15:        }
16:    }
17: }
```

- A. 2
- B. 3
- C. 4
- D. 5
- E. 6
- F. 7

English: G. None of the above

Türkçe: G. Yukarıdakilerin hiçbiri.

Question 7 / Soru 7

English: 7. Which are true about this code? (Choose all that apply.)

Türkçe: 7. Bu kodla ilgili hangileri doğrudur? (Uygun olanların tümünü seçin.)

```
public class KitchenSink {  
    private int numForks;  
  
    public static void main(String[] args) {  
        int numKnives;  
        System.out.print("""  
            "# forks = " + numForks +  
            " # knives = " + numKnives +  
            # cups = 0""");  
    }  
}
```

English: A. The output includes: # forks = 0.

Türkçe: A. Output, # `forks` = 0 içerir.

English: B. The output includes: # knives = 0.

Türkçe: B. Output, # `knives` = 0 içerir.

English: C. The output includes: # cups = 0.

Türkçe: C. Output, # `cups` = 0 içerir.

English: D. The output includes a blank line.

Türkçe: D. Çıktı boş bir satır içeriyor.

English: E. The output includes one or more lines that begin with whitespace.

Türkçe: E. Çıktı, boşlukla başlayan bir veya daha fazla satır içeriyor.

English: F. The code does not compile.

Türkçe: F. Kod derlenmiyor.

Question 8 / Soru 8

English: 8. Which of the following code snippets about var compile without issue when used in a method? (Choose all that apply.)

Türkçe: 8. Bir method içinde kullanıldığında aşağıdaki `var` code snippet'lerinden hangileri sorunsuz derlenir? (Uygun olanların tümünü seçin.)

```
A. var spring = null;
B. var fall = "leaves";
C. var evening = 2; evening = null;
D. var night = Integer.valueOf(3);
E. var day = 1/0;
F. var winter = 12, cold;
G. var fall = 2, autumn = 2;
H. var morning = ""; morning = null;
```

Question 9 / Soru 9

English: 9. Which of the following are correct? (Choose all that apply.)

Türkçe: 9. Aşağıdakilerden hangileri doğrudur? (Uygun olanların tümünü seçin.)

English: A. An instance variable of type float defaults to 0.

Türkçe: A. `float` type'indeki instance variable'ın default value'su 0'dır.

English: B. An instance variable of type char defaults to null.

Türkçe: B. `char` type'indeki instance variable'ın default value'su `null`dir.

English: C. A local variable of type double defaults to 0.0.

Türkçe: C. `double` type'indeki local variable'ın default value'su 0.0'dır.

English: D. A local variable of type int defaults to null.

Türkçe: D. `int` type'indeki local variable'ın default value'su `null`dir.

English: E. A class variable of type String defaults to null.

Türkçe: E. `String` type'indeki class variable'ın default value'su `null`dir.

English: F. A class variable of type String defaults to the empty string "".

Türkçe: F. `String` type'indeki class variable'ın default value'su empty string ""'dır.

English: G. None of the above.

Türkçe: G. Yukarıdakilerin hiçbiri.

Question 10 / Soru 10

English: 10. Which of the following expressions, when inserted independently into the blank line, allow the code to compile? (Choose all that apply.)

Türkçe: 10. Aşağıdaki expression'lardan hangileri boş satıra ayrı ayrı eklendiğinde kodun derlenmesini sağlar? (Uygun olanların tümünü seçin.)

```
public void printMagicData() {
    var magic = _____;
    System.out.println(magic);
}
```

Editor note: Kaynaktaki boş alan, seçeneklerden biri = ile ; arasına yerleştirilecek biçimde korunmuştur.

- A. 3_1
- B. 1_329_.0
- C. 3_13.0_
- D. 5_291._2
- E. 2_234.0_0
- F. 9__6
- G. _1_3_5_0

Question 11 / Soru 11

English: 11. Given the following two class files, what is the maximum number of imports that can be removed and have the code still compile?

Türkçe: 11. Aşağıdaki iki class file'ı verildiğinde kod derlenmeye devam edecek biçimde en fazla kaç `import` kaldırılabilir?

```
// Water.java
package aquarium;
public class Water { }

// Tank.java
package aquarium;
import java.lang.*;
import java.lang.System;
import aquarium.Water;
import aquarium.*;
public class Tank {
    public void print(Water water) {
        System.out.println(water); } }
```

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4

English: F. Does not compile

Türkçe: F. Derlenmez.

Question 12 / Soru 12

English: 12. Which statements about the following class are correct? (Choose all that apply.)

Türkçe: 12. Aşağıdaki class ile ilgili hangi statement'lar doğrudur? (Uygun olanların tümünü seçin.)

```
1: public class ClownFish {  
2:     int gills = 0, double weight=2;  
3:     { int fins = gills; }  
4:     void print(int length = 3) {  
5:         System.out.println(gills);  
6:         System.out.println(weight);  
7:         System.out.println(fins);  
8:         System.out.println(length);  
9:     } }
```

English: A. Line 2 generates a compiler error.

Türkçe: A. 2. satır compiler error oluşturur.

English: B. Line 3 generates a compiler error.

Türkçe: B. 3. satır compiler error oluşturur.

English: C. Line 4 generates a compiler error.

Türkçe: C. 4. satır compiler error oluşturur.

English: D. Line 7 generates a compiler error.

Türkçe: D. 7. satır compiler error oluşturur.

English: E. The code prints 0.

Türkçe: E. Kod 0 yazdırır.

English: F. The code prints 2.0.

Türkçe: F. Kod 2.0 yazdırır.

English: G. The code prints 2.

Türkçe: G. Kod 2 yazdırır.

English: H. The code prints 3.

Türkçe: H. Kod 3 yazdırır.

Question 13 / Soru 13

English: 13. Given the following classes, which of the following snippets can independently be inserted in place of INSERT IMPORTS HERE and have the code compile? (Choose all that apply.)

Türkçe: 13. Aşağıdaki class'lar verildiğinde hangi snippet'ler birbirinden bağımsız olarak INSERT IMPORTS HERE yerine eklenip kodun derlenmesini sağlayabilir? (Uygun olanların tümünü seçin.)

```
package aquarium;  
public class Water {  
    boolean salty = false;  
}
```

```
package aquarium.jellies;
public class Water {
    boolean salty = true;
}
```

```
package employee;
INSERT IMPORTS HERE
public class WaterFiller {
    Water water;
}
```

```
A. import aquarium.*;
B. import aquarium.Water;
   import aquarium.jellies.*;
C. import aquarium.*;
   import aquarium.jellies.Water;
D. import aquarium.*;
   import aquarium.jellies.*;
E. import aquarium.Water;
   import aquarium.jellies.Water;
```

English: F. None of these imports can make the code compile.

Türkçe: F. Bu import'ların hiçbiri kodun derlenmesini sağlayamaz.

Question 14 / Soru 14

English: 14. Which of the following statements about the code snippet are true? (Choose all that apply.)

Türkçe: 14. Code snippet ile ilgili aşağıdaki statement'lerden hangileri doğrudur? (Uygun olanların tümünü seçin.)

```
3: short numPets = 5L;
4: int numGrains = 2.0;
5: String name = "Scruffy";
6: int d = numPets.length();
7: int e = numGrains.length();
8: int f = name.length();
```

English: A. Line 3 generates a compiler error.

Türkçe: A. 3. satır compiler error oluşturur.

English: B. Line 4 generates a compiler error.

Türkçe: B. 4. satır compiler error oluşturur.

English: C. Line 5 generates a compiler error.

Türkçe: C. 5. satır compiler error oluşturur.

English: D. Line 6 generates a compiler error.

Türkçe: D. 6. satır compiler error oluşturur.

English: E. Line 7 generates a compiler error.

Türkçe: E. 7. satır compiler error oluşturur.

English: F. Line 8 generates a compiler error.

Türkçe: F. 8. satır compiler error oluşturur.

Question 15 / Soru 15

English: 15. Which of the following statements about garbage collection are correct? (Choose all that apply.)

Türkçe: 15. Garbage collection ile ilgili aşağıdaki statement'lerden hangileri doğrudur? (Uygun olanların tümünü seçin.)

English: A. Calling `System.gc()` is guaranteed to free up memory by destroying objects eligible for garbage collection.

Türkçe: A. `System.gc()` çağrısının eligible object'leri kaldırarak belleği boşaltması garantidir.

English: B. Garbage collection runs on a set schedule.

Türkçe: B. Garbage collection sabit bir programa göre çalışır.

English: C. Garbage collection allows the JVM to reclaim memory for other objects.

Türkçe: C. Garbage collection, JVM'nin başka object'ler için belleği geri kazanmasını sağlar.

English: D. Garbage collection runs when your program has used up half the available memory.

Türkçe: D. Program kullanılabılır belleğin yarısını tükettiğinde garbage collection çalışır.

English: E. An object may be eligible for garbage collection but never removed from the heap.

Türkçe: E. Bir object garbage collection için eligible olduğu hâlde heap'ten hiç kaldırılmayabilir.

English: F. An object is eligible for garbage collection once no references to it are accessible in the program.

Türkçe: F. Programda kendisine erişebilen hiçbir reference kalmadığında object garbage collection için eligible olur.

English: G. Marking a variable `final` means its associated object will never be garbage collected.

Türkçe: G. Bir variable'ı `final` yapmak, ilişkili object'in hiçbir zaman garbage collection'a alınmayacağı anlamına gelir.

Question 16 / Soru 16

English: 16. Which are true about this code? (Choose all that apply.)

Türkçe: 16. Bu kodla ilgili hangileri doğrudur? (Uygun olanların tümünü seçin.)

```
var blocky = ""
    squirrel \s
    pigeon \
    termite"";
System.out.print(blocky);
```

English: A. It outputs two lines.

Türkçe: A. İki satır çıktı verir.

English: B. It outputs three lines.

Türkçe: B. Üç satır çıktı verir.

English: C. It outputs four lines.

Türkçe: C. Dört satır çıktı verir.

English: D. There is one line with trailing whitespace.

Türkçe: D. Sonunda boşluk bulunan bir satır vardır.

English: E. There are two lines with trailing whitespace.

Türkçe: E. Sonunda boşluk olan iki satır var.

English: F. If we indented each line five characters, it would change the output.

Türkçe: F. Her satıra beş karakter girinti yaparsak çıktı değişir.

Question 17 / Soru 17

English: 17. What lines are printed by the following program? (Choose all that apply.)

Türkçe: 17. Aşağıdaki program hangi satırları yazdırır? (Uygun olanların tümünü seçin.)

```
1: public class WaterBottle {
2:     private String brand;
3:     private boolean empty;
4:     public static float code;
5:     public static void main(String[] args) {
6:         WaterBottle wb = new WaterBottle();
7:         System.out.println("Empty = " + wb.empty);
8:         System.out.println("Brand = " + wb.brand);
9:         System.out.println("Code = " + code);
10:    }
```

English: A. Line 8 generates a compiler error.

Türkçe: A. 8. satır compiler error oluşturur.

English: B. Line 9 generates a compiler error.

Türkçe: B. 9. satır compiler error oluşturur.

English: `C. Empty =`

Türkçe: `C. Empty =`

English: `D. Empty = false`

Türkçe: `D. Empty = false`

English: `E. Brand =`

Türkçe: `E. Brand =`

English: `F. Brand = null`

Türkçe: `F. Brand = null`

English: `G. Code = 0.0`

Türkçe: `G. Code = 0.0`

English: `H. Code = 0f`

Türkçe: `H. Code = 0f`

Question 18 / Soru 18

English: 18. Which of the following statements about var are true? (Choose all that apply.)

Türkçe: 18. `var` hakkında aşağıdaki statement'lerden hangileri doğrudur? (Uygun olanların tümünü seçin.)

English: A. A var can be used as a constructor parameter.

Türkçe: A. `var`, constructor parameter olarak kullanılabilir.

English: B. The type of a var is known at compile time.

Türkçe: B. `var` ile declare edilen variable'ın type'ı compile time'da bilinir.

English: C. A var cannot be used as an instance variable.

Türkçe: C. `var`, instance variable declaration'ında kullanılamaz.

English: D. A var can be used in a multiple variable assignment statement.

Türkçe: D. `var`, multiple-variable assignment statement içinde kullanılabilir.

English: E. The value of a var cannot change at runtime.

Türkçe: E. `var` ile declare edilen variable'ın value'su runtime'da değiştirilemez.

English: F. The type of a var cannot change at runtime.

Türkçe: F. `var` ile declare edilen variable'ın type'ı runtime'da değiştirilemez.

English: G. The word var is a reserved word in Java.

Türkçe: G. `var` sözcüğü Java'da reserved word'dür.

Question 19 / Soru 19

English: 19. Which are true about the following code? (Choose all that apply.)

Türkçe: 19. Aşağıdaki kodla ilgili hangileri doğrudur? (Uygun olanların tümünü seçin.)

```
var num1 = Long.parseLong("100");  
var num2 = Long.valueOf("100");  
System.out.println(Long.max(num1, num2));
```

English: A. The output is 100.

Türkçe: A. Output `100` dür.

English: B. The output is 200.

Türkçe: B. Output `200` dür.

English: C. The code does not compile.

Türkçe: C. Kod derlenmez.

English: D. num1 is a primitive.

Türkçe: D. `num1` bir primitive'dir.

English: E. num2 is a primitive.

Türkçe: E. `num2` bir primitive'dir.

Question 20 / Soru 20

English: 20. Which statements about the following class are correct? (Choose all that apply.)

Türkçe: 20. Aşağıdaki class ile ilgili hangi statement'lar doğrudur? (Uygun olanların tümünü seçin.)

```
1: public class PoliceBox {
2:     String color;
3:     long age;
4:     public void PoliceBox() {
5:         color = "blue";
6:         age = 1200;
7:     }
8:     public static void main(String []time) {
9:         var p = new PoliceBox();
10:        var q = new PoliceBox();
11:        p.color = "green";
12:        p.age = 1400;
13:        p = q;
14:        System.out.println("Q1="+q.color);
15:        System.out.println("Q2="+q.age);
16:        System.out.println("P1="+p.color);
17:        System.out.println("P2="+p.age);
18:    } }
```

English: A. It prints **Q1=blue**.

Türkçe: A. **Q1=blue** yazdırır.

English: B. It prints **Q2=1200**.

Türkçe: B. **Q2=1200** yazdırır.

English: C. It prints **P1=null**.

Türkçe: C. **P1=null** yazdırır.

English: D. It prints **P2=1400**.

Türkçe: D. **P2=1400** yazdırır.

English: E. Line 4 does not compile.

Türkçe: E. 4. satır derlenmez.

English: F. Line 12 does not compile.

Türkçe: F. 12. satır derlenmez.

English: G. Line 13 does not compile.

Türkçe: G. 13. satır derlenmez.

English: H. None of the above.

Türkçe: H. Yukarıdakilerin hiçbiri.

Question 21 / Soru 21

English: 21. What is the output of executing the following class?

Türkçe: 21. Aşağıdaki class çalıştırıldığında çıktı ne olur?

```
1: public class Salmon {
2:     int count;
3:     { System.out.print(count+"-"); }
4:     { count++; }
5:     public Salmon() {
6:         count = 4;
7:         System.out.print(2+"-");
8:     }
9:     public static void main(String[] args) {
10:        System.out.print(7+"-");
11:        var s = new Salmon();
12:        System.out.print(s.count+"-"); } }
```

English: A. 7-0-2-1-

Türkçe: A. 7-0-2-1-

English: B. 7-0-1-

Türkçe: B. 7-0-1-

English: C. 0-7-2-1-

Türkçe: C. 0-7-2-1-

English: D. 7-0-2-4-

Türkçe: D. 7-0-2-4-

English: E. 0-7-1-

Türkçe: E. 0-7-1-

English: F. The class does not compile because of line 3.

Türkçe: F. Class, 3. satır nedeniyle derlenmez.

English: G. The class does not compile because of line 4.

Türkçe: G. Class, 4. satır nedeniyle derlenmez.

English: H. None of the above.

Türkçe: H. Yukarıdakilerin hiçbiri.

Question 22 / Soru 22

English: 22. Given the following class, which of the following lines of code can independently replace INSERT CODE HERE to make the code compile? (Choose all that apply.)

Türkçe: 22. Aşağıdaki class verildiğinde hangi code line'lar birbirinden bağımsız olarak INSERT CODE HERE yerine geçerek kodun derlenmesini sağlar? (Uygun olanların tümünü seçin.)

```
public class Price {  
    public void admission() {  
        INSERT CODE HERE  
        System.out.print(amount);  
    }  
}
```

A. `int Amount = 0b11;`
B. `int amount = 9L;`
C. `int amount = 0xE;`
D. `int amount = 1_2.0;`
E. `double amount = 1_0_.0;`
F. `int amount = 0b101;`
G. `double amount = 9_2.1_2;`
H. `double amount = 1_2_.0_0;`

Question 23 / Soru 23

English: 23. Which statements about the following class are true? (Choose all that apply.)

Türkçe: 23. Aşağıdaki class hakkında hangi statement'lar doğrudur? (Uygun olanların tümünü seçin.)

```
1: public class River {  
2:     int Depth = 1;  
3:     float temp = 50.0;  
4:     public void flow() {  
5:         for (int i = 0; i < 1; i++) {  
6:             int depth = 2;  
7:             depth++;  
8:             temp--;  
9:         }  
10:        System.out.println(depth);  
11:        System.out.println(temp); }  
12:     public static void main(String... s) {  
13:         new River().flow();  
14:     } }
```

English: A. Line 3 generates a compiler error.

Türkçe: A. 3. satır compiler error oluşturur.

English: B. Line 6 generates a compiler error.

Türkçe: B. 6. satır compiler error oluşturur.

English: C. Line 7 generates a compiler error.

Türkçe: C. 7. satır compiler error oluşturur.

English: D. Line 10 generates a compiler error.

Türkçe: D. 10. satır compiler error oluşturur.

English: E. The program prints 3 on line 10.

Türkçe: E. Program 10. satırda 3 yazdırır.

English: F. The program prints 4 on line 10.

Türkçe: F. Program 10. satırda 4 yazdırır.

English: G. The program prints 50.0 on line 11.

Türkçe: G. Program 11. satırda 50.0 yazdırır.

English: H. The program prints 49.0 on line 11.

Türkçe: H. Program 11. satırda 49.0 yazdırır.

Bölüm sonu aktif tekrar

Kaynak dışı özgün çalışma: Cevap vermeden önce her maddeyi `Does not compile` → runtime → output sırasıyla değerlendir.

- 1 JDK, JVM ve bytecode arasındaki ilişkiyi tek cümleyle açıkla.
- 2 Geçerli bir `main()` declaration'ının zorunlu parçalarını say.
- 3 İki wildcard import aynı simple class name'i getirirse ne olacağını belirt.
- 4 `var` kullanımında initializer ve scope bakımından iki sınır yaz.
- 5 Reference scope'u ile object'in garbage collection'a uygunluğu arasındaki farkı açıkla.

Ayrıntılı pekiştirme için [Unit 01 practice quiz](#) belgesini notlara bakmadan çöz.